

Table of Contents

1. Breadth First Search
2. Depth First Search
3. Search For Connected Components
4. Bridge Searching
5. Bridge Searching Online
6. Cutpoints
7. Strongly Connected Components
8. Strong Orientation
9. Dijkstra
10. Dijkstra_sparse
11. Bellman_ford
12. 01_bfs
13. Desopo_pape
14. All Pair Shortest Path Floyd Warshall
15. Fixed_length_paths
16. Mst_prim
17. Mst_kruskal
18. Mst_kruskal_with_dsu
19. Second_best_mst
20. Kirchhoff Theorem
21. Pruefer_code
22. Finding Cycle
23. Finding Negative Cycle In Graph
24. Euler_path
25. Lca
26. Lca_binary_lifting
27. Lca_farachcoltonbender
28. Rmq_linear
29. Lca_tarjan
30. Edmonds_karp
31. Push Relabel
32. Push Relabel Faster
33. Dinic
34. Mpm
35. Flow_with_demands

- 36. Min_cost_flow
- 37. Assignment Problem Min Flow
- 38. Bipartite Check
- 39. Kuhn_maximum_bipartite_matching
- 40. Hungarian Algorithm
- 41. Topological Sort
- 42. Edge_vertex_connectivity
- 43. Tree_painting
- 44. 2SAT
- 45. Hld

Translated

From: e-maxx.ru

Breadth-first search

Breadth first search is one of the basic and essential searching algorithms on graphs.

As a result of how the algorithm works, the path found by breadth first search to any node is the shortest path to that node, i.e the path that contains the smallest number of edges in unweighted graphs.

The algorithm works in $O(n + m)$ time, where n is number of vertices and m is the number of edges.

Description of the algorithm

The algorithm takes as input an unweighted graph and the id of the source vertex s . The input graph can be directed or undirected, it does not matter to the algorithm.

The algorithm can be understood as a fire spreading on the graph: at the zeroth step only the source s is on fire. At each step, the fire burning at each vertex spreads to all of its neighbors. In one iteration of the algorithm, the "ring of fire" is expanded in width by one unit (hence the name of the algorithm).

More precisely, the algorithm can be stated as follows: Create a queue q which will contain the vertices to be processed and a Boolean array $used[]$ which indicates for each vertex, if it has been lit (or visited) or not.

Initially, push the source s to the queue and set $used[s] = true$, and for all other vertices v set $used[v] = false$. Then, loop until the queue is empty and in each iteration, pop a vertex from the front of the queue. Iterate through all the edges going out of this vertex and if some of these edges go to vertices that are not already lit, set them on fire and place them in the queue.

As a result, when the queue is empty, the "ring of fire" contains all vertices reachable from the source s , with each vertex reached in the shortest possible way. You can also calculate the lengths of the shortest paths (which just requires maintaining an array of path lengths $d[]$) as well as save information to restore all of these shortest paths (for this, it is necessary to maintain an array of "parents" $p[]$, which stores for each vertex the vertex from which we reached it).

Implementation

We write code for the described algorithm in C++ and Java.

C++

```
vector<vector<int>>> adj; // adjacency list representation
int n; // number of nodes
int s; // source vertex

queue<int> q;
vector<bool> used(n);
vector<int> d(n), p(n);

q.push(s);
used[s] = true;
p[s] = -1;
while (!q.empty()) {
    int v = q.front();
    q.pop();
    for (int u : adj[v]) {
        if (!used[u]) {
            used[u] = true;
            q.push(u);
            d[u] = d[v] + 1;
            p[u] = v;
        }
    }
}
```

```
    }
}
```

Java

```
ArrayList<ArrayList<Integer>> adj = new ArrayList<>(); // adjacency list representation

int n; // number of nodes
int s; // source vertex

LinkedList<Integer> q = new LinkedList<Integer>();
boolean used[] = new boolean[n];
int d[] = new int[n];
int p[] = new int[n];

q.push(s);
used[s] = true;
p[s] = -1;
while (!q.isEmpty()) {
    int v = q.pop();
    for (int u : adj.get(v)) {
        if (!used[u]) {
            used[u] = true;
            q.push(u);
            d[u] = d[v] + 1;
            p[u] = v;
        }
    }
}
```

If we have to restore and display the shortest path from the source to some vertex u , it can be done in the following manner:

C++

```
if (!used[u]) {
    cout << "No path!";
} else {
    vector<int> path;
    for (int v = u; v != -1; v = p[v])
        path.push_back(v);
    reverse(path.begin(), path.end());
    cout << "Path: ";
    for (int v : path)
        cout << v << " ";
}
```

Java

```
if (!used[u]) {
    System.out.println("No path!");
} else {
    ArrayList<Integer> path = new ArrayList<Integer>();
    for (int v = u; v != -1; v = p[v])
        path.add(v);
    Collections.reverse(path);
    for (int v : path)
        System.out.println(v);
}
```

Applications of BFS

- Find the shortest path from a source to other vertices in an unweighted graph.
- Find all connected components in an undirected graph in $O(n + m)$ time: To do this, we just run BFS starting from each vertex, except for vertices which have already been visited from previous runs. Thus, we perform normal BFS from each of the vertices, but do not reset the array `used[]` each and every time we get a new connected component, and the total running time will still be $O(n + m)$ (performing multiple BFS on the graph without zeroing the array `used[]` is called a series of breadth first searches).

- Finding a solution to a problem or a game with the least number of moves, if each state of the game can be represented by a vertex of the graph, and the transitions from one state to the other are the edges of the graph.
- Finding the shortest path in a graph with weights 0 or 1: This requires just a little modification to normal breadth-first search: Instead of maintaining array `used[]`, we will now check if the distance to vertex is shorter than current found distance, then if the current edge is of zero weight, we add it to the front of the queue else we add it to the back of the queue. This modification is explained in more detail in the article [0-1 BFS](#).
- Finding the shortest cycle in a directed unweighted graph: Start a breadth-first search from each vertex. As soon as we try to go from the current vertex back to the source vertex, we have found the shortest cycle containing the source vertex. At this point we can stop the BFS, and start a new BFS from the next vertex. From all such cycles (at most one from each BFS) choose the shortest.
- Find all the edges that lie on any shortest path between a given pair of vertices (a, b) . To do this, run two breadth first searches: one from a and one from b . Let $d_a[]$ be the array containing shortest distances obtained from the first BFS (from a) and $d_b[]$ be the array containing shortest distances obtained from the second BFS from b . Now for every edge (u, v) it is easy to check whether that edge lies on any shortest path between a and b : the criterion is the condition $d_a[u] + 1 + d_b[v] = d_a[b]$.
- Find all the vertices on any shortest path between a given pair of vertices (a, b) . To accomplish that, run two breadth first searches: one from a and one from b . Let $d_a[]$ be the array containing shortest distances obtained from the first BFS (from a) and $d_b[]$ be the array containing shortest distances obtained from the second BFS (from b). Now for each vertex it is easy to check whether it lies on any shortest path between a and b : the criterion is the condition $d_a[v] + d_b[v] = d_a[b]$.
- Find the shortest walk of even length from a source vertex s to a target vertex t in an unweighted graph: For this, we must construct an auxiliary graph, whose vertices are the state (v, c) , where v - the current node, $c = 0$ or $c = 1$ - the current parity. Any edge (u, v) of the original graph in this new column will turn into two edges $((u, 0), (v, 1))$ and $((u, 1), (v, 0))$. After that we run a BFS to find the shortest walk from the starting vertex $(s, 0)$ to the end vertex $(t, 0)$.
Note: This item uses the term "walk" rather than a "path" for a reason, as the vertices may potentially repeat in the found walk in order to make its length even. The problem of finding the shortest *path* of even length is NP-Complete in directed graphs, and [solvable in linear time](#) in undirected graphs, but with a much more involved approach.

Practice Problems

- [SPOJ: AKBAR](#)
- [SPOJ: NAKANJ](#)
- [SPOJ: WATER](#)
- [SPOJ: MICE AND MAZE](#)
- [Timus: Caravans](#)
- [DevSkill - Halloween Party \(archived\)](#)
- [DevSkill - Ohani And The Link Cut Tree \(archived\)](#)
- [SPOJ - Spiky Mazes](#)
- [SPOJ - Four Chips \(hard\)](#)
- [SPOJ - Inversion Sort](#)
- [Codeforces - Shortest Path](#)
- [SPOJ - Yet Another Multiple Problem](#)
- [UVA 11392 - Binary 3xType Multiple](#)
- [UVA 10968 - KuPellaKeS](#)
- [Codeforces - Police Stations](#)
- [Codeforces - Okabe and City](#)
- [SPOJ - Find the Treasure](#)
- [Codeforces - Bear and Forgotten Tree 2](#)
- [Codeforces - Cycle in Maze](#)
- [UVA - 11312 - Flipping Frustration](#)

- [SPOJ - Ada and Cycle](#)
- [CSES - Labyrinth](#)
- [CSES - Message Route](#)
- [CSES - Monsters](#)

Contributors:

[Param Singh](#) (29.73%) [Jakob Kogler](#) (28.65%) [David Zhang](#) (23.78%) [Morass](#) (7.57%) [Oleksandr Kulkov](#) (3.78%) [Bhuvnesh Jain](#) (2.16%) [Aryaman](#) (1.08%)
[PAVAN BANSAL](#) (1.08%) [Animesh Kumar](#) (0.54%) [Bemwa Malak](#) (0.54%) [Danilo Pereira da Silva](#) (0.54%) [Mariia Mykhailova](#) (0.54%)

Translated

From: e-maxx.ru

Depth First Search

Depth First Search is one of the main graph algorithms.

Depth First Search finds the lexicographical first path in the graph from a source vertex u to each vertex. Depth First Search will also find the shortest paths in a tree (because there only exists one simple path), but on general graphs this is not the case.

The algorithm works in $O(m + n)$ time where n is the number of vertices and m is the number of edges.

Description of the algorithm

The idea behind DFS is to go as deep into the graph as possible, and backtrack once you are at a vertex without any unvisited adjacent vertices.

It is very easy to describe / implement the algorithm recursively: We start the search at one vertex. After visiting a vertex, we further perform a DFS for each adjacent vertex that we haven't visited before. This way we visit all vertices that are reachable from the starting vertex.

For more details check out the implementation.

Applications of Depth First Search

- Find any path in the graph from source vertex u to all vertices.
- Find lexicographical first path in the graph from source u to all vertices.
- Check if a vertex in a tree is an ancestor of some other vertex:

At the beginning and end of each search call we remember the entry and exit "time" of each vertex. Now you can find the answer for any pair of vertices (i, j) in $O(1)$: vertex i is an ancestor of vertex j if and only if $\text{entry}[i] < \text{entry}[j]$ and $\text{exit}[i] > \text{exit}[j]$.

- Find the lowest common ancestor (LCA) of two vertices.
- Topological sorting:

Run a series of depth first searches so as to visit each vertex exactly once in $O(n + m)$ time. The required topological ordering will be the vertices sorted in descending order of exit time.

- Check whether a given graph is acyclic and find cycles in a graph. (As mentioned below by counting back edges in every connected components).
- Find strongly connected components in a directed graph:

First do a topological sorting of the graph. Then transpose the graph and run another series of depth first searches in the order defined by the topological sort. For each DFS call the component created by it is a strongly connected component.

- Find bridges in an undirected graph:

First convert the given graph into a directed graph by running a series of depth first searches and making each edge directed as we go through it, in the direction we went. Second, find the strongly connected components in this directed graph. Bridges are the edges whose ends belong to different strongly connected components.

Classification of edges of a graph

We can classify the edges of a graph, G , using the entry and exit time of the end nodes u and v of the edges (u, v) . These classifications are often used for problems like [finding bridges](#) and [finding articulation points](#).

We perform a DFS and classify the encountered edges using the following rules:

If v is not visited:

- Tree Edge - If v is visited after u then edge (u, v) is called a tree edge. In other words, if v is visited for the first time and u is currently being visited then (u, v) is called tree edge. These edges form a DFS tree and hence the name tree edges.

If v is visited before u :

- **Back edges** - If v is an ancestor of u , then the edge (u, v) is a back edge. v is an ancestor exactly if we already entered v , but not exited it yet. Back edges complete a cycle as there is a path from ancestor v to descendant u (in the recursion of DFS) and an edge from descendant u to ancestor v (back edge), thus a cycle is formed. Cycles can be detected using back edges.
- **Forward Edges** - If v is a descendant of u , then edge (u, v) is a forward edge. In other words, if we already visited and exited v and $\text{entry}[u] < \text{entry}[v]$ then the edge (u, v) forms a forward edge.
- **Cross Edges**: if v is neither an ancestor or descendant of u , then edge (u, v) is a cross edge. In other words, if we already visited and exited v and $\text{entry}[u] > \text{entry}[v]$ then (u, v) is a cross edge.

Theorem. Let G be an undirected graph. Then, performing a DFS upon G will classify every encountered edge as either a tree edge or back edge, i.e., forward and cross edges only exist in directed graphs.

Suppose (u, v) is an arbitrary edge of G and without loss of generality, u is visited before v , i.e., $\text{entry}[u] < \text{entry}[v]$. Because the DFS only processes edges once, there are only two ways in which we can process the edge (u, v) and thus classify it:

- The first time we explore the edge (u, v) is in the direction from u to v . Because $\text{entry}[u] < \text{entry}[v]$, the recursive nature of the DFS means that node v will be fully explored and thus exited before we can "move back up the call stack" to exit node u . Thus, node v must be unvisited when the DFS first explores the edge (u, v) from u to v because otherwise the search would have explored (u, v) from v to u before exiting node v , as nodes u and v are neighbors. Therefore, edge (u, v) is a tree edge.
- The first time we explore the edge (u, v) is in the direction from v to u . Because we discovered node u before discovering node v , and we only process edges once, the only way that we could explore the edge (u, v) in the direction from v to u is if there's another path from u to v that does not involve the edge (u, v) , thus making u an ancestor of v . The edge (u, v) thus completes a cycle as it is going from the descendant, v , to the ancestor, u , which we have not exited yet. Therefore, edge (u, v) is a back edge.

Since there are only two ways to process the edge (u, v) , with the two cases and their resulting classifications outlined above, performing a DFS upon G will therefore classify every encountered edge as either a tree edge or back edge, i.e., forward and cross edges only exist in directed graphs. This completes the proof.

Implementation

```
vector<vector<int>> adj; // graph represented as an adjacency list
int n; // number of vertices

vector<bool> visited;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u])
            dfs(u);
    }
}
```

This is the most simple implementation of Depth First Search. As described in the applications it might be useful to also compute the entry and exit times and vertex color. We will color all vertices with the color 0, if we haven't visited them, with the color 1 if we visited them, and with the color 2, if we already exited the vertex.

Here is a generic implementation that additionally computes those:

```
vector<vector<int>> adj; // graph represented as an adjacency list
int n; // number of vertices

vector<int> color;

vector<int> time_in, time_out;
int dfs_timer = 0;

void dfs(int v) {
    time_in[v] = dfs_timer++;
    color[v] = 1;
    for (int u : adj[v])
        if (color[u] == 0)
```

```
    dfs(u);  
    color[v] = 2;  
    time_out[v] = dfs_timer++;  
}
```

Practice Problems

- [SPOJ: ABCPATH](#)
- [SPOJ: EAGLE1](#)
- [Codeforces: Kefa and Park](#)
- [Timus:Werewolf](#)
- [Timus:Penguin Avia](#)
- [Timus:Two Teams](#)
- [SPOJ - Ada and Island](#)
- [UVA 657 - The die is cast](#)
- [SPOJ - Sheep](#)
- [SPOJ - Path of the Righteous Man](#)
- [SPOJ - Validate the Maze](#)
- [SPOJ - Ghosts having Fun](#)
- [Codeforces - Underground Lab](#)
- [DevSkill - Maze Tester \(archived\)](#)
- [DevSkill - Tourist \(archived\)](#)
- [Codeforces - Anton and Tree](#)
- [Codeforces - Transformation: From A to B](#)
- [Codeforces - One Way Reform](#)
- [Codeforces - Centroids](#)
- [Codeforces - Generate a String](#)
- [Codeforces - Broken Tree](#)
- [Codeforces - Dasha and Puzzle](#)
- [Codeforces - Making genome In Berland](#)
- [Codeforces - Road Improvement](#)
- [Codeforces - Garland](#)
- [Codeforces - Labeling Cities](#)
- [Codeforces - Send the Fool Futher!](#)
- [Codeforces - The tag Game](#)
- [Codeforces - Leha and Another game about graphs](#)
- [Codeforces - Shortest path problem](#)
- [Codeforces - Upgrading Tree](#)
- [Codeforces - From Y to Y](#)
- [Codeforces - Chemistry in Berland](#)
- [Codeforces - Wizards Tour](#)
- [Codeforces - Ring Road](#)
- [Codeforces - Mail Stamps](#)
- [Codeforces - Ant on the Tree](#)
- [SPOJ - Cactus](#)
- [SPOJ - Mixing Chemicals](#)

Contributors:

[Jakob Kogler](#) (27.81%) [Param Singh](#) (20.12%) [Morass](#) (18.34%) [Madhur Chauhan](#) (13.61%) [Mariia Mykhailova](#) (6.51%) [Elliot Ha](#) (5.92%)
[Oleksandr Kulkov](#) (3.55%) [Bhuvnesh Jain](#) (2.37%) [Mayank Pratap Singh](#) (1.18%) [Zombiesalad1337](#) (0.59%)

Translated

From: e-maxx.ru

Search for connected components in a graph

Given an undirected graph G with n nodes and m edges. We are required to find in it all the connected components, i.e, several groups of vertices such that within a group each vertex can be reached from another and no path exists between different groups.

An algorithm for solving the problem

- To solve the problem, we can use Depth First Search or Breadth First Search.
- In fact, we will be doing a series of rounds of DFS: The first round will start from first node and all the nodes in the first connected component will be traversed (found). Then we find the first unvisited node of the remaining nodes, and run Depth First Search on it, thus finding a second connected component. And so on, until all the nodes are visited.
- The total asymptotic running time of this algorithm is $O(n + m)$: In fact, this algorithm will not run on the same vertex twice, which means that each edge will be seen exactly two times (at one end and at the other end).

Implementation

```
int n;
vector<vector<int>> adj;
vector<bool> used;
vector<int> comp;

void dfs(int v) {
    used[v] = true;
    comp.push_back(v);
    for (int u : adj[v]) {
        if (!used[u])
            dfs(u);
    }
}

void find_comps() {
    fill(used.begin(), used.end(), 0);
    for (int v = 0; v < n; ++v) {
        if (!used[v]) {
            comp.clear();
            dfs(v);
            cout << "Component:";
            for (int u : comp)
                cout << ' ' << u;
            cout << endl;
        }
    }
}
```

- The most important function that is used is `find_comps()` which finds and displays connected components of the graph.
- The graph is stored in adjacency list representation, i.e `adj[v]` contains a list of vertices that have edges from the vertex `v`.
- Vector `comp` contains a list of nodes in the current connected component.

Iterative implementation of the code

Deeply recursive functions are in general bad. Every single recursive call will require a little bit of memory in the stack, and per default programs only have a limited amount of stack space. So when you do a recursive DFS over a connected graph with millions of nodes, you might run into stack overflows.

It is always possible to translate a recursive program into an iterative program, by manually maintaining a stack data structure. Since this data structure is allocated on the heap, no stack overflow will occur.

```
int n;
vector<vector<int>> adj;
vector<bool> used;
vector<int> comp;

void dfs(int v) {
    stack<int> st;
    st.push(v);

    while (!st.empty()) {
        int curr = st.top();
        st.pop();
        if (!used[curr]) {
            used[curr] = true;
            comp.push_back(curr);
            for (int i = adj[curr].size() - 1; i >= 0; i--) {
                st.push(adj[curr][i]);
            }
        }
    }
}

void find_comps() {
    fill(used.begin(), used.end(), 0);
    for (int v = 0; v < n; ++v) {
        if (!used[v]) {
            comp.clear();
            dfs(v);
            cout << "Component:" ;
            for (int u : comp)
                cout << ' ' << u;
            cout << endl ;
        }
    }
}
```

Practice Problems

- [SPOJ: CT23E](#)
- [CODECHEF: GERALD07](#)
- [CSES : Building Roads](#)

Contributors:

[Jakob Kogler](#) (31.78%) [Pratik Rai](#) (26.17%) [saurabh0402](#) (25.23%) [Oleksandr Kulkov](#) (5.61%) [Gabriel Silva Simões](#) (3.74%) [Sunil Sangwan](#) (2.8%)
[Hoang Ho](#) (1.87%) [Aryaman](#) (0.93%) [Rodion Gorkovenko](#) (0.93%) [Vedant Borkar](#) (0.93%)

Translated From: e-maxx.ru

Finding bridges in a graph in $O(N + M)$

We are given an undirected graph. A bridge is defined as an edge which, when removed, makes the graph disconnected (or more precisely, increases the number of connected components in the graph). The task is to find all bridges in the given graph.

Informally, the problem is formulated as follows: given a map of cities connected with roads, find all "important" roads, i.e. roads which, when removed, cause disappearance of a path between some pair of cities.

The algorithm described here is based on [depth first search](#) and has $O(N + M)$ complexity, where N is the number of vertices and M is the number of edges in the graph.

Note that there is also the article [Finding Bridges Online](#) - unlike the offline algorithm described here, the online algorithm is able to maintain the list of all bridges in a changing graph (assuming that the only type of change is addition of new edges).

Algorithm

Pick an arbitrary vertex of the graph r_{root} and run [depth first search](#) from it. Note the following fact (which is easy to prove):

- Let's say we are in the DFS, looking through the edges starting from vertex v . The current edge (v, to) is a bridge if and only if none of the vertices to and its descendants in the DFS traversal tree has a back-edge to vertex v or any of its ancestors. Indeed, this condition means that there is no other way from v to to except for edge (v, to) .

Now we have to learn to check this fact for each vertex efficiently. We'll use "time of entry into node" computed by the depth first search.

So, let $tin[v]$ denote entry time for node v . We introduce an array low which will let us store the earliest entry time of the node found in the DFS search that a node v can reach with a single edge from itself or its descendants. $low[v]$ is the minimum of $tin[v]$, the entry times $tin[p]$ for each node p that is connected to node v via a back-edge (v, p) and the values of $low[to]$ for each vertex to which is a direct descendant of v in the DFS tree:

$$low[v] = \min \left\{ \begin{array}{l} tin[v] \\ tin[p] \quad \text{for all } p \text{ for which } (v, p) \text{ is a back edge} \\ low[to] \quad \text{for all } to \text{ for which } (v, to) \text{ is a tree edge} \end{array} \right\}$$

Now, there is a back edge from vertex v or one of its descendants to one of its ancestors if and only if vertex v has a child to for which $low[to] \leq tin[v]$. If $low[to] = tin[v]$, the back edge comes directly to v , otherwise it comes to one of the ancestors of v .

Thus, the current edge (v, to) in the DFS tree is a bridge if and only if $low[to] > tin[v]$.

Implementation

The implementation needs to distinguish three cases: when we go down the edge in DFS tree, when we find a back edge to an ancestor of the vertex and when we return to a parent of the vertex. These are the cases:

- `visited[to] = false` - the edge is part of DFS tree;
- `visited[to] = true && to != parent` - the edge is back edge to one of the ancestors;
- `to == parent` - the edge leads back to parent in DFS tree.

To implement this, we need a depth first search function which accepts the parent vertex of the current node.

For the cases of multiple edges, we need to be careful when ignoring the edge from the parent. To solve this issue, we can add a flag `parent_skipped` which will ensure we only skip the parent once.

```

void IS_BRIDGE(int v,int to); // some function to process the found bridge
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

Main function is `find_bridges`; it performs necessary initialization and starts depth first search in each connected component of the graph.

Function `IS_BRIDGE(a, b)` is some function that will process the fact that edge (a, b) is a bridge, for example, print it.

Note that this implementation malfunctions if the graph has multiple edges, since it ignores them. Of course, multiple edges will never be a part of the answer, so `IS_BRIDGE` can check additionally that the reported bridge is not a multiple edge. Alternatively it's possible to pass to `dfs` the index of the edge used to enter the vertex instead of the parent vertex (and store the indices of all vertices).

Practice Problems

- [UVA #796 "Critical Links"](#) [difficulty: low]
- [UVA #610 "Street Directions"](#) [difficulty: medium]
- [Case of the Computer Network \(Codeforces Round #310 Div. 1 E\)](#) [difficulty: hard]
- [UVA 12363 - Hedge Mazes](#)
- [UVA 315 - Network](#)
- [GYM - Computer Network \(J\)](#)
- [SPOJ - King Graffs Defense](#)
- [SPOJ - Critical Edges](#)
- [Codeforces - Break Up](#)
- [Codeforces - Tourist Reform](#)
- [Codeforces - Non-academic problem](#)

Contributors:

[Nalin Bhardwaj](#) (31.82%)
 [Jakob Kogler](#) (18.18%)
 [Mariia Mykhailova](#) (12.73%)
 [pramana](#) (10.91%)
 [NaimSS](#) (8.18%)
 [Morass](#) (5.45%)
 [wiktorkuchta](#) (5.45%)
[Oleksandr Kulkov](#) (2.73%)
 [Bhuvnesh Jain](#) (1.82%)
 [Nikhil Kumar Tomar](#) (0.91%)
 [Tarun Khullar](#) (0.91%)
 [Vatsal Sharma](#) (0.91%)

Translated From: e-maxx.ru

Finding Bridges Online

We are given an undirected graph. A bridge is an edge whose removal makes the graph disconnected (or, more precisely, increases the number of connected components). Our task is to find all the bridges in the given graph.

Informally this task can be put as follows: we have to find all the "important" roads on the given road map, i.e. such roads that the removal of any of them will lead to some cities being unreachable from others.

There is already the article [Finding Bridges in \$O\(N + M\)\$](#) which solves this task with a [Depth First Search](#) traversal. This algorithm will be much more complicated, but it has one big advantage: the algorithm described in this article works online, which means that the input graph doesn't have to be known in advance. The edges are added once at a time, and after each addition the algorithm recounts all the bridges in the current graph. In other words the algorithm is designed to work efficiently on a dynamic, changing graph.

More rigorously the statement of the problem is as follows: Initially the graph is empty and consists of n vertices. Then we receive pairs of vertices (a, b) , which denote an edge added to the graph. After each received edge, i.e. after adding each edge, output the current number of bridges in the graph.

It is also possible to maintain a list of all bridges as well as explicitly support the 2-edge-connected components.

The algorithm described below works in $O(n \log n + m)$ time, where m is the number of edges. The algorithm is based on the data structure [Disjoint Set Union](#). However the implementation in this article takes $O(n \log n + m \log n)$ time, because it uses the simplified version of the DSU without Union by Rank.

Algorithm

First let's define a k -edge-connected component: it is a connected component that remains connected whenever you remove fewer than k edges.

It is very easy to see, that the bridges partition the graph into 2-edge-connected components. If we compress each of those 2-edge-connected components into vertices and only leave the bridges as edges in the compressed graph, then we obtain an acyclic graph, i.e. a forest.

The algorithm described below maintains this forest explicitly as well as the 2-edge-connected components.

It is clear that initially, when the graph is empty, it contains n 2-edge-connected components, which by themselves are not connect.

When adding the next edge (a, b) there can occur three situations:

- Both vertices a and b are in the same 2-edge-connected component - then this edge is not a bridge, and does not change anything in the forest structure, so we can just skip this edge.
Thus, in this case the number of bridges does not change.
- The vertices a and b are in completely different connected components, i.e. each one is part of a different tree. In this case, the edge (a, b) becomes a new bridge, and these two trees are combined into one (and all the old bridges remain).
Thus, in this case the number of bridges increases by one.
- The vertices a and b are in one connected component, but in different 2-edge-connected components. In this case, this edge forms a cycle along with some of the old bridges. All these bridges end being bridges, and the resulting cycle must be compressed into a new 2-edge-connected component.
Thus, in this case the number of bridges decreases by one or more.

Consequently the whole task is reduced to the effective implementation of all these operations over the forest of 2-edge-connected components.

Data Structures for storing the forest

The only data structure that we need is [Disjoint Set Union](#). In fact we will make two copies of this structure: one will be to maintain the connected components, the other to maintain the 2-edge-connected components. And in addition we store the structure of the trees in the forest of 2-edge-connected components via pointers: Each 2-edge-connected component will store the index `par[]` of its ancestor in the tree.

We will now consistently disassemble every operation that we need to learn to implement:

- Check whether the two vertices lie in the same connected / 2-edge-connected component. It is done with the usual DSU algorithm, we just find and compare the representatives of the DSUs.
- Joining two trees for some edge (a, b) . Since it could turn out that neither the vertex a nor the vertex b are the roots of their trees, the only way to connect these two trees is to re-root one of them. For example you can re-root the tree of vertex a , and then attach it to another tree by setting the ancestor of a to b .

However the question about the effectiveness of the re-rooting operation arises: in order to re-root the tree with the root r to the vertex v , it is necessary to visit all vertices on the path between v and r and redirect the pointers `par[]` in the opposite direction, and also change the references to the ancestors in the DSU that is responsible for the connected components.

Thus, the cost of re-rooting is $O(h)$, where h is the height of the tree. You can make an even worse estimate by saying that the cost is $O(\text{size})$ where size is the number of vertices in the tree. The final complexity will not differ.

We now apply a standard technique: we re-root the tree that contains fewer vertices. Then it is intuitively clear that the worst case is when two trees of approximately equal sizes are combined, but then the result is a tree of twice the size. This does not allow this situation to happen many times.

In general the total cost can be written in the form of a recurrence:

$$T(n) = \max_{k=1 \dots n-1} \{T(k) + T(n-k) + O(\min(k, n-k))\}$$

$T(n)$ is the number of operations necessary to obtain a tree with n vertices by means of re-rooting and unifying trees. A tree of size n can be created by combining two smaller trees of size k and $n-k$. This recurrence has the solution $T(n) = O(n \log n)$.

Thus, the total time spent on all re-rooting operations will be $O(n \log n)$ if we always re-root the smaller of the two trees.

We will have to maintain the size of each connected component, but the data structure DSU makes this possible without difficulty.

- Searching for the cycle formed by adding a new edge (a, b) . Since a and b are already connected in the tree we need to find the [Lowest Common Ancestor](#) of the vertices a and b . The cycle will consist of the paths from b to the LCA, from the LCA to a and the edge a to b .

After finding the cycle we compress all vertices of the detected cycle into one vertex. This means that we already have a complexity proportional to the cycle length, which means that we also can use any LCA algorithm proportional to the length, and don't have to use any fast one.

Since all information about the structure of the tree is available is the ancestor array `par[]`, the only reasonable LCA algorithm is the following: mark the vertices a and b as visited, then we go to their ancestors `par[a]` and `par[b]` and mark them, then advance to their ancestors and so on, until we reach an already marked vertex. This vertex is the LCA that we are looking for, and we can find the vertices on the cycle by traversing the path from a and b to the LCA again.

It is obvious that the complexity of this algorithm is proportional to the length of the desired cycle.

- Compression of the cycle by adding a new edge (a, b) in a tree.

We need to create a new 2-edge-connected component, which will consist of all vertices of the detected cycle (also the detected cycle itself could consist of some 2-edge-connected components, but this does not change anything). In addition it is necessary to compress them in such a way that the structure of the tree is not disturbed, and all pointers `par[]` and two DSUs are still correct.

The easiest way to achieve this is to compress all the vertices of the cycle to their LCA. In fact the LCA is the highest of the vertices, i.e. its ancestor pointer `par[]` remains unchanged. For all the other vertices of the loop the ancestors do not need to be updated, since these vertices simply cease to exist. But in the DSU of the 2-edge-connected components all these vertices will simply point to the LCA.

We will implement the DSU of the 2-edge-connected components without the Union by rank optimization, therefore we will get the complexity $O(\log n)$ on average per query. To achieve the complexity $O(1)$ on average per query, we need to combine the vertices of the cycle according to Union by rank, and then assign `par[]` accordingly.

Implementation

Here is the final implementation of the whole algorithm.

As mentioned before, for the sake of simplicity the DSU of the 2-edge-connected components is written without Union by rank, therefore the resulting complexity will be $O(\log n)$ on average.

Also in this implementation the bridges themselves are not stored, only their count `bridges`. However it will not be difficult to create a `set` of all bridges.

Initially you call the function `init()`, which initializes the two DSUs (creating a separate set for each vertex, and setting the size equal to one), and sets the ancestors `par`.

The main function is `add_edge(a, b)`, which processes and adds a new edge.

```
vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
int bridges;
int lca_iteration;
vector<int> last_visit;

void init(int n) {
    par.resize(n);
    dsu_2ecc.resize(n);
    dsu_cc.resize(n);
    dsu_cc_size.resize(n);
    lca_iteration = 0;
    last_visit.assign(n, 0);
    for (int i=0; i<n; ++i) {
        dsu_2ecc[i] = i;
        dsu_cc[i] = i;
        dsu_cc_size[i] = 1;
        par[i] = -1;
    }
    bridges = 0;
}

int find_2ecc(int v) {
    if (v == -1)
        return -1;
    return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
}

int find_cc(int v) {
    v = find_2ecc(v);
    return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
}

void make_root(int v) {
    int root = v;
    int child = -1;
    while (v != -1) {
        int p = find_2ecc(par[v]);
        par[v] = child;
        dsu_cc[v] = root;
        child = v;
        v = p;
    }
    dsu_cc_size[root] = dsu_cc_size[child];
}

void merge_path (int a, int b) {
    ++lca_iteration;
    vector<int> path_a, path_b;
    int lca = -1;
    while (lca == -1) {
        if (a != -1) {
            a = find_2ecc(a);
            path_a.push_back(a);
            if (last_visit[a] == lca_iteration){
                lca = a;
                break;
            }
            last_visit[a] = lca_iteration;
            a = par[a];
        }
    }
}
```

```

    }
    if (b != -1) {
        b = find_2ecc(b);
        path_b.push_back(b);
        if (last_visit[b] == lca_iteration){
            lca = b;
            break;
        }
        last_visit[b] = lca_iteration;
        b = par[b];
    }
}

for (int v : path_a) {
    dsu_2ecc[v] = lca;
    if (v == lca)
        break;
    --bridges;
}
for (int v : path_b) {
    dsu_2ecc[v] = lca;
    if (v == lca)
        break;
    --bridges;
}
}

void add_edge(int a, int b) {
    a = find_2ecc(a);
    b = find_2ecc(b);
    if (a == b)
        return;

    int ca = find_cc(a);
    int cb = find_cc(b);

    if (ca != cb) {
        ++bridges;
        if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
            swap(a, b);
            swap(ca, cb);
        }
        make_root(a);
        par[a] = dsu_cc[a] = b;
        dsu_cc_size[cb] += dsu_cc_size[a];
    } else {
        merge_path(a, b);
    }
}

```

The DSU for the 2-edge-connected components is stored in the vector `dsu_2ecc`, and the function returning the representative is `find_2ecc(v)`. This function is used many times in the rest of the code, since after the compression of several vertices into one all these vertices cease to exist, and instead only the leader has the correct ancestor `par` in the forest of 2-edge-connected components.

The DSU for the connected components is stored in the vector `dsu_cc`, and there is also an additional vector `dsu_cc_size` to store the component sizes. The function `find_cc(v)` returns the leader of the connectivity component (which is actually the root of the tree).

The re-rooting of a tree `make_root(v)` works as described above: it traverses from the vertex `v` via the ancestors to the root vertex, each time redirecting the ancestor `par` in the opposite direction. The link to the representative of the connected component `dsu_cc` is also updated, so that it points to the new root vertex. After re-rooting we have to assign the new root the correct size of the connected component. Also we have to be careful that we call `find_2ecc()` to get the representatives of the 2-edge-connected component, rather than some other vertex that have already been compressed.

The cycle finding and compression function `merge_path(a, b)` is also implemented as described above. It searches for the LCA of `a` and `b` by rising these nodes in parallel, until we meet a vertex for the second time. For efficiency purposes we choose a unique identifier for each LCA finding call, and mark the traversed vertices with it. This works in $O(1)$, while other approaches like using set perform worse. The passed paths are stored in the vectors `path_a` and `path_b`, and we use them to walk through them a second time up to the LCA, thereby obtaining all vertices of the cycle. All the vertices of the cycle get compressed by attaching them to the LCA, hence the average complexity is $O(\log n)$ (since we don't use Union by rank). All the edges we pass have been bridges, so we subtract 1 for each edge in the cycle.

Finally the query function `add_edge(a, b)` determines the connected components in which the vertices `a` and `b` lie. If they lie in different connectivity components, then a smaller tree is re-rooted and then attached to the larger tree. Otherwise if the vertices `a` and `b` lie in one tree, but in different 2-edge-connected components, then the function `merge_path(a, b)` is called, which will detect the cycle and compress it into one 2-edge-connected component.

Contributors:

[Jakob Kogler](#) (92.78%) [Outkarsh](#) (2.53%) [Oleksandr Kulkov](#) (2.17%) [Le Bao Hiep](#) (0.72%) [fffelix-huang](#) (0.72%) [Abanoub Ashraf](#) (0.36%)
[Avinash Pandey](#) (0.36%) [wiktorkuchta](#) (0.36%)

Translated From: e-maxx.ru

Finding articulation points in a graph in $O(N + M)$

We are given an undirected graph. An articulation point (or cut vertex) is defined as a vertex which, when removed along with associated edges, makes the graph disconnected (or more precisely, increases the number of connected components in the graph). The task is to find all articulation points in the given graph.

The algorithm described here is based on [depth first search](#) and has $O(N + M)$ complexity, where N is the number of vertices and M is the number of edges in the graph.

Algorithm

Pick an arbitrary vertex of the graph r_{root} and run [depth first search](#) from it. Note the following fact (which is easy to prove):

- Let's say we are in the DFS, looking through the edges starting from vertex $v \neq r_{root}$. If the current edge (v, to) is such that none of the vertices to or its descendants in the DFS traversal tree has a back-edge to any of ancestors of v , then v is an articulation point. Otherwise, v is not an articulation point.
- Let's consider the remaining case of $v = r_{root}$. This vertex will be the point of articulation if and only if this vertex has more than one child in the DFS tree.

Now we have to learn to check this fact for each vertex efficiently. We'll use "time of entry into node" computed by the depth first search.

So, let $tin[v]$ denote entry time for node v . We introduce an array $low[v]$ which will let us check the fact for each vertex v . $low[v]$ is the minimum of $tin[v]$, the entry times $tin[p]$ for each node p that is connected to node v via a back-edge (v, p) and the values of $low[to]$ for each vertex to which is a direct descendant of v in the DFS tree:

$$low[v] = \min \begin{cases} tin[v] \\ tin[p] & \text{for all } p \text{ for which } (v, p) \text{ is a back edge} \\ low[to] & \text{for all } to \text{ for which } (v, to) \text{ is a tree edge} \end{cases}$$

Now, there is a back edge from vertex v or one of its descendants to one of its ancestors if and only if vertex v has a child to for which $low[to] < tin[v]$. If $low[to] = tin[v]$, the back edge comes directly to v , otherwise it comes to one of the ancestors of v .

Thus, the vertex v in the DFS tree is an articulation point if and only if $low[to] \geq tin[v]$.

Implementation

The implementation needs to distinguish three cases: when we go down the edge in DFS tree, when we find a back edge to an ancestor of the vertex and when we return to a parent of the vertex. These are the cases:

- $visited[to] = false$ - the edge is part of DFS tree;
- $visited[to] = true \ \&\& \neq parent$ - the edge is back edge to one of the ancestors;
- $to = parent$ - the edge leads back to parent in DFS tree.

To implement this, we need a depth first search function which accepts the parent vertex of the current node.

```
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph

vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
```

```

visited[v] = true;
tin[v] = low[v] = timer++;
int children=0;
for (int to : adj[v]) {
    if (to == p) continue;
    if (visited[to]) {
        low[v] = min(low[v], tin[to]);
    } else {
        dfs(to, v);
        low[v] = min(low[v], low[to]);
        if (low[to] >= tin[v] && p != -1)
            IS_CUTPOINT(v);
        ++children;
    }
}
if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

Main function is `find_cutpoints`; it performs necessary initialization and starts depth first search in each connected component of the graph.

Function `IS_CUTPOINT(a)` is some function that will process the fact that vertex `a` is an articulation point, for example, print it (Caution that this can be called multiple times for a vertex).

Practice Problems

- [UVA #10199 "Tourist Guide"](#) [difficulty: low]
- [UVA #315 "Network"](#) [difficulty: low]
- [SPOJ - Submerging Islands](#)
- [Codeforces - Cutting Figure](#)

Contributors:

[Pranjal Jain](#) (56.52%) [Jakob Kogler](#) (21.74%) [wiktorkuchta](#) (14.13%) [Oleksandr Kulkov](#) (3.26%) [Avinash Pandey](#) (2.17%) [Madhur Chauhan](#) (2.17%)

Translated From: e-maxx.ru

Strongly connected components and the condensation graph

Definitions

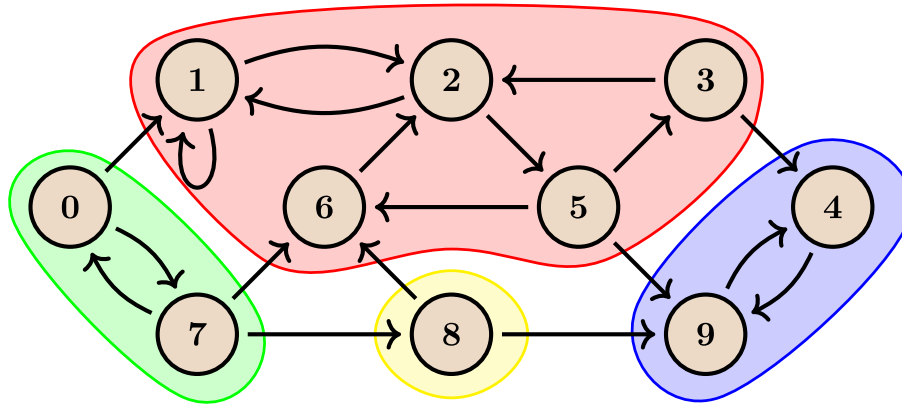
Let $G = (V, E)$ be a directed graph with vertices V and edges $E \subseteq V \times V$. We denote with $n = |V|$ the number of vertices and with $m = |E|$ the number of edges in G . It is easy to extend all definitions in this article to multigraphs, but we will not focus on that.

A subset of vertices $C \subseteq V$ is called a **strongly connected component** if the following conditions hold:

- for all $u, v \in C$, if $u \neq v$ there exists a path from u to v and a path from v to u , and
- C is maximal, in the sense that no vertex can be added without violating the above condition.

We denote with $\text{SCC}(G)$ the set of strongly connected components of G . These strongly connected components do not intersect with each other, and cover all vertices in the graph. Thus, the set $\text{SCC}(G)$ is a partition of V .

Consider this graph G_{example} , in which the strongly connected components are highlighted:

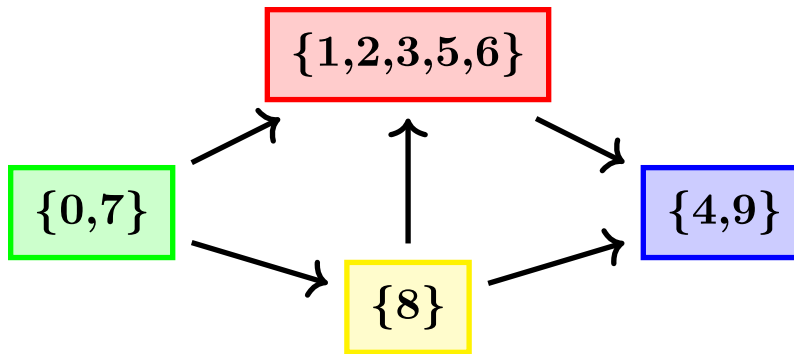


Here we have $\text{SCC}(G_{\text{example}}) = \{\{0, 7\}, \{1, 2, 3, 5, 6\}, \{4, 9\}, \{8\}\}$. We can confirm that within each strongly connected component, all vertices are reachable from each other.

We define the **condensation graph** $G^{\text{SCC}} = (V^{\text{SCC}}, E^{\text{SCC}})$ as follows:

- the vertices of G^{SCC} are the strongly connected components of G ; i.e., $V^{\text{SCC}} = \text{SCC}(G)$, and
- for all vertices C_i, C_j of the condensation graph, there is an edge from C_i to C_j if and only if $C_i \neq C_j$ and there exist $a \in C_i$ and $b \in C_j$ such that there is an edge from a to b in G .

The condensation graph of G_{example} looks as follows:



The most important property of the condensation graph is that it is **acyclic**. Indeed, there are no 'self-loops' in the condensation graph by definition, and if there were a cycle going through two or more vertices (strongly connected components) in the condensation graph, then due to reachability, the union of these strongly connected components would have to be one strongly connected component itself: contradiction.

The algorithm described in the next section finds all strongly connected components in a given graph. After that, the condensation graph can be constructed.

Description of the algorithm

The described algorithm was independently suggested by Kosaraju and Sharir around 1980. It is based on two series of **depth first search**, with a runtime of $O(n + m)$.

In the first step of the algorithm, we perform a sequence of depth first searches (**dfs**), visiting the entire graph. That is, as long as there are still unvisited vertices, we take one of them, and initiate a depth first search from that vertex. For each vertex, we keep track of the *exit time* $t_{out}[v]$. This is the 'timestamp' at which the execution of **dfs** on vertex v finishes, i.e., the moment at which all vertices reachable from v have been visited and the algorithm is back at v . The timestamp counter should *not* be reset between consecutive calls to **dfs**. The exit times play a key role in the algorithm, which will become clear when we discuss the following theorem.

First, we define the exit time $t_{out}[C]$ of a strongly connected component C as the maximum of the values $t_{out}[v]$ for all $v \in C$. Furthermore, in the proof of the theorem, we will mention the *entry time* $t_{in}[v]$ for each vertex $v \in G$. The number $t_{in}[v]$ represents the 'timestamp' at which the recursive function **dfs** is called on vertex v in the first step of the algorithm. For a strongly connected component C , we define $t_{in}[C]$ to be the minimum of the values $t_{in}[v]$ for all $v \in C$.

Theorem. Let C and C' be two different strongly connected components, and let there be an edge from C to C' in the condensation graph. Then, $t_{out}[C] > t_{out}[C']$.

Proof. There are two different cases, depending on which component will first be reached by depth first search:

- Case 1: the component C was reached first (i.e., $t_{in}[C] < t_{in}[C']$). In this case, depth first search visits some vertex $v \in C$ at some moment at which all other vertices of the components C and C' are not visited yet. Since there is an edge from C to C' in the condensation graph, not only are all other vertices in C reachable from v in G , but all vertices in C' are reachable as well. This means that this **dfs** execution, which is running from vertex v , will also visit all other vertices of the components C and C' in the future, so these vertices will be descendants of v in the depth first search tree. This implies that for each vertex $u \in (C \cup C') \setminus \{v\}$, we have that $t_{out}[v] > t_{out}[u]$. Therefore, $t_{out}[C] > t_{out}[C']$, which completes this case of the proof.
- Case 2: the component C' was reached first (i.e., $t_{in}[C] > t_{in}[C']$). In this case, depth first search visits some vertex $v \in C'$ at some moment at which all other vertices of the components C and C' are not visited yet. Since there is an edge from C to C' in the condensation graph, C is not reachable from C' , by the acyclicity property. Hence, the **dfs** execution that is running from vertex v will not reach any vertices of C , but it will visit all vertices of C' . The vertices of C will be visited by some **dfs** execution later during this step of the algorithm, so indeed we have $t_{out}[C] > t_{out}[C']$. This completes the proof.

The proved theorem is very important for finding strongly connected components. It means that any edge in the condensation graph goes from a component with a larger value of t_{out} to a component with a smaller value.

If we sort all vertices $v \in V$ in decreasing order of their exit time $t_{out}[v]$, then the first vertex u will belong to the "root" strongly connected component, which has no incoming edges in the condensation graph. Now we want to run some type of search from this vertex u so that it will

visit all vertices in its strongly connected component, but not other vertices. By repeatedly doing so, we can gradually find all strongly connected components: we remove all vertices belonging to the first found component, then we find the next remaining vertex with the largest value of t_{out} , and run this search from it, and so on. In the end, we will have found all strongly connected components. In order to find a search method that behaves like we want, we consider the following theorem:

Theorem. Let G^T denote the *transpose graph* of G , obtained by reversing the edge directions in G . Then, $\text{SCC}(G) = \text{SCC}(G^T)$. Furthermore, the condensation graph of G^T is the transpose of the condensation graph of G .

The proof is omitted (but straightforward). As a consequence of this theorem, there will be no edges from the "root" component to the other components in the condensation graph of G^T . Thus, in order to visit the whole "root" strongly connected component, containing vertex v , we can just run a depth first search from vertex v in the transpose graph G^T ! This will visit precisely all vertices of this strongly connected component. As was mentioned before, we can then remove these vertices from the graph. Then, we find the next vertex with a maximal value of $t_{\text{out}}[v]$, and run the search in the transpose graph starting from that vertex to find the next strongly connected component. Repeating this, we find all strongly connected components.

Thus, in summary, we discussed the following algorithm to find strongly connected components:

- Step 1. Run a sequence of depth first searches on G , which will yield some list (e.g. `order`) of vertices, sorted on increasing exit time t_{out} .
- Step 2. Build the transpose graph G^T , and run a series of depth first searches on the vertices in reverse order (i.e., in decreasing order of exit times). Each depth first search will yield one strongly connected component.
- Step 3 (optional). Build the condensation graph.

The runtime complexity of the algorithm is $O(n + m)$, because depth first search is performed twice. Building the condensation graph is also $O(n + m)$.

Finally, it is appropriate to mention [topological sort](#) here. In step 1, we find the vertices in the order of increasing exit time. If G is acyclic, this corresponds to a (reversed) topological sort of G . In step 2, the algorithm finds strongly connected components in decreasing order of their exit times. Thus, it finds components - vertices of the condensation graph - in an order corresponding to a topological sort of the condensation graph.

Implementation

```
vector<bool> visited; // keeps track of which vertices are already visited

// runs depth first search starting at vertex v.
// each visited vertex is appended to the output vector when dfs leaves it.
void dfs(int v, vector<vector<int>> const& adj, vector<int> &output) {
    visited[v] = true;
    for (auto u : adj[v])
        if (!visited[u])
            dfs(u, adj, output);
    output.push_back(v);
}

// input: adj -- adjacency list of G
// output: components -- the strongly connected components in G
// output: adj_cond -- adjacency list of G^SCC (by root vertices)
void strongly_connected_components(vector<vector<int>> const& adj,
                                   vector<vector<int>> &components,
                                   vector<vector<int>> &adj_cond) {

    int n = adj.size();
    components.clear(), adj_cond.clear();

    vector<int> order; // will be a sorted list of G's vertices by exit time

    visited.assign(n, false);

    // first series of depth first searches
    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs(i, adj, order);

    // create adjacency list of G^T
    vector<vector<int>> adj_rev(n);
    for (int v = 0; v < n; v++)
        for (int u : adj[v])
            adj_rev[u].push_back(v);
```

```

visited.assign(n, false);
reverse(order.begin(), order.end());

vector<int> roots(n, 0); // gives the root vertex of a vertex's SCC

// second series of depth first searches
for (auto v : order)
    if (!visited[v]) {
        std::vector<int> component;
        dfs(v, adj_rev, component);
        components.push_back(component);
        int root = *min_element(begin(component), end(component));
        for (auto u : component)
            roots[u] = root;
    }

// add edges to condensation graph
adj_cond.assign(n, {});
for (int v = 0; v < n; v++)
    for (auto u : adj[v])
        if (roots[v] != roots[u])
            adj_cond[roots[v]].push_back(roots[u]);
}

```

The function `dfs` implements depth first search. It takes as input an adjacency list and a starting vertex. It also takes a reference to the vector `output`: each visited vertex will be appended to `output` when `dfs` leaves that vertex.

Note that we use the function `dfs` both in the first and second step of the algorithm. In the first step, we pass in the adjacency list of G , and during consecutive calls to `dfs`, we keep passing in the same 'output vector' `order`, so that eventually we obtain a list of vertices in increasing order of exit times. In the second step, we pass in the adjacency list of G^T , and in each call, we pass in an empty 'output vector' `component`, which will give us one strongly connected component at a time.

When building the adjacency list of the condensation graph, we select the *root* of each component as the first vertex in its list of vertices (this is an arbitrary choice). This root vertex represents its entire SCC. For each vertex v , the value `roots[v]` indicates the root vertex of the SCC which v belongs to.

Our condensation graph is now given by the vertices `components` (one strongly connected component corresponds to one vertex in the condensation graph), and the adjacency list is given by `adj_cond`, using only the root vertices of the strongly connected components. Notice that we generate one edge from C to C' in G^{SCC} for each edge from some $a \in C$ to some $b \in C'$ in G (if $C \neq C'$). This implies that in our implementation, we can have multiple edges between two components in the condensation graph.

Literature

- Thomas Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein. Introduction to Algorithms [2005].
- M. Sharir. A strong-connectivity algorithm and its applications in data-flow analysis [1979].

Practice Problems

- [SPOJ - Good Travels](#)
- [SPOJ - Lego](#)
- [Codechef - Chef and Round Run](#)
- [UVA - 11838 - Come and Go](#)
- [UVA 247 - Calling Circles](#)
- [UVA 13057 - Prove Them All](#)
- [UVA 12645 - Water Supply](#)
- [UVA 11770 - Lighting Away](#)
- [UVA 12926 - Trouble in Terrorist Town](#)
- [UVA 11324 - The Largest Clique](#)
- [UVA 11709 - Trust groups](#)
- [UVA 12745 - Wishmaster](#)

- [SPOJ - True Friends](#)
- [SPOJ - Capital City](#)
- [Codeforces - Scheme](#)
- [SPOJ - Ada and Panels](#)
- [CSES - Flight Routes Check](#)
- [CSES - Planets and Kingdoms](#)
- [CSES - Coin Collector](#)
- [Codeforces - Checkposts](#)

Contributors:

[el-sambal](#) (52.05%) [Dmitry Dobrynin](#) (15.79%) [Gaurang Tandon](#) (8.77%) [Morass](#) (7.6%) [Oleksandr Kulkov](#) (5.85%) [Aryaman](#) (1.75%)
[Rodion Gorkovenko](#) (1.75%) [Shadab Zafar](#) (1.75%) [Chirag Agarwal](#) (1.17%) [Jakob Kogler](#) (1.17%) [Mikael Mello](#) (1.17%) [Tanmay Sinha](#) (0.58%)
[chubakueno](#) (0.58%)

Original

Strong Orientation

A **strong orientation** of an undirected graph is an assignment of a direction to each edge that makes it a **strongly connected graph**. That is, after the *orientation* we should be able to visit any vertex from any vertex by following the directed edges.

Solution

Of course, this cannot be done to *every* graph. Consider a **bridge** in a graph. We have to assign a direction to it and by doing so we make this bridge "crossable" in only one direction. That means we can't go from one of the bridge's ends to the other, so we can't make the graph strongly connected.

Now consider a **DFS** through a bridgeless connected graph. Clearly, we will visit each vertex. And since there are no bridges, we can remove any DFS tree edge and still be able to go from below the edge to above the edge by using a path that contains at least one back edge. From this follows that from any vertex we can go to the root of the DFS tree. Also, from the root of the DFS tree we can visit any vertex we choose. We found a strong orientation!

In other words, to strongly orient a bridgeless connected graph, run a DFS on it and let the DFS tree edges point away from the DFS root and all other edges from the descendant to the ancestor in the DFS tree.

The result that bridgeless connected graphs are exactly the graphs that have strong orientations is called **Robbins' theorem**.

Problem extension

Let's consider the problem of finding a graph orientation so that the number of SCCs is minimal.

Of course, each graph component can be considered separately. Now, since only bridgeless graphs are strongly orientable, let's remove all bridges temporarily. We end up with some number of bridgeless components (exactly *how many components there were at the beginning + how many bridges there were*) and we know that we can strongly orient each of them.

We were only allowed to orient edges, not remove them, but it turns out we can orient the bridges arbitrarily. Of course, the easiest way to orient them is to run the algorithm described above without modifications on each original connected component.

Implementation

Here, the input is n — the number of vertices, m — the number of edges, then m lines describing the edges.

The output is the minimal number of SCCs on the first line and on the second line a string of m characters, either $>$ — telling us that the corresponding edge from the input is oriented from the left to the right vertex (as in the input), or $<$ — the opposite.

This is a bridge search algorithm modified to also orient the edges, you can as well orient the edges as a first step and count the SCCs on the oriented graph as a second.

```
vector<vector<pair<int, int>>> adj; // adjacency list - vertex and edge pairs
vector<pair<int, int>> edges;

vector<int> tin, low;
int bridge_cnt;
string orient;
vector<bool> edge_used;
void find_bridges(int v) {
    static int time = 0;
    low[v] = tin[v] = time++;
    for (auto p : adj[v]) {
        if (edge_used[p.second]) continue;
        edge_used[p.second] = true;
        orient[p.second] = v == edges[p.second].first ? '>' : '<';
    }
}
```

```

    int nv = p.first;
    if (tin[nv] == -1) { // if nv is not visited yet
        find_bridges(nv);
        low[v] = min(low[v], low[nv]);
        if (low[nv] > tin[v]) {
            // a bridge between v and nv
            bridge_cnt++;
        }
    } else {
        low[v] = min(low[v], tin[nv]);
    }
}
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    adj.resize(n);
    tin.resize(n, -1);
    low.resize(n, -1);
    orient.resize(m);
    edges.resize(m);
    edge_used.resize(m);
    for (int i = 0; i < m; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        a--; b--;
        adj[a].push_back({b, i});
        adj[b].push_back({a, i});
        edges[i] = {a, b};
    }
    int comp_cnt = 0;
    for (int v = 0; v < n; v++) {
        if (tin[v] == -1) {
            comp_cnt++;
            find_bridges(v);
        }
    }
    printf("%d\n%s\n", comp_cnt + bridge_cnt, orient.c_str());
}

```

Practice Problems

- [26th Polish OI - Osiedla](#)

Contributors:

[Wiktor Kuchta](#) (71.55%) [wiktorkuchta](#) (17.24%) [Jakob Kogler](#) (6.03%) [Oleksandr Kulkov](#) (4.31%) [Atulbedwal](#) (0.86%)

Translated From: e-maxx.ru

Dijkstra Algorithm

You are given a directed or undirected weighted graph with n vertices and m edges. The weights of all edges are non-negative. You are also given a starting vertex s . This article discusses finding the lengths of the shortest paths from a starting vertex s to all other vertices, and output the shortest paths themselves.

This problem is also called **single-source shortest paths problem**.

Algorithm

Here is an algorithm described by the Dutch computer scientist Edsger W. Dijkstra in 1959.

Let's create an array $d[]$ where for each vertex v we store the current length of the shortest path from s to v in $d[v]$. Initially $d[s] = 0$, and for all other vertices this length equals infinity. In the implementation a sufficiently large number (which is guaranteed to be greater than any possible path length) is chosen as infinity.

$$d[v] = \infty, v \neq s$$

In addition, we maintain a Boolean array $u[]$ which stores for each vertex v whether it's marked. Initially all vertices are unmarked:

$$u[v] = \text{false}$$

The Dijkstra's algorithm runs for n iterations. At each iteration a vertex v is chosen as unmarked vertex which has the least value $d[v]$:

Evidently, in the first iteration the starting vertex s will be selected.

The selected vertex v is marked. Next, from vertex v **relaxations** are performed: all edges of the form (v, to) are considered, and for each vertex to the algorithm tries to improve the value $d[to]$. If the length of the current edge equals len , the code for relaxation is:

$$d[to] = \min(d[to], d[v] + len)$$

After all such edges are considered, the current iteration ends. Finally, after n iterations, all vertices will be marked, and the algorithm terminates. We claim that the found values $d[v]$ are the lengths of shortest paths from s to all vertices v .

Note that if some vertices are unreachable from the starting vertex s , the values $d[v]$ for them will remain infinite. Obviously, the last few iterations of the algorithm will choose those vertices, but no useful work will be done for them. Therefore, the algorithm can be stopped as soon as the selected vertex has infinite distance to it.

Restoring Shortest Paths

Usually one needs to know not only the lengths of shortest paths but also the shortest paths themselves. Let's see how to maintain sufficient information to restore the shortest path from s to any vertex. We'll maintain an array of predecessors $p[]$ in which for each vertex $v \neq s$, $p[v]$ is the penultimate vertex in the shortest path from s to v . Here we use the fact that if we take the shortest path to some vertex v and remove v from this path, we'll get a path ending in at vertex $p[v]$, and this path will be the shortest for the vertex $p[v]$. This array of predecessors can be used to restore the shortest path to any vertex: starting with v , repeatedly take the predecessor of the current vertex until we reach the starting vertex s to get the required shortest path with vertices listed in reverse order. So, the shortest path P to the vertex v is equal to:

$$P = (s, \dots, p[p[p[v]]], p[p[v]], p[v], v)$$

Building this array of predecessors is very simple: for each successful relaxation, i.e. when for some selected vertex v , there is an improvement in the distance to some vertex to , we update the predecessor vertex for to with vertex v :

$$p[to] = v$$

Proof

The main assertion on which Dijkstra's algorithm correctness is based is the following:

After any vertex v becomes marked, the current distance to it $d[v]$ is the shortest, and will no longer change.

The proof is done by induction. For the first iteration this statement is obvious: the only marked vertex is s , and the distance to it $d[s] = 0$ is indeed the length of the shortest path to s . Now suppose this statement is true for all previous iterations, i.e. for all already marked vertices; let's prove that it is not violated after the current iteration completes. Let v be the vertex selected in the current iteration, i.e. v is the vertex that the algorithm will mark. Now we have to prove that $d[v]$ is indeed equal to the length of the shortest path to it $l[v]$.

Consider the shortest path P to the vertex v . This path can be split into two parts: P_1 which consists of only marked nodes (at least the starting vertex s is part of P_1), and the rest of the path P_2 (it may include a marked vertex, but it always starts with an unmarked vertex). Let's denote the first vertex of the path P_2 as p , and the last vertex of the path P_1 as q .

First we prove our statement for the vertex p , i.e. let's prove that $d[p] = l[p]$. This is almost obvious: on one of the previous iterations we chose the vertex q and performed relaxation from it. Since (by virtue of the choice of vertex p) the shortest path to p is the shortest path to q plus edge (p, q) , the relaxation from q set the value of $d[p]$ to the length of the shortest path $l[p]$.

Since the edges' weights are non-negative, the length of the shortest path $l[p]$ (which we just proved to be equal to $d[p]$) does not exceed the length $l[v]$ of the shortest path to the vertex v . Given that $l[v] \leq d[v]$ (because Dijkstra's algorithm could not have found a shorter way than the shortest possible one), we get the inequality:

$$d[p] = l[p] \leq l[v] \leq d[v]$$

On the other hand, since both vertices p and v are unmarked, and the current iteration chose vertex v , not p , we get another inequality:

$$d[p] \geq d[v]$$

From these two inequalities we conclude that $d[p] = d[v]$, and then from previously found equations we get:

$$d[v] = l[v]$$

Q.E.D.

Implementation

Dijkstra's algorithm performs n iterations. On each iteration it selects an unmarked vertex v with the lowest value $d[v]$, marks it and checks all the edges (v, to) attempting to improve the value $d[to]$.

The running time of the algorithm consists of:

- n searches for a vertex with the smallest value $d[v]$ among $O(n)$ unmarked vertices
- m relaxation attempts

For the simplest implementation of these operations on each iteration vertex search requires $O(n)$ operations, and each relaxation can be performed in $O(1)$. Hence, the resulting asymptotic behavior of the algorithm is:

$$O(n^2 + m)$$

This complexity is optimal for dense graph, i.e. when $m \approx n^2$. However in sparse graphs, when m is much smaller than the maximal number of edges n^2 , the problem can be solved in $O(n \log n + m)$ complexity. The algorithm and implementation can be found on the article [Dijkstra on sparse graphs](#).

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);
    vector<bool> u(n, false);

    d[s] = 0;
    for (int i = 0; i < n; i++) {
        int v = -1;
        for (int j = 0; j < n; j++) {
            if (!u[j] && (v == -1 || d[j] < d[v]))
                v = j;
        }

        if (d[v] == INF)
            break;

        u[v] = true;
        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
            }
        }
    }
}
```

Here the graph `adj` is stored as adjacency list: for each vertex v `adj[v]` contains the list of edges going from this vertex, i.e. the list of `pair<int, int>` where the first element in the pair is the vertex at the other end of the edge, and the second element is the edge weight.

The function takes the starting vertex s and two vectors that will be used as return values.

First of all, the code initializes arrays: distances `d[]`, labels `u[]` and predecessors `p[]`. Then it performs n iterations. At each iteration the vertex v is selected which has the smallest distance `d[v]` among all the unmarked vertices. If the distance to selected vertex v is equal to infinity, the algorithm stops. Otherwise the vertex is marked, and all the edges going out from this vertex are checked. If relaxation along the edge is possible (i.e. distance `d[to]` can be improved), the distance `d[to]` and predecessor `p[to]` are updated.

After performing all the iterations array `d[]` stores the lengths of the shortest paths to all vertices, and array `p[]` stores the predecessors of all vertices (except starting vertex s). The path to any vertex t can be restored in the following way:

```
vector<int> restore_path(int s, int t, vector<int> const& p) {
    vector<int> path;

    for (int v = t; v != s; v = p[v])
        path.push_back(v);
    path.push_back(s);

    reverse(path.begin(), path.end());
    return path;
}
```

References

- [Edsger Dijkstra. A note on two problems in connexion with graphs \[1959\]](#)
- [Thomas Cormen, Charles Leiserson, Ronald Rivest, Clifford Stein. Introduction to Algorithms \[2005\]](#)

Practice Problems

- [Timus - Ivan's Car \[Difficulty:Medium\]](#)
- [Timus - Sightseeing Trip](#)
- [SPOJ - SHPATH \[Difficulty:Easy\]](#)
- [Codeforces - Dijkstra? \[Difficulty:Easy\]](#)
- [Codeforces - Shortest Path](#)
- [Codeforces - Jzzhu and Cities](#)
- [Codeforces - The Classic Problem](#)
- [Codeforces - President and Roads](#)
- [Codeforces - Complete The Graph](#)
- [TopCoder - SkiResorts](#)
- [TopCoder - MaliciousPath](#)
- [SPOJ - Ada and Trip](#)
- [LA - 3850 - Here We Go\(relians\) Again](#)
- [GYM - Destination Unknown \(D\)](#)
- [UVA 12950 - Even Obsession](#)
- [GYM - Journey to Grece \(A\)](#)
- [UVA 13030 - Brain Fry](#)
- [UVA 1027 - Toll](#)
- [UVA 11377 - Airport Setup](#)
- [Codeforces - Dynamic Shortest Path](#)
- [UVA 11813 - Shopping](#)
- [UVA 11833 - Route Change](#)
- [SPOJ - Easy Dijkstra Problem](#)
- [LA - 2819 - Cave Raider](#)
- [UVA 12144 - Almost Shortest Path](#)
- [UVA 12047 - Highest Paid Toll](#)
- [UVA 11514 - Batman](#)
- [Codeforces - Team Rocket Rises Again](#)
- [UVA - 11338 - Minefield](#)
- [UVA 11374 - Airport Express](#)
- [UVA 11097 - Poor My Problem](#)
- [UVA 13172 - The music teacher](#)
- [Codeforces - Dirty Arkady's Kitchen](#)
- [SPOJ - Delivery Route](#)
- [SPOJ - Costly Chess](#)
- [CSES - Shortest Routes 1](#)
- [CSES - Flight Discount](#)
- [CSES - Flight Routes](#)

Contributors:

[code_master5](#) (42.27%) [Jakob Kogler](#) (20.62%) [Mariia Mykhailova](#) (16.49%) [Morass](#) (11.34%) [Oleksandr Kulkov](#) (3.09%) [Aryaman](#) (1.55%)
[Chaman Agrawal](#) (1.55%) [Rodion Gorkovenko](#) (1.55%) [Bhuvnesh Jain](#) (1.03%) [PirateOfAndaman](#) (0.52%)

Translated From: e-maxx.ru

Dijkstra on sparse graphs

For the statement of the problem, the algorithm with implementation and proof can be found on the article [Dijkstra's algorithm](#).

Algorithm

We recall in the derivation of the complexity of Dijkstra's algorithm we used two factors: the time of finding the unmarked vertex with the smallest distance $d[v]$, and the time of the relaxation, i.e. the time of changing the values $d[to]$.

In the simplest implementation these operations require $O(n)$ and $O(1)$ time. Therefore, since we perform the first operation $O(n)$ times, and the second one $O(m)$ times, we obtained the complexity $O(n^2 + m)$.

It is clear, that this complexity is optimal for a dense graph, i.e. when $m \approx n^2$. However in sparse graphs, when m is much smaller than the maximal number of edges n^2 , the complexity gets less optimal because of the first term. Thus it is necessary to improve the execution time of the first operation (and of course without greatly affecting the second operation by much).

To accomplish that we can use a variation of multiple auxiliary data structures. The most efficient is the **Fibonacci heap**, which allows the first operation to run in $O(\log n)$, and the second operation in $O(1)$. Therefore we will get the complexity $O(n \log n + m)$ for Dijkstra's algorithm, which is also the theoretical minimum for the shortest path search problem. Therefore this algorithm works optimal, and Fibonacci heaps are the optimal data structure. There doesn't exist any data structure, that can perform both operations in $O(1)$, because this would also allow to sort a list of random numbers in linear time, which is impossible. Interestingly there exists an algorithm by Thorup that finds the shortest path in $O(m)$ time, however only works for integer weights, and uses a completely different idea. So this doesn't lead to any contradictions. Fibonacci heaps provide the optimal complexity for this task. However they are quite complex to implement, and also have a quite large hidden constant.

As a compromise you can use data structures, that perform both types of operations (extracting a minimum and updating an item) in $O(\log n)$. Then the complexity of Dijkstra's algorithm is $O(n \log n + m \log n) = O(m \log n)$.

C++ provides two such data structures: `set` and `priority_queue`. The first is based on red-black trees, and the second one on heaps. Therefore `priority_queue` has a smaller hidden constant, but also has a drawback: it doesn't support the operation of removing an element. Because of this we need to do a "workaround", that actually leads to a slightly worse factor $\log m$ instead of $\log n$ (although in terms of complexity they are identical).

Implementation

set

Let us start with the container `set`. Since we need to store vertices ordered by their values $d[]$, it is convenient to store actual pairs: the distance and the index of the vertex. As a result in a `set` pairs are automatically sorted by their distances.

```
const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    set<pair<int, int>> q;
    q.insert({0, s});
    while (!q.empty()) {
        int v = q.begin()->second;
        q.erase(q.begin());

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;
```

```

        if (d[v] + len < d[to]) {
            q.erase({d[to], to});
            d[to] = d[v] + len;
            p[to] = v;
            q.insert({d[to], to});
        }
    }
}

```

We don't need the array `u[]` from the normal Dijkstra's algorithm implementation any more. We will use the `set` to store that information, and also find the vertex with the shortest distance with it. It kinda acts like a queue. The main loops executes until there are no more vertices in the set/queue. A vertex with the smallest distance gets extracted, and for each successful relaxation we first remove the old pair, and then after the relaxation add the new pair into the queue.

priority_queue

The main difference to the implementation with `set` is that in many languages, including C++, we cannot remove elements from the `priority_queue` (although heaps can support that operation in theory). Therefore we have to use a workaround: We simply don't delete the old pair from the queue. As a result a vertex can appear multiple times with different distance in the queue at the same time. Among these pairs we are only interested in the pairs where the first element is equal to the corresponding value in `d[]`, all the other pairs are old. Therefore we need to make a small modification: at the beginning of each iteration, after extracting the next pair, we check if it is an important pair or if it is already an old and handled pair. This check is important, otherwise the complexity can increase up to $O(nm)$.

By default a `priority_queue` sorts elements in descending order. To make it sort the elements in ascending order, we can either store the negated distances in it, or pass it a different sorting function. We will do the second option.

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

void dijkstra(int s, vector<int> & d, vector<int> & p) {
    int n = adj.size();
    d.assign(n, INF);
    p.assign(n, -1);

    d[s] = 0;
    using pii = pair<int, int>;
    priority_queue<pii, vector<pii>, greater<pii>> q;
    q.push({0, s});
    while (!q.empty()) {
        int v = q.top().second;
        int d_v = q.top().first;
        q.pop();
        if (d_v != d[v])
            continue;

        for (auto edge : adj[v]) {
            int to = edge.first;
            int len = edge.second;

            if (d[v] + len < d[to]) {
                d[to] = d[v] + len;
                p[to] = v;
                q.push({d[to], to});
            }
        }
    }
}

```

In practice the `priority_queue` version is a little bit faster than the version with `set`.

Interestingly, a [2007 technical report](#) concluded the variant of the algorithm not using decrease-key operations ran faster than the decrease-key variant, with a greater performance gap for sparse graphs.

Getting rid of pairs

You can improve the performance a little bit more if you don't store pairs in the containers, but only the vertex indices. In this case we must overload the comparison operator: it must compare two vertices using the distances stored in `d[]`.

As a result of the relaxation, the distance of some vertices will change. However the data structure will not resort itself automatically. In fact changing distances of vertices in the queue, might destroy the data structure. As before, we need to remove the vertex before we relax it, and then insert it again afterwards.

Since we only can remove from `set`, this optimization is only applicable for the `set` method, and doesn't work with `priority_queue` implementation. In practice this significantly increases the performance, especially when larger data types are used to store distances, like `long` or `double`.

Contributors:

[Jakob Kogler](#) (90.13%) [Oleksandr Kulkov](#) (3.95%) [jxu](#) (2.63%) [Thejazeto Lhousa](#) (1.32%) [Avinash Pandey](#) (0.66%) [dj_-park](#) (0.66%) [wiktorkuchta](#) (0.66%)

Translated From: e-maxx.ru

Bellman-Ford Algorithm

Single source shortest path with negative weight edges

Suppose that we are given a weighted directed graph G with n vertices and m edges, and some specified vertex v . You want to find the length of shortest paths from vertex v to every other vertex.

Unlike the Dijkstra algorithm, this algorithm can also be applied to graphs containing negative weight edges. However, if the graph contains a negative cycle, then, clearly, the shortest path to some vertices may not exist (due to the fact that the weight of the shortest path must be equal to minus infinity); however, this algorithm can be modified to signal the presence of a cycle of negative weight, or even deduce this cycle.

The algorithm bears the name of two American scientists: Richard Bellman and Lester Ford. Ford actually invented this algorithm in 1956 during the study of another mathematical problem, which eventually reduced to a subproblem of finding the shortest paths in the graph, and Ford gave an outline of the algorithm to solve this problem. Bellman in 1958 published an article devoted specifically to the problem of finding the shortest path, and in this article he clearly formulated the algorithm in the form in which it is known to us now.

Description of the algorithm

Let us assume that the graph contains no negative weight cycle. The case of presence of a negative weight cycle will be discussed below in a separate section.

We will create an array of distances $d[0 \dots n - 1]$, which after execution of the algorithm will contain the answer to the problem. In the beginning we fill it as follows: $d[v] = 0$, and all other elements $d[]$ equal to infinity ∞ .

The algorithm consists of several phases. Each phase scans through all edges of the graph, and the algorithm tries to produce **relaxation** along each edge (a, b) having weight c . Relaxation along the edges is an attempt to improve the value $d[b]$ using value $d[a] + c$. In fact, it means that we are trying to improve the answer for this vertex using edge (a, b) and current answer for vertex a .

It is claimed that $n - 1$ phases of the algorithm are sufficient to correctly calculate the lengths of all shortest paths in the graph (again, we believe that the cycles of negative weight do not exist). For unreachable vertices the distance $d[]$ will remain equal to infinity ∞ .

Implementation

Unlike many other graph algorithms, for Bellman-Ford algorithm, it is more convenient to represent the graph using a single list of all edges (instead of n lists of edges - edges from each vertex). We start the implementation with a structure `edge` for representing the edges. The input to the algorithm are numbers n, m , list e of edges and the starting vertex v . All the vertices are numbered 0 to $n - 1$.

The simplest implementation

The constant `INF` denotes the number "infinity" — it should be selected in such a way that it is greater than all possible path lengths.

```
struct Edge {
    int a, b, cost;
};

int n, m, v;
vector<Edge> edges;
const int INF = 1000000000;

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (int i = 0; i < n - 1; ++i)
        for (Edge e : edges)
            if (d[e.a] < INF)
                d[e.b] = min(d[e.b], d[e.a] + e.cost);
}
```

```

    // display d, for example, on the screen
}

```

The check `if (d[e.a] < INF)` is needed only if the graph contains negative weight edges: no such verification would result in relaxation from the vertices to which paths have not yet found, and incorrect distance, of the type $\infty - 1$, $\infty - 2$ etc. would appear.

A better implementation

This algorithm can be somewhat speeded up: often we already get the answer in a few phases and no useful work is done in remaining phases, just a waste visiting all edges. So, let's keep the flag, to tell whether something changed in the current phase or not, and if any phase, nothing changed, the algorithm can be stopped. (This optimization does not improve the asymptotic behavior, i.e., some graphs will still need all $n - 1$ phases, but significantly accelerates the behavior of the algorithm "on an average", i.e., on random graphs.)

With this optimization, it is generally unnecessary to restrict manually the number of phases of the algorithm to $n - 1$ — the algorithm will stop after the desired number of phases.

```

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    for (;;) {
        bool any = false;

        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    any = true;
                }

        if (!any)
            break;
    }
    // display d, for example, on the screen
}

```

Retrieving Path

Let us now consider how to modify the algorithm so that it not only finds the length of shortest paths, but also allows to reconstruct the shortest paths.

For that, let's create another array $p[0 \dots n - 1]$, where for each vertex we store its "predecessor", i.e. the penultimate vertex in the shortest path leading to it. In fact, the shortest path to any vertex a is a shortest path to some vertex $p[a]$, to which we added a at the end of the path.

Note that the algorithm works on the same logic: it assumes that the shortest distance to one vertex is already calculated, and, tries to improve the shortest distance to other vertices from that vertex. Therefore, at the time of improvement we just need to remember $p[]$, i.e. the vertex from which this improvement has occurred.

Following is an implementation of the Bellman-Ford with the retrieval of shortest path to a given node t :

```

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);

    for (;;) {
        bool any = false;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    p[e.b] = e.a;
                    any = true;
                }

        if (!any)
            break;
    }

    if (d[t] == INF)

```

```

        cout << "No path from " << v << " to " << t << ".";
    else {
        vector<int> path;
        for (int cur = t; cur != -1; cur = p[cur])
            path.push_back(cur);
        reverse(path.begin(), path.end());

        cout << "Path from " << v << " to " << t << ": ";
        for (int u : path)
            cout << u << ' ';
    }
}

```

Here starting from the vertex t , we go through the predecessors till we reach starting vertex with no predecessor, and store all the vertices in the path in the list `path`. This list is a shortest path from v to t , but in reverse order, so we call `reverse()` function over `path` and then output the path.

The proof of the algorithm

First, note that for all unreachable vertices u the algorithm will work correctly, the label $d[u]$ will remain equal to infinity (because the algorithm Bellman-Ford will find some way to all reachable vertices from the start vertex v , and relaxation for all other remaining vertices will never happen).

Let us now prove the following assertion: After the execution of i_{th} phase, the Bellman-Ford algorithm correctly finds all shortest paths whose number of edges does not exceed i .

In other words, for any vertex a let us denote the k number of edges in the shortest path to it (if there are several such paths, you can take any). According to this statement, the algorithm guarantees that after k_{th} phase the shortest path for vertex a will be found.

Proof: Consider an arbitrary vertex a to which there is a path from the starting vertex v , and consider a shortest path to it ($p_0 = v, p_1, \dots, p_k = a$). Before the first phase, the shortest path to the vertex $p_0 = v$ was found correctly. During the first phase, the edge (p_0, p_1) has been checked by the algorithm, and therefore, the distance to the vertex p_1 was correctly calculated after the first phase. Repeating this statement k times, we see that after k_{th} phase the distance to the vertex $p_k = a$ gets calculated correctly, which we wanted to prove.

The last thing to notice is that any shortest path cannot have more than $n - 1$ edges. Therefore, the algorithm sufficiently goes up to the $(n - 1)_{th}$ phase. After that, it is guaranteed that no relaxation will improve the distance to some vertex.

The case of a negative cycle

Everywhere above we considered that there is no negative cycle in the graph (precisely, we are interested in a negative cycle that is reachable from the starting vertex v , and, for an unreachable cycles nothing in the above algorithm changes). In the presence of a negative cycle(s), there are further complications associated with the fact that distances to all vertices in this cycle, as well as the distances to the vertices reachable from this cycle is not defined — they should be equal to minus infinity $(-\infty)$.

It is easy to see that the Bellman-Ford algorithm can endlessly do the relaxation among all vertices of this cycle and the vertices reachable from it. Therefore, if you do not limit the number of phases to $n - 1$, the algorithm will run indefinitely, constantly improving the distance from these vertices.

Hence we obtain the **criterion for presence of a cycle of negative weights reachable for source vertex v** : after $(n - 1)_{th}$ phase, if we run algorithm for one more phase, and it performs at least one more relaxation, then the graph contains a negative weight cycle that is reachable from v ; otherwise, such a cycle does not exist.

Moreover, if such a cycle is found, the Bellman-Ford algorithm can be modified so that it retrieves this cycle as a sequence of vertices contained in it. For this, it is sufficient to remember the last vertex x for which there was a relaxation in n_{th} phase. This vertex will either lie on a negative weight cycle, or is reachable from it. To get the vertices that are guaranteed to lie on a negative cycle, starting from the vertex x , pass through to the predecessors n times. In this way, we will get to the vertex y , which is guaranteed to lie on a negative cycle. We have to go from this vertex, through the predecessors, until we get back to the same vertex y (and it will happen, because relaxation in a negative weight cycle occur in a circular manner).

Implementation:

```

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);
    int x;
    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = max(-INF, d[e.a] + e.cost);
                    p[e.b] = e.a;
                    x = e.b;
                }
    }

    if (x == -1)
        cout << "No negative cycle from " << v;
    else {
        int y = x;
        for (int i = 0; i < n; ++i)
            y = p[y];

        vector<int> path;
        for (int cur = y;; cur = p[cur]) {
            path.push_back(cur);
            if (cur == y && path.size() > 1)
                break;
        }
        reverse(path.begin(), path.end());

        cout << "Negative cycle: ";
        for (int u : path)
            cout << u << ' ';
    }
}

```

Due to the presence of a negative cycle, for n iterations of the algorithm, the distances may go far in the negative range (to negative numbers of the order of $-nmW$, where W is the maximum absolute value of any weight in the graph). Hence in the code, we adopted additional measures against the integer overflow as follows:

```
d[e.b] = max(-INF, d[e.a] + e.cost);
```

The above implementation looks for a negative cycle reachable from some starting vertex v ; however, the algorithm can be modified to just looking for any negative cycle in the graph. For this we need to put all the distance $d[i]$ to zero and not infinity — as if we are looking for the shortest path from all vertices simultaneously; the validity of the detection of a negative cycle is not affected.

For more on this topic — see separate article, [Finding a negative cycle in the graph](#).

Shortest Path Faster Algorithm (SPFA)

SPFA is an improvement of the Bellman-Ford algorithm which takes advantage of the fact that not all attempts at relaxation will work. The main idea is to create a queue containing only the vertices that were relaxed but that still could further relax their neighbors. And whenever you can relax some neighbor, you should put him in the queue. This algorithm can also be used to detect negative cycles as the Bellman-Ford.

The worst case of this algorithm is equal to the $O(nm)$ of the Bellman-Ford, but in practice it works much faster and some [people claim that it works even in \$O\(m\)\$ on average](#). However be careful, because this algorithm is deterministic and it is easy to create counterexamples that make the algorithm run in $O(nm)$.

There are some care to be taken in the implementation, such as the fact that the algorithm continues forever if there is a negative cycle. To avoid this, it is possible to create a counter that stores how many times a vertex has been relaxed and stop the algorithm as soon as some vertex got relaxed for the n -th time. Note, also there is no reason to put a vertex in the queue if it is already in.

```

const int INF = 1000000000;
vector<vector<pair<int, int>>> adj;

bool spfa(int s, vector<int>& d) {
    int n = adj.size();
    d.assign(n, INF);

```

```

vector<int> cnt(n, 0);
vector<bool> inqueue(n, false);
queue<int> q;

d[s] = 0;
q.push(s);
inqueue[s] = true;
while (!q.empty()) {
    int v = q.front();
    q.pop();
    inqueue[v] = false;

    for (auto edge : adj[v]) {
        int to = edge.first;
        int len = edge.second;

        if (d[v] + len < d[to]) {
            d[to] = d[v] + len;
            if (!inqueue[to]) {
                q.push(to);
                inqueue[to] = true;
                cnt[to]++;
                if (cnt[to] > n)
                    return false; // negative cycle
            }
        }
    }
}
return true;
}

```

Related problems in online judges

A list of tasks that can be solved using the Bellman-Ford algorithm:

- [E-OLYMP #1453 "Ford-Bellman"](#) [difficulty: low]
- [UVA #423 "MPI Maelstrom"](#) [difficulty: low]
- [UVA #534 "Frogger"](#) [difficulty: medium]
- [UVA #10099 "The Tourist Guide"](#) [difficulty: medium]
- [UVA #515 "King"](#) [difficulty: medium]
- [UVA 12519 - The Farnsworth Parabox](#)

See also the problem list in the article [Finding the negative cycle in a graph](#). * CSES - High Score * CSES - Cycle Finding

Contributors:

[code_master5](#) (53.48%)
 [wilwxk](#) (17.95%)
 [Jakob Kogler](#) (16.12%)
 [Bhuvnesh Jain](#) (3.66%)
 [Oleksandr Kulkov](#) (2.57%)
 [Rodion Gorkovenko](#) (2.56%)
[Aryaman](#) (0.73%)
 [Aman Dwivedi](#) (0.37%)
 [DiegoHDMGZ](#) (0.37%)
 [João Oliveira](#) (0.37%)
 [Kshitiz Srivastava](#) (0.37%)
 [Morass](#) (0.37%)
 [Oscar Rendón](#) (0.37%)
[Sachin Patel](#) (0.37%)
[prpr](#) (0.37%)

Original

0-1 BFS

It is well-known, that you can find the shortest paths between a single source and all other vertices in $O(|E|)$ using [Breadth First Search](#) in an **unweighted graph**, i.e. the distance is the minimal number of edges that you need to traverse from the source to another vertex. We can interpret such a graph also as a weighted graph, where every edge has the weight 1. If not all edges in graph have the same weight, then we need a more general algorithm, like [Dijkstra](#) which runs in $O(|V|^2 + |E|)$ or $O(|E| \log |V|)$ time.

However if the weights are more constrained, we can often do better. In this article we demonstrate how we can use BFS to solve the SSSP (single-source shortest path) problem in $O(|E|)$, if the weight of each edge is either 0 or 1.

Algorithm

We can develop the algorithm by closely studying Dijkstra's algorithm and thinking about the consequences that our special graph implies. The general form of Dijkstra's algorithm is (here a `set` is used for the priority queue):

```
d.assign(n, INF);
d[s] = 0;
set<pair<int, int>> q;
q.insert({0, s});
while (!q.empty()) {
    int v = q.begin()->second;
    q.erase(q.begin());

    for (auto edge : adj[v]) {
        int u = edge.first;
        int w = edge.second;

        if (d[v] + w < d[u]) {
            q.erase({d[u], u});
            d[u] = d[v] + w;
            q.insert({d[u], u});
        }
    }
}
```

We can notice that the difference between the distances between the source `s` and two other vertices in the queue differs by at most one. Especially, we know that $d[v] \leq d[u] \leq d[v] + 1$ for each $u \in Q$. The reason for this is, that we only add vertices with equal distance or with distance plus one to the queue during each iteration. Assuming there exists a u in the queue with $d[u] - d[v] > 1$, then u must have been insert in the queue via a different vertex t with $d[t] \geq d[u] - 1 > d[v]$. However this is impossible, since Dijkstra's algorithm iterates over the vertices in increasing order.

This means, that the order of the queue looks like this:

$$Q = \underbrace{\square \square \square}_{d[v]}, \dots, \underbrace{\square \square \square}_{d[v]}, \underbrace{\square \square \square}_{d[v]+1}, \dots, \underbrace{\square \square \square}_{d[v]+1}$$

This structure is so simple, that we don't need an actual priority queue, i.e. using a balanced binary tree would be an overkill. We can simply use a normal queue, and append new vertices at the beginning if the corresponding edge has weight 0, i.e. if $d[u] = d[v]$, or at the end if the edge has weight 1, i.e. if $d[u] = d[v] + 1$. This way the queue still remains sorted at all time.

```
vector<int> d(n, INF);
d[s] = 0;
deque<int> q;
q.push_front(s);
while (!q.empty()) {
    int v = q.front();
```

```

q.pop_front();
for (auto edge : adj[v]) {
    int u = edge.first;
    int w = edge.second;
    if (d[v] + w < d[u]) {
        d[u] = d[v] + w;
        if (w == 1)
            q.push_back(u);
        else
            q.push_front(u);
    }
}
}
}

```

Dial's algorithm

We can extend this even further if we allow the weights of the edges to be even bigger. If every edge in the graph has a weight $\leq k$, then the distances of vertices in the queue will differ by at most k from the distance of v to the source. So we can keep $k + 1$ buckets for the vertices in the queue, and whenever the bucket corresponding to the smallest distance gets empty, we make a cyclic shift to get the bucket with the next higher distance. This extension is called **Dial's algorithm**.

Practice problems

- [CodeChef - Chef and Reversing](#)
- [Labyrinth](#)
- [KATHTHI](#)
- [DoNotTurn](#)
- [Ocean Currents](#)
- [Olya and Energy Drinks](#)
- [Three States](#)
- [Colliding Traffic](#)
- [CHamber of Secrets](#)
- [Spiral Maximum](#)
- [Minimum Cost to Make at Least One Valid Path in a Grid](#)

Contributors:

[Jakob Kogler](#) (75.26%) [Salil Gokhale](#) (9.28%) [Oleksandr Kulkov](#) (5.15%) [P4R7H0 K2](#) (4.12%) [Tarun Khullar](#) (4.12%) [DIVYANSH DIXIT](#) (1.03%)
[knotri](#) (1.03%)

Translated

From: e-maxx.ru

D'Esopo-Pape algorithm

Given a graph with n vertices and m edges with weights w_i and a starting vertex v_0 . The task is to find the shortest path from the vertex v_0 to every other vertex.

The algorithm from D'Esopo-Pape will work faster than [Dijkstra's algorithm](#) and the [Bellman-Ford algorithm](#) in most cases, and will also work for negative edges. However not for negative cycles.

Description

Let the array d contain the shortest path lengths, i.e. d_i is the current length of the shortest path from the vertex v_0 to the vertex i . Initially this array is filled with infinity for every vertex, except $d_{v_0} = 0$. After the algorithm finishes, this array will contain the shortest distances.

Let the array p contain the current ancestors, i.e. p_i is the direct ancestor of the vertex i on the current shortest path from v_0 to i . Just like the array d , the array p changes gradually during the algorithm and at the end takes its final values.

Now to the algorithm. At each step three sets of vertices are maintained:

- M_0 - vertices, for which the distance has already been calculated (although it might not be the final distance)
- M_1 - vertices, for which the distance currently is calculated
- M_2 - vertices, for which the distance has not yet been calculated

The vertices in the set M_1 are stored in a bidirectional queue (deque).

At each step of the algorithm we take a vertex from the set M_1 (from the front of the queue). Let u be the selected vertex. We put this vertex u into the set M_0 . Then we iterate over all edges coming out of this vertex. Let v be the second end of the current edge, and w its weight.

- If v belongs to M_2 , then v is inserted into the set M_1 by inserting it at the back of the queue. d_v is set to $d_u + w$.
- If v belongs to M_1 , then we try to improve the value of d_v : $d_v = \min(d_v, d_u + w)$. Since v is already in M_1 , we don't need to insert it into M_1 and the queue.
- If v belongs to M_0 , and if d_v can be improved $d_v > d_u + w$, then we improve d_v and insert the vertex v back to the set M_1 , placing it at the beginning of the queue.

And of course, with each update in the array d we also have to update the corresponding element in the array p .

Implementation

We will use an array m to store in which set each vertex is currently.

```
struct Edge {
    int to, w;
};

int n;
vector<vector<Edge>> adj;

const int INF = 1e9;

void shortest_paths(int v0, vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<int> m(n, 2);
    deque<int> q;
    q.push_back(v0);
    p.assign(n, -1);
```

```
while (!q.empty()) {
    int u = q.front();
    q.pop_front();
    m[u] = 0;
    for (Edge e : adj[u]) {
        if (d[e.to] > d[u] + e.w) {
            d[e.to] = d[u] + e.w;
            p[e.to] = u;
            if (m[e.to] == 2) {
                m[e.to] = 1;
                q.push_back(e.to);
            } else if (m[e.to] == 0) {
                m[e.to] = 1;
                q.push_front(e.to);
            }
        }
    }
}
```

Complexity

The algorithm usually performs quite fast - in most cases, even faster than Dijkstra's algorithm. However there exist cases for which the algorithm takes exponential time, making it unsuitable in the worst-case. See discussions on [Stack Overflow](#) and [Codeforces](#) for reference.

Contributors:

[Jakob Kogler](#) (89.25%) [Oleksandr Kulkov](#) (7.53%) [Koustav Chowdhury](#) (2.15%) [jxu](#) (1.08%)

Translated

From: e-maxx.ru

Floyd-Warshall Algorithm

Given a directed or an undirected weighted graph G with n vertices. The task is to find the length of the shortest path d_{ij} between each pair of vertices i and j .

The graph may have negative weight edges, but no negative weight cycles.

If there is such a negative cycle, you can just traverse this cycle over and over, in each iteration making the cost of the path smaller. So you can make certain paths arbitrarily small, or in other words that shortest path is undefined. That automatically means that an undirected graph cannot have any negative weight edges, as such an edge forms already a negative cycle as you can move back and forth along that edge as long as you like.

This algorithm can also be used to detect the presence of negative cycles. The graph has a negative cycle if at the end of the algorithm, the distance from a vertex v to itself is negative.

This algorithm has been simultaneously published in articles by Robert Floyd and Stephen Warshall in 1962. However, in 1959, Bernard Roy published essentially the same algorithm, but its publication went unnoticed.

Description of the algorithm

The key idea of the algorithm is to partition the process of finding the shortest path between any two vertices to several incremental phases.

Let us number the vertices starting from 1 to n . The matrix of distances is $d[i][j]$.

Before k -th phase ($k = 1 \dots n$), $d[i][j]$ for any vertices i and j stores the length of the shortest path between the vertex i and vertex j , which contains only the vertices $\{1, 2, \dots, k-1\}$ as internal vertices in the path.

In other words, before k -th phase the value of $d[i][j]$ is equal to the length of the shortest path from vertex i to the vertex j , if this path is allowed to enter only the vertex with numbers smaller than k (the beginning and end of the path are not restricted by this property).

It is easy to make sure that this property holds for the first phase. For $k = 0$, we can fill matrix with $d[i][j] = w_{ij}$ if there exists an edge between i and j with weight w_{ij} and $d[i][j] = \infty$ if there doesn't exist an edge. In practice ∞ will be some high value. As we shall see later, this is a requirement for the algorithm.

Suppose now that we are in the k -th phase, and we want to compute the matrix $d[i][j]$ so that it meets the requirements for the $(k+1)$ -th phase. We have to fix the distances for some vertices pairs (i, j) . There are two fundamentally different cases:

- The shortest way from the vertex i to the vertex j with internal vertices from the set $\{1, 2, \dots, k\}$ coincides with the shortest path with internal vertices from the set $\{1, 2, \dots, k-1\}$.

In this case, $d[i][j]$ will not change during the transition.

- The shortest path with internal vertices from $\{1, 2, \dots, k\}$ is shorter.

This means that the new, shorter path passes through the vertex k . This means that we can split the shortest path between i and j into two paths: the path between i and k , and the path between k and j . It is clear that both this paths only use internal vertices of $\{1, 2, \dots, k-1\}$ and are the shortest such paths in that respect. Therefore we already have computed the lengths of those paths before, and we can compute the length of the shortest path between i and j as $d[i][k] + d[k][j]$.

Combining these two cases we find that we can recalculate the length of all pairs (i, j) in the k -th phase in the following way:

$$d_{\text{new}}[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

Thus, all the work that is required in the k -th phase is to iterate over all pairs of vertices and recalculate the length of the shortest path between them. As a result, after the n -th phase, the value $d[i][j]$ in the distance matrix is the length of the shortest path between i and j , or is ∞ if the

path between the vertices i and j does not exist.

A last remark - we don't need to create a separate distance matrix d_{new} for temporarily storing the shortest paths of the k -th phase, i.e. all changes can be made directly in the matrix d at any phase. In fact at any k -th phase we are at most improving the distance of any path in the distance matrix, hence we cannot worsen the length of the shortest path for any pair of the vertices that are to be processed in the $(k + 1)$ -th phase or later.

The time complexity of this algorithm is obviously $O(n^3)$.

Implementation

Let d is a 2D array of size $n \times n$, which is filled according to the 0-th phase as explained earlier. Also we will set $d[i][i] = 0$ for any i at the 0-th phase.

Then the algorithm is implemented as follows:

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

It is assumed that if there is no edge between any two vertices i and j , then the matrix at $d[i][j]$ contains a large number (large enough so that it is greater than the length of any path in this graph). Then this edge will always be unprofitable to take, and the algorithm will work correctly.

However if there are negative weight edges in the graph, special measures have to be taken. Otherwise the resulting values in matrix may be of the form $\infty - 1$, $\infty - 2$, etc., which, of course, still indicates that between the respective vertices doesn't exist a path. Therefore, if the graph has negative weight edges, it is better to write the Floyd-Warshall algorithm in the following way, so that it does not perform transitions using paths that don't exist.

```
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
        }
    }
}
```

Retrieving the sequence of vertices in the shortest path

It is easy to maintain additional information with which it will be possible to retrieve the shortest path between any two given vertices in the form of a sequence of vertices.

For this, in addition to the distance matrix d , a matrix of ancestors p must be maintained, which will contain the number of the phase where the shortest distance between two vertices was last modified. It is clear that the number of the phase is nothing more than a vertex in the middle of the desired shortest path. Now we just need to find the shortest path between vertices i and $p[i][j]$, and between $p[i][j]$ and j . This leads to a simple recursive reconstruction algorithm of the shortest path.

The case of real weights

If the weights of the edges are not integer but real, it is necessary to take the errors, which occur when working with float types, into account.

The Floyd-Warshall algorithm has the unpleasant effect, that the errors accumulate very quickly. In fact if there is an error in the first phase of δ , this error may propagate to the second iteration as 2δ , to the third iteration as 4δ , and so on.

To avoid this the algorithm can be modified to take the error ($\text{EPS} = \delta$) into account by using following comparison:

```
if (d[i][k] + d[k][j] < d[i][j] - EPS)
    d[i][j] = d[i][k] + d[k][j];
```

The case of negative cycles

Formally the Floyd-Warshall algorithm does not apply to graphs containing negative weight cycle(s). But for all pairs of vertices i and j for which there doesn't exist a path starting at i , visiting a negative cycle, and end at j , the algorithm will still work correctly.

For the pair of vertices for which the answer does not exist (due to the presence of a negative cycle in the path between them), the Floyd algorithm will store any number (perhaps highly negative, but not necessarily) in the distance matrix. However it is possible to improve the Floyd-Warshall algorithm, so that it carefully treats such pairs of vertices, and outputs them, for example as $-\text{INF}$.

This can be done in the following way: let us run the usual Floyd-Warshall algorithm for a given graph. Then a shortest path between vertices i and j does not exist, if and only if, there is a vertex t that is reachable from i and also from j , for which $d[t][t] < 0$.

In addition, when using the Floyd-Warshall algorithm for graphs with negative cycles, we should keep in mind that situations may arise in which distances can get exponentially fast into the negative. Therefore integer overflow must be handled by limiting the minimal distance by some value (e.g. $-\text{INF}$).

To learn more about finding negative cycles in a graph, see the separate article [Finding a negative cycle in the graph](#).

Practice Problems

- [UVA: Page Hopping](#)
- [SPOJ: Possible Friends](#)
- [CODEFORCES: Greg and Graph](#)
- [SPOJ: CHICAGO - 106 miles to Chicago](#)
- [UVA 10724 - Road Construction](#)
- [UVA 117 - The Postal Worker Rings Once](#)
- [Codeforces - Traveling Graph](#)
- [UVA - 1198 - The Geodetic Set Problem](#)
- [UVA - 10048 - Audiophobia](#)
- [UVA - 125 - Numbering Paths](#)
- [LOJ - Travel Company](#)
- [UVA 423 - MPI Maelstrom](#)
- [UVA 1416 - Warfare And Logistics](#)
- [UVA 1233 - USHER](#)
- [UVA 10793 - The Orc Attack](#)
- [UVA 10099 The Tourist Guide](#)
- [UVA 869 - Airline Comparison](#)
- [UVA 13211 - Geonosis](#)
- [SPOJ - Defend the Rohan](#)
- [Codeforces - Roads in Berland](#)
- [Codeforces - String Problem](#)
- [GYM - Manic Moving \(C\)](#)
- [SPOJ - Arbitrage](#)
- [UVA - 12179 - Randomly-priced Tickets](#)
- [LOJ - 1086 - Jogging Trails](#)
- [SPOJ - Ingredients](#)
- [CSES - Shortest Routes II](#)

Contributors:

Jakob Kogler (40.83%) code_master5 (30.18%) Morass (12.43%) Bhuvnesh Jain (3.55%) Oleksandr Kulkov (3.55%) Sunil Sangwan (2.96%)
 Bergur Snorrason (2.37%) RodionGork (1.18%) Aryaman (0.59%) Kartikay singh (0.59%) LUTLJS (0.59%) Mariia Mykhailova (0.59%) ondrahb (0.59%)

Translated From: e-maxx.ru

Number of paths of fixed length / Shortest paths of fixed length

The following article describes solutions to these two problems built on the same idea: reduce the problem to the construction of matrix and compute the solution with the usual matrix multiplication or with a modified multiplication.

Number of paths of a fixed length

We are given a directed, unweighted graph G with n vertices and we are given an integer k . The task is the following: for each pair of vertices (i, j) we have to find the number of paths of length k between these vertices. Paths don't have to be simple, i.e. vertices and edges can be visited any number of times in a single path.

We assume that the graph is specified with an adjacency matrix, i.e. the matrix G of size $n \times n$, where each element $G[i][j]$ equal to 1 if the vertex i is connected with j by an edge, and 0 if they are not connected by an edge. The following algorithm works also in the case of multiple edges: if some pair of vertices (i, j) is connected with m edges, then we can record this in the adjacency matrix by setting $G[i][j] = m$. Also the algorithm works if the graph contains loops (a loop is an edge that connect a vertex with itself).

It is obvious that the constructed adjacency matrix is the answer to the problem for the case $k = 1$. It contains the number of paths of length 1 between each pair of vertices.

We will build the solution iteratively: Let's assume we know the answer for some k . Here we describe a method how we can construct the answer for $k + 1$. Denote by C_k the matrix for the case k , and by C_{k+1} the matrix we want to construct. With the following formula we can compute every entry of C_{k+1} :

$$C_{k+1}[i][j] = \sum_{p=1}^n C_k[i][p] \cdot G[p][j]$$

It is easy to see that the formula computes nothing other than the product of the matrices C_k and G :

$$C_{k+1} = C_k \cdot G$$

Thus the solution of the problem can be represented as follows:

$$C_k = \underbrace{G \cdot G \cdot \dots \cdot G}_{k \text{ times}} = G^k$$

It remains to note that the matrix products can be raised to a high power efficiently using [Binary exponentiation](#). This gives a solution with $O(n^3 \log k)$ complexity.

Shortest paths of a fixed length

We are given a directed weighted graph G with n vertices and an integer k . For each pair of vertices (i, j) we have to find the length of the shortest path between i and j that consists of exactly k edges.

We assume that the graph is specified by an adjacency matrix, i.e. via the matrix G of size $n \times n$ where each element $G[i][j]$ contains the length of the edges from the vertex i to the vertex j . If there is no edge between two vertices, then the corresponding element of the matrix will be assigned to infinity ∞ .

It is obvious that in this form the adjacency matrix is the answer to the problem for $k = 1$. It contains the lengths of shortest paths between each pair of vertices, or ∞ if a path consisting of one edge doesn't exist.

Again we can build the solution to the problem iteratively: Let's assume we know the answer for some k . We show how we can compute the answer for $k + 1$. Let us denote L_k the matrix for k and L_{k+1} the matrix we want to build. Then the following formula computes each entry of L_{k+1} :

$$L_{k+1}[i][j] = \min_{p=1 \dots n} (L_k[i][p] + G[p][j])$$

When looking closer at this formula, we can draw an analogy with the matrix multiplication: in fact the matrix L_k is multiplied by the matrix G , the only difference is that instead in the multiplication operation we take the minimum instead of the sum, and the sum instead of the multiplication as the inner operation.

$$L_{k+1} = L_k \odot G,$$

where the operation $\odot$ is defined as follows:

$$A \odot B = C \iff C_{ij} = \min_{p=1 \dots n} (A_{ip} + B_{pj})$$

Thus the solution of the task can be represented using the modified multiplication:

$$L_k = \underbrace{G \odot \dots \odot G}_{k \text{ times}} = G^{\odot k}$$

It remains to note that we also can compute this exponentiation efficiently with [Binary exponentiation](#), because the modified multiplication is obviously associative. So also this solution has $O(n^3 \log k)$ complexity.

Generalization of the problems for paths with length up to k

The above solutions solve the problems for a fixed k . However the solutions can be adapted for solving problems for which the paths are allowed to contain no more than k edges.

This can be done by slightly modifying the input graph.

We duplicate each vertex: for each vertex v we create one more vertex v' and add the edge (v, v') and the loop (v', v') . The number of paths between i and j with at most k edges is the same number as the number of paths between i and j' with exactly $k + 1$ edges, since there is a bijection that maps every path $[p_0 = i, p_1, \dots, p_{m-1}, p_m = j]$ of length $m \leq k$ to the path $[p_0 = i, p_1, \dots, p_{m-1}, p_m = j, j', \dots, j']$ of length $k + 1$.

The same trick can be applied to compute the shortest paths with at most k edges. We again duplicate each vertex and add the two mentioned edges with weight 0.

Contributors:

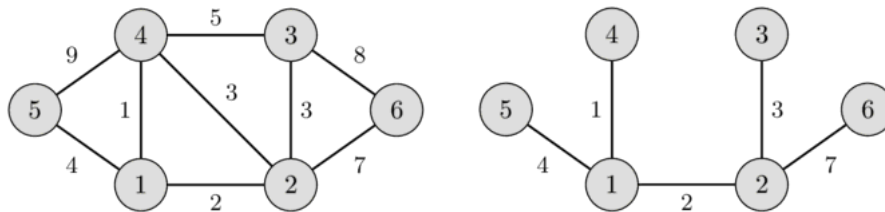
[Jakob Kogler](#) (90.32%) [Oleksandr Kulkov](#) (7.53%) [Shashank Sahu](#) (1.08%) [wiktorkuchta](#) (1.08%)

Translated From: e-maxx.ru

Minimum spanning tree - Prim's algorithm

Given a weighted, undirected graph G with n vertices and m edges. You want to find a spanning tree of this graph which connects all vertices and has the least weight (i.e. the sum of weights of edges is minimal). A spanning tree is a set of edges such that any vertex can reach any other by exactly one simple path. The spanning tree with the least weight is called a minimum spanning tree.

In the left image you can see a weighted undirected graph, and in the right image you can see the corresponding minimum spanning tree.



It is easy to see that any spanning tree will necessarily contain $n - 1$ edges.

This problem appears quite naturally in a lot of problems. For instance in the following problem: there are n cities and for each pair of cities we are given the cost to build a road between them (or we know that is physically impossible to build a road between them). We have to build roads, such that we can get from each city to every other city, and the cost for building all roads is minimal.

Prim's Algorithm

This algorithm was originally discovered by the Czech mathematician Vojtěch Jarník in 1930. However this algorithm is mostly known as Prim's algorithm after the American mathematician Robert Clay Prim, who rediscovered and republished it in 1957. Additionally Edsger Dijkstra published this algorithm in 1959.

Algorithm description

Here we describe the algorithm in its simplest form. The minimum spanning tree is built gradually by adding edges one at a time. At first the spanning tree consists only of a single vertex (chosen arbitrarily). Then the minimum weight edge outgoing from this vertex is selected and added to the spanning tree. After that the spanning tree already consists of two vertices. Now select and add the edge with the minimum weight that has one end in an already selected vertex (i.e. a vertex that is already part of the spanning tree), and the other end in an unselected vertex. And so on, i.e. every time we select and add the edge with minimal weight that connects one selected vertex with one unselected vertex. The process is repeated until the spanning tree contains all vertices (or equivalently until we have $n - 1$ edges).

In the end the constructed spanning tree will be minimal. If the graph was originally not connected, then there doesn't exist a spanning tree, so the number of selected edges will be less than $n - 1$.

Proof

Let the graph G be connected, i.e. the answer exists. We denote by T the resulting graph found by Prim's algorithm, and by S the minimum spanning tree. Obviously T is indeed a spanning tree and a subgraph of G . We only need to show that the weights of S and T coincide.

Consider the first time in the algorithm when we add an edge to T that is not part of S . Let us denote this edge with e , its ends by a and b , and the set of already selected vertices as V ($a \in V$ and $b \notin V$, or vice versa).

In the minimal spanning tree S the vertices a and b are connected by some path P . On this path we can find an edge f such that one end of f lies in V and the other end doesn't. Since the algorithm chose e instead of f , it means that the weight of f is greater or equal to the weight of e .

We add the edge e to the minimum spanning tree S and remove the edge f . By adding e we created a cycle, and since f was also part of the only cycle, by removing it the resulting graph is again free of cycles. And because we only removed an edge from a cycle, the resulting graph is still connected.

The resulting spanning tree cannot have a larger total weight, since the weight of e was not larger than the weight of f , and it also cannot have a smaller weight since S was a minimum spanning tree. This means that by replacing the edge f with e we generated a different minimum spanning tree. And e has to have the same weight as f .

Thus all the edges we pick in Prim's algorithm have the same weights as the edges of any minimum spanning tree, which means that Prim's algorithm really generates a minimum spanning tree.

Implementation

The complexity of the algorithm depends on how we search for the next minimal edge among the appropriate edges. There are multiple approaches leading to different complexities and different implementations.

Trivial implementations: $O(nm)$ and $O(n^2 + m \log n)$

If we search the edge by iterating over all possible edges, then it takes $O(m)$ time to find the edge with the minimal weight. The total complexity will be $O(nm)$. In the worst case this is $O(n^3)$, really slow.

This algorithm can be improved if we only look at one edge from each already selected vertex. For example we can sort the edges from each vertex in ascending order of their weights, and store a pointer to the first valid edge (i.e. an edge that goes to a non-selected vertex). Then after finding and selecting the minimal edge, we update the pointers. This gives a complexity of $O(n^2 + m)$, and for sorting the edges an additional $O(m \log n)$, which gives the complexity $O(n^2 \log n)$ in the worst case.

Below we consider two slightly different algorithms, one for dense and one for sparse graphs, both with a better complexity.

Dense graphs: $O(n^2)$

We approach this problem from a different angle: for every not yet selected vertex we will store the minimum edge to an already selected vertex.

Then during a step we only have to look at these minimum weight edges, which will have a complexity of $O(n)$.

After adding an edge some minimum edge pointers have to be recalculated. Note that the weights only can decrease, i.e. the minimal weight edge of every not yet selected vertex might stay the same, or it will be updated by an edge to the newly selected vertex. Therefore this phase can also be done in $O(n)$.

Thus we received a version of Prim's algorithm with the complexity $O(n^2)$.

In particular this implementation is very convenient for the Euclidean Minimum Spanning Tree problem: we have n points on a plane and the distance between each pair of points is the Euclidean distance between them, and we want to find a minimum spanning tree for this complete graph. This task can be solved by the described algorithm in $O(n^2)$ time and $O(n)$ memory, which is not possible with [Kruskal's algorithm](#).

```
int n;
vector<vector<int>> adj; // adjacency matrix of graph
const int INF = 1000000000; // weight INF means there is no edge

struct Edge {
    int w = INF, to = -1;
};

void prim() {
    int total_weight = 0;
    vector<bool> selected(n, false);
    vector<Edge> min_e(n);
    min_e[0].w = 0;

    for (int i=0; i<n; ++i) {
        int v = -1;
        for (int j = 0; j < n; ++j) {
            if (!selected[j] && (v == -1 || min_e[j].w < min_e[v].w))
                v = j;
        }
    }
}
```

```

    if (min_e[v].w == INF) {
        cout << "No MST!" << endl;
        exit(0);
    }

    selected[v] = true;
    total_weight += min_e[v].w;
    if (min_e[v].to != -1)
        cout << v << " " << min_e[v].to << endl;

    for (int to = 0; to < n; ++to) {
        if (adj[v][to] < min_e[to].w)
            min_e[to] = {adj[v][to], v};
    }
}

cout << total_weight << endl;
}

```

The adjacency matrix `adj[][]` of size $n \times n$ stores the weights of the edges, and it uses the weight `INF` if there doesn't exist an edge between two vertices. The algorithm uses two arrays: the flag `selected[]`, which indicates which vertices we already have selected, and the array `min_e[]` which stores the edge with minimal weight to a selected vertex for each not-yet-selected vertex (it stores the weight and the end vertex). The algorithm does n steps, in each iteration the vertex with the smallest edge weight is selected, and the `min_e[]` of all other vertices gets updated.

Sparse graphs: $O(m \log n)$

In the above described algorithm it is possible to interpret the operations of finding the minimum and modifying some values as set operations. These two classical operations are supported by many data structure, for example by `set` in C++ (which are implemented via red-black trees).

The main algorithm remains the same, but now we can find the minimum edge in $O(\log n)$ time. On the other hand recomputing the pointers will now take $O(n \log n)$ time, which is worse than in the previous algorithm.

But when we consider that we only need to update $O(m)$ times in total, and perform $O(n)$ searches for the minimal edge, then the total complexity will be $O(m \log n)$. For sparse graphs this is better than the above algorithm, but for dense graphs this will be slower.

```

const int INF = 1000000000;

struct Edge {
    int w = INF, to = -1;
    bool operator<(Edge const& other) const {
        return make_pair(w, to) < make_pair(other.w, other.to);
    }
};

int n;
vector<vector<Edge>> adj;

void prim() {
    int total_weight = 0;
    vector<Edge> min_e(n);
    min_e[0].w = 0;
    set<Edge> q;
    q.insert({0, 0});
    vector<bool> selected(n, false);
    for (int i = 0; i < n; ++i) {
        if (q.empty()) {
            cout << "No MST!" << endl;
            exit(0);
        }

        int v = q.begin()->to;
        selected[v] = true;
        total_weight += q.begin()->w;
        q.erase(q.begin());

        if (min_e[v].to != -1)
            cout << v << " " << min_e[v].to << endl;

        for (Edge e : adj[v]) {
            if (!selected[e.to] && e.w < min_e[e.to].w) {
                q.erase({min_e[e.to].w, e.to});
                min_e[e.to] = {e.w, v};
            }
        }
    }
}

```

```

        q.insert({e.w, e.to});
    }
}

cout << total_weight << endl;
}

```

Here the graph is represented via a adjacency list `adj[]`, where `adj[v]` contains all edges (in form of weight and target pairs) for the vertex `v`. `min_e[v]` will store the weight of the smallest edge from vertex `v` to an already selected vertex (again in the form of a weight and target pair). In addition the queue `q` is filled with all not yet selected vertices in the order of increasing weights `min_e`. The algorithm does `n` steps, on each of which it selects the vertex `v` with the smallest weight `min_e` (by extracting it from the beginning of the queue), and then looks through all the edges from this vertex and updates the values in `min_e` (during an update we also need to also remove the old edge from the queue `q` and put in the new edge).

Contributors:

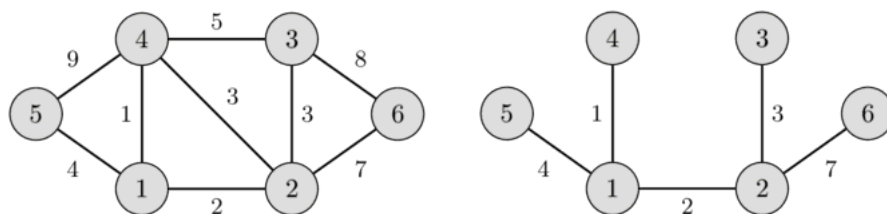
[Jakob Kogler](#) (92.52%)
 [Oleksandr Kulkov](#) (2.8%)
 [Michael Hayter](#) (1.87%)
 [Avinash Pandey](#) (0.93%)
 [Aleksandr Mishukhin](#) (0.47%)
 [Ekanshi Agrawal](#) (0.47%)
[Jeong-yeop \(Terence\) Kim](#) (0.47%)
 [Naman Bhalla](#) (0.47%)

Translated From: e-maxx.ru

Minimum spanning tree - Kruskal's algorithm

Given a weighted undirected graph. We want to find a subtree of this graph which connects all vertices (i.e. it is a spanning tree) and has the least weight (i.e. the sum of weights of all the edges is minimum) of all possible spanning trees. This spanning tree is called a minimum spanning tree.

In the left image you can see a weighted undirected graph, and in the right image you can see the corresponding minimum spanning tree.



This article will discuss few important facts associated with minimum spanning trees, and then will give the simplest implementation of Kruskal's algorithm for finding minimum spanning tree.

Properties of the minimum spanning tree

- A minimum spanning tree of a graph is unique, if the weight of all the edges are distinct. Otherwise, there may be multiple minimum spanning trees. (Specific algorithms typically output one of the possible minimum spanning trees).
- Minimum spanning tree is also the tree with minimum product of weights of edges. (It can be easily proved by replacing the weights of all edges with their logarithms)
- In a minimum spanning tree of a graph, the maximum weight of an edge is the minimum possible from all possible spanning trees of that graph. (This follows from the validity of Kruskal's algorithm).
- The maximum spanning tree (spanning tree with the sum of weights of edges being maximum) of a graph can be obtained similarly to that of the minimum spanning tree, by changing the signs of the weights of all the edges to their opposite and then applying any of the minimum spanning tree algorithm.

Kruskal's algorithm

This algorithm was described by Joseph Bernard Kruskal, Jr. in 1956.

Kruskal's algorithm initially places all the nodes of the original graph isolated from each other, to form a forest of single node trees, and then gradually merges these trees, combining at each iteration any two of all the trees with some edge of the original graph. Before the execution of the algorithm, all edges are sorted by weight (in non-decreasing order). Then begins the process of unification: pick all edges from the first to the last (in sorted order), and if the ends of the currently picked edge belong to different subtrees, these subtrees are combined, and the edge is added to the answer. After iterating through all the edges, all the vertices will belong to the same sub-tree, and we will get the answer.

The simplest implementation

The following code directly implements the algorithm described above, and is having $O(M \log M + N^2)$ time complexity. Sorting edges requires $O(M \log N)$ (which is the same as $O(M \log M)$) operations. Information regarding the subtree to which a vertex belongs is maintained with the help of an array `tree_id[]` - for each vertex `v`, `tree_id[v]` stores the number of the tree, to which `v` belongs. For each edge, whether it belongs to the ends of different trees, can be determined in $O(1)$. Finally, the union of the two trees is carried out in $O(N)$ by a simple pass through `tree_id[]` array. Given that the total number of merge operations is $N - 1$, we obtain the asymptotic behavior of $O(M \log N + N^2)$.

```

struct Edge {
    int u, v, weight;
    bool operator<(Edge const& other) {
        return weight < other.weight;
    }
};

int n;
vector<Edge> edges;

int cost = 0;
vector<int> tree_id(n);
vector<Edge> result;
for (int i = 0; i < n; i++)
    tree_id[i] = i;

sort(edges.begin(), edges.end());

for (Edge e : edges) {
    if (tree_id[e.u] != tree_id[e.v]) {
        cost += e.weight;
        result.push_back(e);

        int old_id = tree_id[e.u], new_id = tree_id[e.v];
        for (int i = 0; i < n; i++) {
            if (tree_id[i] == old_id)
                tree_id[i] = new_id;
        }
    }
}

```

Proof of correctness

Why does Kruskal's algorithm give us the correct result?

If the original graph was connected, then also the resulting graph will be connected. Because otherwise there would be two components that could be connected with at least one edge. Though this is impossible, because Kruskal would have chosen one of these edges, since the ids of the components are different. Also the resulting graph doesn't contain any cycles, since we forbid this explicitly in the algorithm. Therefore the algorithm generates a spanning tree.

So why does this algorithm give us a minimum spanning tree?

We can show the proposal "if F is a set of edges chosen by the algorithm at any stage in the algorithm, then there exists a MST that contains all edges of F " using induction.

The proposal is obviously true at the beginning, the empty set is a subset of any MST.

Now let's assume F is some edge set at any stage of the algorithm, T is a MST containing F and e is the new edge we want to add using Kruskal.

If e generates a cycle, then we don't add it, and so the proposal is still true after this step.

In case that T already contains e , the proposal is also true after this step.

In case T doesn't contain the edge e , then $T + e$ will contain a cycle C . This cycle will contain at least one edge f , that is not in F . The set of edges $T - f + e$ will also be a spanning tree. Notice that the weight of f cannot be smaller than the weight of e , because otherwise Kruskal would have chosen f earlier. It also cannot have a bigger weight, since that would make the total weight of $T - f + e$ smaller than the total weight of T , which is impossible since T is already a MST. This means that the weight of e has to be the same as the weight of f . Therefore $T - f + e$ is also a MST, and it contains all edges from $F + e$. So also here the proposal is still fulfilled after the step.

This proves the proposal. Which means that after iterating over all edges the resulting edge set will be connected, and will be contained in a MST, which means that it has to be a MST already.

Improved implementation

We can use the **Disjoint Set Union (DSU)** data structure to write a faster implementation of the Kruskal's algorithm with the time complexity of about $O(M \log N)$. [This article](#) details such an approach.

Practice Problems

- [SPOJ - Koicost](#)
- [SPOJ - MaryBMW](#)
- [Codechef - Fullmetal Alchemist](#)
- [Codeforces - Edges in MST](#)
- [UVA 12176 - Bring Your Own Horse](#)
- [UVA 10600 - ACM Contest and Blackout](#)
- [UVA 10724 - Road Construction](#)
- [Hackerrank - Roads in HackerLand](#)
- [UVA 11710 - Expensive subway](#)
- [Codechef - Chefland and Electricity](#)
- [UVA 10307 - Killing Aliens in Borg Maze](#)
- [Codeforces - Flea](#)
- [Codeforces - Igon in Museum](#)
- [Codeforces - Hongcow Builds a Nation](#)
- [UVA - 908 - Re-connecting Computer Sites](#)
- [UVA 1208 - Oreon](#)
- [UVA 1235 - Anti Brute Force Lock](#)
- [UVA 10034 - Freckles](#)
- [UVA 11228 - Transportation system](#)
- [UVA 11631 - Dark roads](#)
- [UVA 11733 - Airports](#)
- [UVA 11747 - Heavy Cycle Edges](#)
- [SPOJ - Blinet](#)
- [SPOJ - Help the Old King](#)
- [Codeforces - Hierarchy](#)
- [SPOJ - Modems](#)
- [CSES - Road Reparation](#)
- [CSES - Road Construction](#)

Contributors:

[Jakob Kogler](#) (46.15%) [code_master5](#) (23.78%) [Morass](#) (14.69%) [Oleksandr Kulkov](#) (4.2%) [Mariia Mykhailova](#) (3.5%) [Bhuvnesh Jain](#) (2.8%)
[Aryaman](#) (1.4%) [wiktorkuchta](#) (1.4%) [Max](#) (0.7%) [Rodion Gorkovenko](#) (0.7%) [Tanmay Sinha](#) (0.7%)

Translated From: e-maxx.ru

Minimum spanning tree - Kruskal with Disjoint Set Union

For an explanation of the MST problem and the Kruskal algorithm, first see the [main article on Kruskal's algorithm](#).

In this article we will consider the data structure "[Disjoint Set Union](#)" for implementing Kruskal's algorithm, which will allow the algorithm to achieve the time complexity of $O(M \log N)$.

Description

Just as in the simple version of the Kruskal algorithm, we sort all the edges of the graph in non-decreasing order of weights. Then put each vertex in its own tree (i.e. its set) via calls to the `make_set` function - it will take a total of $O(N)$. We iterate through all the edges (in sorted order) and for each edge determine whether the ends belong to different trees (with two `find_set` calls in $O(1)$ each). Finally, we need to perform the union of the two trees (sets), for which the DSU `union_sets` function will be called - also in $O(1)$. So we get the total time complexity of $O(M \log N + N + M) = O(M \log N)$.

Implementation

Here is an implementation of Kruskal's algorithm with Union by Rank.

```
vector<int> parent, rank;

void make_set(int v) {
    parent[v] = v;
    rank[v] = 0;
}

int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}

void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (rank[a] < rank[b])
            swap(a, b);
        parent[b] = a;
        if (rank[a] == rank[b])
            rank[a]++;
    }
}

struct Edge {
    int u, v, weight;
    bool operator<(Edge const& other) {
        return weight < other.weight;
    }
};

int n;
vector<Edge> edges;

int cost = 0;
vector<Edge> result;
parent.resize(n);
rank.resize(n);
for (int i = 0; i < n; i++)
    make_set(i);
```

```
sort(edges.begin(), edges.end());

for (Edge e : edges) {
    if (find_set(e.u) != find_set(e.v)) {
        cost += e.weight;
        result.push_back(e);
        union_sets(e.u, e.v);
    }
}
```

Notice: since the MST will contain exactly $N - 1$ edges, we can stop the for loop once we found that many.

Practice Problems

See [main article on Kruskal's algorithm](#) for the list of practice problems on this topic.

Contributors:

[Jakob Kogler](#) (57.83%) [code_master5](#) (22.89%) [Oleksandr Kulkov](#) (7.23%) [Mariia Mykhailova](#) (3.61%) [Amandeep Singh](#) (2.41%) [Bhuvnesh Jain](#) (2.41%)
[Rodion Gorkovenko](#) (2.41%) [Chaman Agrawal](#) (1.2%)

Original

Second Best Minimum Spanning Tree

A Minimum Spanning Tree T is a tree for the given graph G which spans over all vertices of the given graph and has the minimum weight sum of all the edges, from all the possible spanning trees. A second best MST T' is a spanning tree, that has the second minimum weight sum of all the edges, from all the possible spanning trees of the graph G .

Observation

Let T be the Minimum Spanning Tree of a graph G . It can be observed, that the second best Minimum Spanning Tree differs from T by only one edge replacement. (For a proof of this statement refer to problem 23-1 [here](#)).

So we need to find an edge e_{new} which is in not in T , and replace it with an edge in T (let it be e_{old}) such that the new graph $T' = (T \cup \{e_{\text{new}}\}) \setminus \{e_{\text{old}}\}$ is a spanning tree and the weight difference $(e_{\text{new}} - e_{\text{old}})$ is minimum.

Using Kruskal's Algorithm

We can use Kruskal's algorithm to find the MST first, and then just try to remove a single edge from it and replace it with another.

1. Sort the edges in $O(E \log E)$, then find a MST using Kruskal in $O(E)$.
2. For each edge in the MST (we will have $V - 1$ edges in it) temporarily exclude it from the edge list so that it cannot be chosen.
3. Then, again try to find a MST in $O(E)$ using the remaining edges.
4. Do this for all the edges in MST, and take the best of all.

Note: we don't need to sort the edges again in for Step 3.

So, the overall time complexity will be $O(E \log V + E + VE) = O(VE)$.

Modeling into a Lowest Common Ancestor (LCA) problem

In the previous approach we tried all possibilities of removing one edge of the MST. Here we will do the exact opposite. We try to add every edge that is not already in the MST.

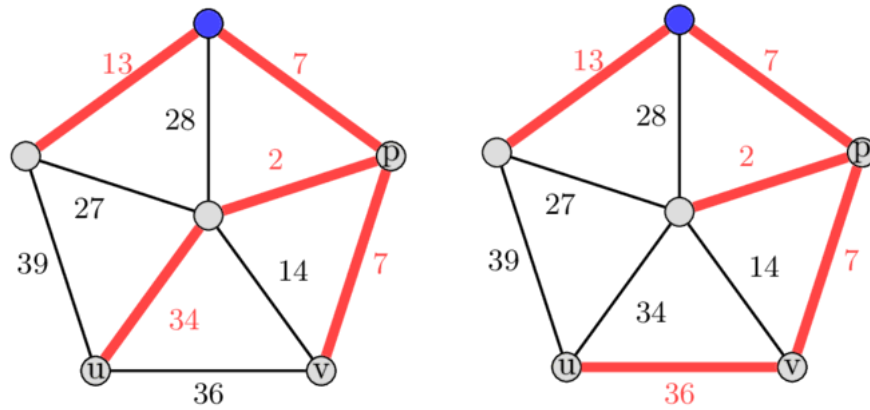
1. Sort the edges in $O(E \log E)$, then find a MST using Kruskal in $O(E)$.
2. For each edge e not already in the MST, temporarily add it to the MST, creating a cycle. The cycle will pass through the LCA.
3. Find the edge k with maximal weight in the cycle that is not equal to e , by following the parents of the nodes of edge e , up to the LCA.
4. Remove k temporarily, creating a new spanning tree.
5. Compute the weight difference $\delta = \text{weight}(e) - \text{weight}(k)$, and remember it together with the changed edge.
6. Repeat step 2 for all other edges, and return the spanning tree with the smallest weight difference to the MST.

The time complexity of the algorithm depends on how we compute the k s, which are the maximum weight edges in step 2 of this algorithm. One way to compute them efficiently in $O(E \log V)$ is to transform the problem into a Lowest Common Ancestor (LCA) problem.

We will preprocess the LCA by rooting the MST and will also compute the maximum edge weights for each node on the paths to their ancestors. This can be done using [Binary Lifting](#) for LCA.

The final time complexity of this approach is $O(E \log V)$.

For example:



In the image left is the MST and right is the second best MST.

In the given graph suppose we root the MST at the blue vertex on the top, and then run our algorithm by start picking the edges not in MST. Let the edge picked first be the edge (u, v) with weight 36. Adding this edge to the tree forms a cycle $36 - 7 - 2 - 34$.

Now we will find the maximum weight edge in this cycle by finding the $LCA(u, v) = p$. We compute the maximum weight edge on the paths from u to p and from v to p . Note: the $LCA(u, v)$ can also be equal to u or v in some case. In this example we will get the edge with weight 34 as maximum edge weight in the cycle. By removing the edge we get a new spanning tree, that has a weight difference of only 2.

After doing this also with all other edges that are not part of the initial MST, we can see that this spanning tree was also the second best spanning tree overall. Choosing the edge with weight 14 will increase the weight of the tree by 7, choosing the edge with weight 27 increases it by 14, choosing the edge with weight 28 increases it by 21, and choosing the edge with weight 39 will increase the tree by 5.

Implementation

```
struct edge {
    int s, e, w, id;
    bool operator<(const struct edge& other) { return w < other.w; }
};
typedef struct edge Edge;

const int N = 2e5 + 5;
long long res = 0, ans = 1e18;
int n, m, a, b, w, id, l = 21;
vector<Edge> edges;
vector<int> h(N, 0), parent(N, -1), size(N, 0), present(N, 0);
vector<vector<pair<int, int>>> adj(N), dp(N, vector<pair<int, int>>(1));
vector<vector<int>> up(N, vector<int>(1, -1));

pair<int, int> combine(pair<int, int> a, pair<int, int> b) {
    vector<int> v = {a.first, a.second, b.first, b.second};
    int topTwo = -3, topOne = -2;
    for (int c : v) {
        if (c > topOne) {
            topTwo = topOne;
            topOne = c;
        } else if (c > topTwo && c < topOne) {
            topTwo = c;
        }
    }
    return {topOne, topTwo};
}

void dfs(int u, int par, int d) {
    h[u] = 1 + h[par];
    up[u][0] = par;
    dp[u][0] = {d, -1};
    for (auto v : adj[u]) {
        if (v.first != par) {
            dfs(v.first, u, v.second);
        }
    }
}
```

```

}

pair<int, int> lca(int u, int v) {
    pair<int, int> ans = {-2, -3};
    if (h[u] < h[v]) {
        swap(u, v);
    }
    for (int i = 1 - 1; i >= 0; i--) {
        if (h[u] - h[v] >= (1 << i)) {
            ans = combine(ans, dp[u][i]);
            u = up[u][i];
        }
    }
    if (u == v) {
        return ans;
    }
    for (int i = 1 - 1; i >= 0; i--) {
        if (up[u][i] != -1 && up[v][i] != -1 && up[u][i] != up[v][i]) {
            ans = combine(ans, combine(dp[u][i], dp[v][i]));
            u = up[u][i];
            v = up[v][i];
        }
    }
    ans = combine(ans, combine(dp[u][0], dp[v][0]));
    return ans;
}

int main(void) {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        size[i] = 1;
    }
    for (int i = 1; i <= m; i++) {
        cin >> a >> b >> w; // 1-indexed
        edges.push_back({a, b, w, i - 1});
    }
    sort(edges.begin(), edges.end());
    for (int i = 0; i <= m - 1; i++) {
        a = edges[i].s;
        b = edges[i].e;
        w = edges[i].w;
        id = edges[i].id;
        if (unite_set(a, b)) {
            adj[a].emplace_back(b, w);
            adj[b].emplace_back(a, w);
            present[id] = 1;
            res += w;
        }
    }
    dfs(1, 0, 0);
    for (int i = 1; i <= 1 - 1; i++) {
        for (int j = 1; j <= n; ++j) {
            if (up[j][i - 1] != -1) {
                int v = up[j][i - 1];
                up[j][i] = up[v][i - 1];
                dp[j][i] = combine(dp[j][i - 1], dp[v][i - 1]);
            }
        }
    }
    for (int i = 0; i <= m - 1; i++) {
        id = edges[i].id;
        w = edges[i].w;
        if (!present[id]) {
            auto rem = lca(edges[i].s, edges[i].e);
            if (rem.first != w) {
                if (ans > res + w - rem.first) {
                    ans = res + w - rem.first;
                }
            } else if (rem.second != -1) {
                if (ans > res + w - rem.second) {
                    ans = res + w - rem.second;
                }
            }
        }
    }
    cout << ans << "\n";
    return 0;
}

```

References

1. Competitive Programming-3, by Steven Halim
2. web.mit.edu

Problems

- [Codeforces - Minimum spanning tree for each edge](#)

Contributors:

[gampu](#) (62.25%) [Amandeep Singh](#) (30.39%) [Michael Hayter](#) (2.45%) [Oleksandr Kulkov](#) (2.45%) [Jakob Kogler](#) (0.98%) [erikoui](#) (0.98%)
[Bergur Snorrason](#) (0.49%)

Translated

From: e-maxx.ru

Kirchhoff's theorem. Finding the number of spanning trees

Problem: You are given a connected undirected graph (with possible multiple edges) represented using an adjacency matrix. Find the number of different spanning trees of this graph.

The following formula was proven by Kirchhoff in 1847.

Kirchhoff's matrix tree theorem

Let A be the adjacency matrix of the graph: $A_{u,v}$ is the number of edges between u and v . Let D be the degree matrix of the graph: a diagonal matrix with $D_{u,u}$ being the degree of vertex u (including multiple edges and loops - edges which connect vertex u with itself).

The Laplacian matrix of the graph is defined as $L = D - A$. According to Kirchhoff's theorem, all cofactors of this matrix are equal to each other, and they are equal to the number of spanning trees of the graph. The (i, j) cofactor of a matrix is the product of $(-1)^{i+j}$ with the determinant of the matrix that you get after removing the i -th row and j -th column. So you can, for example, delete the last row and last column of the matrix L , and the absolute value of the determinant of the resulting matrix will give you the number of spanning trees.

The determinant of the matrix can be found in $O(N^3)$ by using the [Gaussian method](#).

The proof of this theorem is quite difficult and is not presented here; for an outline of the proof and variations of the theorem for graphs without multiple edges and for directed graphs refer to [Wikipedia](#).

Relation to Kirchhoff's circuit laws

Kirchhoff's matrix tree theorem and Kirchhoff's laws for electrical circuit are related in a beautiful way. It is possible to show (using Ohm's law and Kirchhoff's first law) that resistance R_{ij} between two points of the circuit i and j is

$$R_{ij} = \frac{L^{(i,j)}}{|L^j|}.$$

Here the matrix L is obtained from the matrix of inverse resistances A ($A_{i,j}$ is inverse of the resistance of the conductor between points i and j) using the procedure described in Kirchhoff's matrix tree theorem. T^j is the matrix with row and column j removed, $T^{(i,j)}$ is the matrix with two rows and two columns i and j removed.

Kirchhoff's theorem gives this formula geometric meaning.

Practice Problems

- [CODECHEF: Roads in Stars](#)
- [SPOJ: Maze](#)
- [CODECHEF: Complement Spanning Trees](#)

Contributors:

[Jakob Kogler](#) (34.15%) [thanhtnguyen](#) (21.95%) [Mariia Mykhailova](#) (17.07%) [Oleksandr Kulkov](#) (14.63%) [Sunil Sangwan](#) (9.76%) [Avinash Pandey](#) (2.44%)

Translated From: e-maxx.ru

Prüfer code

In this article we will look at the so-called **Prüfer code** (or Prüfer sequence), which is a way of encoding a labeled tree into a sequence of numbers in a unique way.

With the help of the Prüfer code we will prove **Cayley's formula** (which specified the number of spanning trees in a complete graph). Also we show the solution to the problem of counting the number of ways of adding edges to a graph to make it connected.

Note, we will not consider trees consisting of a single vertex - this is a special case in which multiple statements clash.

Prüfer code

The Prüfer code is a way of encoding a labeled tree with n vertices using a sequence of $n - 2$ integers in the interval $[0; n - 1]$. This encoding also acts as a **bijection** between all spanning trees of a complete graph and the numerical sequences.

Although using the Prüfer code for storing and operating on tree is impractical due the specification of the representation, the Prüfer codes are used frequently: mostly in solving combinatorial problems.

The inventor - Heinz Prüfer - proposed this code in 1918 as a proof for Cayley's formula.

Building the Prüfer code for a given tree

The Prüfer code is constructed as follows. We will repeat the following procedure $n - 2$ times: we select the leaf of the tree with the smallest number, remove it from the tree, and write down the number of the vertex that was connected to it. After $n - 2$ iterations there will only remain 2 vertices, and the algorithm ends.

Thus the Prüfer code for a given tree is a sequence of $n - 2$ numbers, where each number is the number of the connected vertex, i.e. this number is in the interval $[0, n - 1]$.

The algorithm for computing the Prüfer code can be implemented easily with $O(n \log n)$ time complexity, simply by using a data structure to extract the minimum (for instance `set` or `priority_queue` in C++), which contains a list of all the current leaves.

```
vector<vector<int>>> adj;

vector<int> pruefer_code() {
    int n = adj.size();
    set<int> leafs;
    vector<int> degree(n);
    vector<bool> killed(n, false);
    for (int i = 0; i < n; i++) {
        degree[i] = adj[i].size();
        if (degree[i] == 1)
            leafs.insert(i);
    }

    vector<int> code(n - 2);
    for (int i = 0; i < n - 2; i++) {
        int leaf = *leafs.begin();
        leafs.erase(leafs.begin());
        killed[leaf] = true;

        int v;
        for (int u : adj[leaf]) {
            if (!killed[u])
                v = u;
        }

        code[i] = v;
        if (--degree[v] == 1)
            leafs.insert(v);
    }
}
```

```

        leafs.insert(v);
    }
    return code;
}

```

However the construction can also be implemented in linear time. Such an approach is described in the next section.

Building the Prüfer code for a given tree in linear time

The essence of the algorithm is to use a **moving pointer**, which will always point to the current leaf vertex that we want to remove.

At first glance this seems impossible, because during the process of constructing the Prüfer code the leaf number can increase and decrease. However after a closer look, this is actually not true. The number of leaves will not increase. Either the number decreases by one (we remove one leaf vertex and don't gain a new one), or it stay the same (we remove one leaf vertex and gain another one). In the first case there is no other way than searching for the next smallest leaf vertex. In the second case, however, we can decide in $O(1)$ time, if we can continue using the vertex that became a new leaf vertex, or if we have to search for the next smallest leaf vertex. And in quite a lot of times we can continue with the new leaf vertex.

To do this we will use a variable `ptr`, which will indicate that in the set of vertices between 0 and `ptr` is at most one leaf vertex, namely the current one. All other vertices in that range are either already removed from the tree, or have still more than one adjacent vertices. At the same time we say, that we haven't removed any leaf vertices bigger than `ptr` yet.

This variable is already very helpful in the first case. After removing the current leaf node, we know that there cannot be a leaf node between 0 and `ptr`, therefore we can start the search for the next one directly at `ptr + 1`, and we don't have to start the search back at vertex 0. And in the second case, we can further distinguish two cases: Either the newly gained leaf vertex is smaller than `ptr`, then this must be the next leaf vertex, since we know that there are no other vertices smaller than `ptr`. Or the newly gained leaf vertex is bigger. But then we also know that it has to be bigger than `ptr`, and can start the search again at `ptr + 1`.

Even though we might have to perform multiple linear searches for the next leaf vertex, the pointer `ptr` only increases and therefore the time complexity in total is $O(n)$.

```

vector<vector<int>> adj;
vector<int> parent;

void dfs(int v) {
    for (int u : adj[v]) {
        if (u != parent[v]) {
            parent[u] = v;
            dfs(u);
        }
    }
}

vector<int> pruefer_code() {
    int n = adj.size();
    parent.resize(n);
    parent[n-1] = -1;
    dfs(n-1);

    int ptr = -1;
    vector<int> degree(n);
    for (int i = 0; i < n; i++) {
        degree[i] = adj[i].size();
        if (degree[i] == 1 && ptr == -1)
            ptr = i;
    }

    vector<int> code(n - 2);
    int leaf = ptr;
    for (int i = 0; i < n - 2; i++) {
        int next = parent[leaf];
        code[i] = next;
        if (--degree[next] == 1 && next < ptr) {
            leaf = next;
        } else {
            ptr++;
            while (degree[ptr] != 1)

```

```

        ptr++;
        leaf = ptr;
    }
}

return code;
}

```

In the code we first find for each its ancestor `parent[i]`, i.e. the ancestor that this vertex will have once we remove it from the tree. We can find this ancestor by rooting the tree at the vertex $n - 1$. This is possible because the vertex $n - 1$ will never be removed from the tree. We also compute the degree for each vertex. `ptr` is the pointer that indicates the minimum size of the remaining leaf vertices (except the current one `leaf`). We will either assign the current leaf vertex with `next`, if this one is also a leaf vertex and it is smaller than `ptr`, or we start a linear search for the smallest leaf vertex by increasing the pointer.

It can be easily seen, that this code has the complexity $O(n)$.

Some properties of the Prüfer code

- After constructing the Prüfer code two vertices will remain. One of them is the highest vertex $n - 1$, but nothing else can be said about the other one.
- Each vertex appears in the Prüfer code exactly a fixed number of times - its degree minus one. This can be easily checked, since the degree will get smaller every time we record its label in the code, and we remove it once the degree is 1. For the two remaining vertices this fact is also true.

Restoring the tree using the Prüfer code

To restore the tree it suffice to only focus on the property discussed in the last section. We already know the degree of all the vertices in the desired tree. Therefore we can find all leaf vertices, and also the first leaf that was removed in the first step (it has to be the smallest leaf). This leaf vertex was connected to the vertex corresponding to the number in the first cell of the Prüfer code.

Thus we found the first edge removed by when then the Prüfer code was generated. We can add this edge to the answer and reduce the degrees at both ends of the edge.

We will repeat this operation until we have used all numbers of the Prüfer code: we look for the minimum vertex with degree equal to 1, connect it with the next vertex from the Prüfer code, and reduce the degree.

In the end we only have two vertices left with degree equal to 1. These are the vertices that didn't got removed by the Prüfer code process. We connect them to get the last edge of the tree. One of them will always be the vertex $n - 1$.

This algorithm can be **implemented** easily in $O(n \log n)$: we use a data structure that supports extracting the minimum (for example `set<>` or `priority_queue<>` in C++) to store all the leaf vertices.

The following implementation returns the list of edges corresponding to the tree.

```

vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
    int n = code.size() + 2;
    vector<int> degree(n, 1);
    for (int i : code)
        degree[i]++;

    set<int> leaves;
    for (int i = 0; i < n; i++) {
        if (degree[i] == 1)
            leaves.insert(i);
    }

    vector<pair<int, int>> edges;
    for (int v : code) {
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());

        edges.emplace_back(leaf, v);
        if (--degree[v] == 1)
            leaves.insert(v);
    }
}

```

```
edges.emplace_back(*leaves.begin(), n-1);
return edges;
}
```

Restoring the tree using the Prüfer code in linear time

To obtain the tree in linear time we can apply the same technique used to obtain the Prüfer code in linear time.

We don't need a data structure to extract the minimum. Instead we can notice that, after processing the current edge, only one vertex becomes a leaf. Therefore we can either continue with this vertex, or we find a smaller one with a linear search by moving a pointer.

```
vector<pair<int, int>> pruefer_decode(vector<int> const& code) {
    int n = code.size() + 2;
    vector<int> degree(n, 1);
    for (int i : code)
        degree[i]++;

    int ptr = 0;
    while (degree[ptr] != 1)
        ptr++;
    int leaf = ptr;

    vector<pair<int, int>> edges;
    for (int v : code) {
        edges.emplace_back(leaf, v);
        if (--degree[v] == 1 && v < ptr) {
            leaf = v;
        } else {
            ptr++;
            while (degree[ptr] != 1)
                ptr++;
            leaf = ptr;
        }
    }
    edges.emplace_back(leaf, n-1);
    return edges;
}
```

Bijection between trees and Prüfer codes

For each tree there exists a Prüfer code corresponding to it. And for each Prüfer code we can restore the original tree.

It follows that also every Prüfer code (i.e. a sequence of $n - 2$ numbers in the range $[0; n - 1]$) corresponds to a tree.

Therefore all trees and all Prüfer codes form a bijection (a **one-to-one correspondence**).

Cayley's formula

Cayley's formula states that the **number of spanning trees in a complete labeled graph** with n vertices is equal to:

$$n^{n-2}$$

There are multiple proofs for this formula. Using the Prüfer code concept this statement comes without any surprise.

In fact any Prüfer code with $n - 2$ numbers from the interval $[0; n - 1]$ corresponds to some tree with n vertices. So we have n^{n-2} different such Prüfer codes. Since each such tree is a spanning tree of a complete graph with n vertices, the number of such spanning trees is also n^{n-2} .

Number of ways to make a graph connected

The concept of Prüfer codes are even more powerful. It allows to create a lot more general formulas than Cayley's formula.

In this problem we are given a graph with n vertices and m edges. The graph currently has k components. We want to compute the number of ways of adding $k - 1$ edges so that the graph becomes connected (obviously $k - 1$ is the minimum number necessary to make the graph connected).

Let us derive a formula for solving this problem.

We use $s_1, \dots, s_k$ for the sizes of the connected components in the graph. We cannot add edges within a connected component. Therefore it turns out that this problem is very similar to the search for the number of spanning trees of a complete graph with k vertices. The only difference is that each vertex has actually the size s_i : each edge connecting the vertex i , actually multiplies the answer by s_i .

Thus in order to calculate the number of possible ways it is important to count how often each of the k vertices is used in the connecting tree. To obtain a formula for the problem it is necessary to sum the answer over all possible degrees.

Let $d_1, \dots, d_k$ be the degrees of the vertices in the tree after connecting the vertices. The sum of the degrees is twice the number of edges:

$$\sum_{i=1}^k d_i = 2k - 2$$

If the vertex i has degree d_i , then it appears $d_i - 1$ times in the Prüfer code. The Prüfer code for a tree with k vertices has length $k - 2$. So the number of ways to choose a code with $k - 2$ numbers where the number i appears exactly $d_i - 1$ times is equal to the **multinomial coefficient**

$$\binom{k-2}{d_1-1, d_2-1, \dots, d_k-1} = \frac{(k-2)!}{(d_1-1)!(d_2-1)! \cdots (d_k-1)!}.$$

The fact that each edge adjacent to the vertex i multiplies the answer by s_i we receive the answer, assuming that the degrees of the vertices are $d_1, \dots, d_k$:

$$s_1^{d_1} \cdot s_2^{d_2} \cdots s_k^{d_k} \cdot \binom{k-2}{d_1-1, d_2-1, \dots, d_k-1}$$

To get the final answer we need to sum this for all possible ways to choose the degrees:

$$\sum_{\substack{d_i \geq 1 \\ \sum_{i=1}^k d_i = 2k-2}} s_1^{d_1} \cdot s_2^{d_2} \cdots s_k^{d_k} \cdot \binom{k-2}{d_1-1, d_2-1, \dots, d_k-1}$$

Currently this looks like a really horrible answer, however we can use the **multinomial theorem**, which says:

$$(x_1 + \cdots + x_m)^p = \sum_{\substack{c_i \geq 0 \\ \sum_{i=1}^m c_i = p}} x_1^{c_1} \cdot x_2^{c_2} \cdots x_m^{c_m} \cdot \binom{p}{c_1, c_2, \dots, c_m}$$

This look already pretty similar. To use it we only need to substitute with $e_i = d_i - 1$:

$$\sum_{\substack{e_i \geq 0 \\ \sum_{i=1}^k e_i = k-2}} s_1^{e_1+1} \cdot s_2^{e_2+1} \cdots s_k^{e_k+1} \cdot \binom{k-2}{e_1, e_2, \dots, e_k}$$

After applying the multinomial theorem we get the **answer to the problem**:

$$s_1 \cdot s_2 \cdots s_k \cdot (s_1 + s_2 + \cdots + s_k)^{k-2} = s_1 \cdot s_2 \cdots s_k \cdot n^{k-2}$$

By accident this formula also holds for $k = 1$.

Practice problems

- [UVA #10843 - Anne's game](#)
- [Timus #1069 - Prufer Code](#)
- [Codeforces - Clues](#)
- [Topcoder - TheCitiesAndRoadsDivTwo](#)

Contributors:

[Jakob Kogler](#) (98.17%) [Oleksandr Kulkov](#) (1.83%)

Translated

From: e-maxx.ru

Checking a graph for acyclicity and finding a cycle in $O(M)$

Consider a directed or undirected graph without loops and multiple edges. We have to check whether it is acyclic, and if it is not, then find any cycle.

We can solve this problem by using [Depth First Search](#) in $O(M)$ where M is number of edges.

Algorithm

We will run a series of DFS in the graph. Initially all vertices are colored white (0). From each unvisited (white) vertex, start the DFS, mark it gray (1) while entering and mark it black (2) on exit. If DFS moves to a gray vertex, then we have found a cycle (if the graph is undirected, the edge to parent is not considered). The cycle itself can be reconstructed using parent array.

Implementation

Here is an implementation for directed graph.

```
int n;
vector<vector<int>>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Cycle found: ";
    }
}
```

```

        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

```

Here is an implementation for undirected graph. Note that in the undirected version, if a vertex `v` gets colored black, it will never be visited again by the DFS. This is because we already explored all connected edges of `v` when we first visited it. The connected component containing `v` (after removing the edge between `v` and its parent) must be a tree, if the DFS has completed processing `v` without finding a cycle. So we don't even need to distinguish between gray and black states. Thus we can turn the char vector `color` into a boolean vector `visited`.

```

int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing vertex and its parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if (u == par) continue; // skipping edge to parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}

```

Practice problems:

- [AtCoder : Reachability in Functional Graph](#)
- [CSES : Round Trip](#)
- [CSES : Round Trip II](#)

Contributors:

[Ankit Sangwan](#) (32.84%)
 [Jakob Kogler](#) (25.37%)
 [Pranjal Jain](#) (24.63%)
 [Michael Tan](#) (9.7%)
 [Aryaman](#) (2.99%)
 [Oleksandr Kulkov](#) (2.24%)
[Tarun Khullar](#) (1.49%)
 [Nadezhda R](#) (0.75%)

Translated From: e-maxx.ru

Finding a negative cycle in the graph

You are given a directed weighted graph G with N vertices and M edges. Find any cycle of negative weight in it, if such a cycle exists.

In another formulation of the problem you have to find all pairs of vertices which have a path of arbitrarily small weight between them.

It is convenient to use different algorithms to solve these two variations of the problem, so we'll discuss both of them here.

Using Bellman-Ford algorithm

Bellman-Ford algorithm allows you to check whether there exists a cycle of negative weight in the graph, and if it does, find one of these cycles.

The details of the algorithm are described in the article on the [Bellman-Ford](#) algorithm. Here we'll describe only its application to this problem.

The standard implementation of Bellman-Ford looks for a negative cycle reachable from some starting vertex v ; however, the algorithm can be modified to just looking for any negative cycle in the graph. For this we need to put all the distance $d[i]$ to zero and not infinity – as if we are looking for the shortest path from all vertices simultaneously; the validity of the detection of a negative cycle is not affected.

Do N iterations of Bellman-Ford algorithm. If there were no changes on the last iteration, there is no cycle of negative weight in the graph. Otherwise take a vertex the distance to which has changed, and go from it via its ancestors until a cycle is found. This cycle will be the desired cycle of negative weight.

Implementation

```
struct Edge {
    int a, b, cost;
};

int n;
vector<Edge> edges;
const int INF = 1000000000;

void solve() {
    vector<int> d(n, 0);
    vector<int> p(n, -1);
    int x;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            if (d[e.a] + e.cost < d[e.b]) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
        }
    }

    if (x == -1) {
        cout << "No negative cycle found.";
    } else {
        for (int i = 0; i < n; ++i)
            x = p[x];

        vector<int> cycle;
        for (int v = x; v = p[v]) {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());
    }
}
```

```

        cout << "Negative cycle: ";
        for (int v : cycle)
            cout << v << ' ';
        cout << endl;
    }
}

```

Using Floyd-Warshall algorithm

The Floyd-Warshall algorithm allows to solve the second variation of the problem - finding all pairs of vertices (i, j) which don't have a shortest path between them (i.e. a path of arbitrarily small weight exists).

Again, the details can be found in the [Floyd-Warshall](#) article, and here we describe only its application.

Run Floyd-Warshall algorithm on the graph. Initially $d[v][v] = 0$ for each v . But after running the algorithm $d[v][v]$ will be smaller than 0 if there exists a negative length path from v to v . We can use this to also find all pairs of vertices that don't have a shortest path between them. We iterate over all pairs of vertices (i, j) and for each pair we check whether they have a shortest path between them. To do this try all possibilities for an intermediate vertex t . (i, j) doesn't have a shortest path, if one of the intermediate vertices t has $d[t][t] < 0$ (i.e. t is part of a cycle of negative weight), t can be reached from i and j can be reached from t . Then the path from i to j can have arbitrarily small weight. We will denote this with $-\text{INF}$.

Implementation

```

for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        for (int t = 0; t < n; ++t) {
            if (d[i][t] < INF && d[t][t] < 0 && d[t][j] < INF)
                d[i][j] = -INF;
        }
    }
}

```

Practice Problems

- [UVA: Wormholes](#)
- [SPOJ: Alice in Amsterdam, I mean Wonderland](#)
- [SPOJ: Johnsons Algorithm](#)

Contributors:

[Jakob Kogler](#) (36.7%) = (35.78%) [Mariia Mykhailova](#) (7.34%) [Oleksandr Kulkov](#) (5.5%) [Bhuvnesh Jain](#) (3.67%) [Michael Hayter](#) (2.75%)
[Shyamal Vadera](#) (2.75%) [Sunil Sangwan](#) (2.75%) [Harsh Mathur](#) (1.83%) [Deepanshu Raj](#) (0.92%)

Translated

From: e-maxx.ru

Finding the Eulerian path in $O(M)$

A Eulerian path is a path in a graph that passes through all of its edges exactly once. A Eulerian cycle is a Eulerian path that is a cycle.

The problem is to find the Eulerian path in an **undirected multigraph with loops**.

Algorithm

First we can check if there is an Eulerian path. We can use the following theorem. An Eulerian cycle exists if and only if the degrees of all vertices are even. And an Eulerian path exists if and only if the number of vertices with odd degrees is two (or zero, in the case of the existence of a Eulerian cycle). In addition, of course, the graph must be sufficiently connected (i.e., if you remove all isolated vertices from it, you should get a connected graph).

To find the Eulerian path / Eulerian cycle we can use the following strategy: We find all simple cycles and combine them into one - this will be the Eulerian cycle. If the graph is such that the Eulerian path is not a cycle, then add the missing edge, find the Eulerian cycle, then remove the extra edge.

Looking for all cycles and combining them can be done with a simple recursive procedure:

```
procedure FindEulerPath(V)
  1. iterate through all the edges outgoing from vertex V;
     remove this edge from the graph,
     and call FindEulerPath from the second end of this edge;
  2. add vertex V to the answer.
```

The complexity of this algorithm is obviously linear with respect to the number of edges.

But we can write the same algorithm in the non-recursive version:

```
stack St;
put start vertex in St;
until St is empty
  let V be the value at the top of St;
  if degree(V) = 0, then
    add V to the answer;
    remove V from the top of St;
  otherwise
    find any edge coming out of V;
    remove it from the graph;
    put the second end of this edge in St;
```

It is easy to check the equivalence of these two forms of the algorithm. However, the second form is obviously faster, and the code will be much more efficient.

The Domino problem

We give here a classical Eulerian cycle problem - the Domino problem.

There are N dominoes, as it is known, on both ends of the Domino one number is written (usually from 1 to 6, but in our case it is not important). You want to put all the dominoes in a row so that the numbers on any two adjacent dominoes, written on their common side, coincide. Dominoes are allowed to turn.

Reformulate the problem. Let the numbers written on the bottoms be the vertices of the graph, and the dominoes be the edges of this graph (each Domino with numbers (a, b) are the edges (a, b) and (b, a)). Then our problem is reduced to the problem of finding the Eulerian path in this graph.

Implementation

The program below searches for and outputs a Eulerian loop or path in a graph, or outputs -1 if it does not exist.

First, the program checks the degree of vertices: if there are no vertices with an odd degree, then the graph has an Euler cycle, if there are 2 vertices with an odd degree, then in the graph there is only an Euler path (but no Euler cycle), if there are more than 2 such vertices, then in the graph there is no Euler cycle or Euler path. To find the Euler path (not a cycle), let's do this: if V_1 and V_2 are two vertices of odd degree, then just add an edge (V_1, V_2) , in the resulting graph we find the Euler cycle (it will obviously exist), and then remove the "fictitious" edge (V_1, V_2) from the answer. We will look for the Euler cycle exactly as described above (non-recursive version), and at the same time at the end of this algorithm we will check whether the graph was connected or not (if the graph was not connected, then at the end of the algorithm some edges will remain in the graph, and in this case we need to print -1). Finally, the program takes into account that there can be isolated vertices in the graph.

Notice that we use an adjacency matrix in this problem. Also this implementation handles finding the next with brute-force, which requires to iterate over the complete row in the matrix over and over. A better way would be to store the graph as an adjacency list, and remove edges in $O(1)$ and mark the reversed edges in separate list. This way we can achieve an $O(N)$ algorithm.

```
int main() {
    int n;
    vector<vector<int>>> g(n, vector<int>(n));
    // reading the graph in the adjacency matrix

    vector<int> deg(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            deg[i] += g[i][j];
    }

    int first = 0;
    while (first < n && !deg[first])
        ++first;
    if (first == n) {
        cout << -1;
        return 0;
    }

    int v1 = -1, v2 = -1;
    bool bad = false;
    for (int i = 0; i < n; ++i) {
        if (deg[i] & 1) {
            if (v1 == -1)
                v1 = i;
            else if (v2 == -1)
                v2 = i;
            else
                bad = true;
        }
    }

    if (v1 != -1)
        ++g[v1][v2], ++g[v2][v1];

    stack<int> st;
    st.push(first);
    vector<int> res;
    while (!st.empty()) {
        int v = st.top();
        int i;
        for (i = 0; i < n; ++i)
            if (g[v][i])
                break;
        if (i == n) {
            res.push_back(v);
            st.pop();
        } else {
            --g[v][i];
            --g[i][v];
            st.push(i);
        }
    }
}
```

```

    if (v1 != -1) {
        for (size_t i = 0; i + 1 < res.size(); ++i) {
            if ((res[i] == v1 && res[i + 1] == v2) ||
                (res[i] == v2 && res[i + 1] == v1)) {
                vector<int> res2;
                for (size_t j = i + 1; j < res.size(); ++j)
                    res2.push_back(res[j]);
                for (size_t j = 1; j <= i; ++j)
                    res2.push_back(res[j]);
                res = res2;
                break;
            }
        }
    }

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (g[i][j])
                bad = true;
        }
    }

    if (bad) {
        cout << -1;
    } else {
        for (int x : res)
            cout << x << " ";
    }
}

```

Practice problems:

- [CSES : Mail Delivery](#)
- [CSES : Teleporters Path](#)

Contributors:

[Amandeep Singh](#) (88.48%)
 [Anish Aggarwal](#) (3.03%)
 [Aryaman](#) (1.82%)
 [Jakob Kogler](#) (1.82%)
 [Oleksandr Kulkov](#) (1.82%)
 [Anirudh Pupneja](#) (0.61%)
[Anton Ālgmyr](#) (0.61%)
 [Ashish Ahuja](#) (0.61%)
 [Shyamal Vadera](#) (0.61%)
 [Sohaib Uddin Syed](#) (0.61%)

Translated

From: e-maxx.ru

Lowest Common Ancestor - $O(\sqrt{N})$ and $O(\log N)$ with $O(N)$ preprocessing

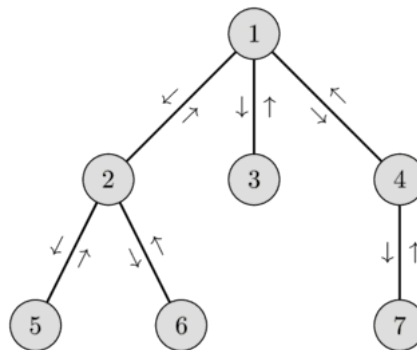
Given a tree G . Given queries of the form (v_1, v_2) , for each query you need to find the lowest common ancestor (or least common ancestor), i.e. a vertex v that lies on the path from the root to v_1 and the path from the root to v_2 , and the vertex should be the lowest. In other words, the desired vertex v is the most bottom ancestor of v_1 and v_2 . It is obvious that their lowest common ancestor lies on a shortest path from v_1 and v_2 . Also, if v_1 is the ancestor of v_2 , v_1 is their lowest common ancestor.

The Idea of the Algorithm

Before answering the queries, we need to **preprocess** the tree. We make a **DFS** traversal starting at the root and we build a list `euler` which stores the order of the vertices that we visit (a vertex is added to the list when we first visit it, and after the return of the DFS traversals to its children). This is also called an Euler tour of the tree. It is clear that the size of this list will be $O(N)$. We also need to build an array `first[0..N-1]` which stores for each vertex i its first occurrence in `euler`. That is, the first position in `euler` such that `euler[first[i]] = i`. Also by using the DFS we can find the height of each node (distance from root to it) and store it in the array `height[0..N-1]`.

So how can we answer queries using the Euler tour and the additional two arrays? Suppose the query is a pair of v_1 and v_2 . Consider the vertices that we visit in the Euler tour between the first visit of v_1 and the first visit of v_2 . It is easy to see, that the $LCA(v_1, v_2)$ is the vertex with the lowest height on this path. We already noticed, that the LCA has to be part of the shortest path between v_1 and v_2 . Clearly it also has to be the vertex with the smallest height. And in the Euler tour we essentially use the shortest path, except that we additionally visit all subtrees that we find on the path. But all vertices in these subtrees are lower in the tree than the LCA and therefore have a larger height. So the $LCA(v_1, v_2)$ can be uniquely determined by finding the vertex with the smallest height in the Euler tour between `first(v1)` and `first(v2)`.

Let's illustrate this idea. Consider the following graph and the Euler tour with the corresponding heights:



Vertices:	1	2	5	2	6	2	1	3	1	4	7	4	1
Heights:	1	2	3	2	3	2	1	2	1	2	3	2	1

The tour starting at vertex 6 and ending at 4 we visit the vertices $[6, 2, 1, 3, 1, 4]$. Among those vertices the vertex 1 has the lowest height, therefore $LCA(6, 4) = 1$.

To recap: to answer a query we just need to **find the vertex with smallest height** in the array `euler` in the range from `first[v1]` to `first[v2]`. Thus, **the LCA problem is reduced to the RMQ problem** (finding the minimum in an range problem).

Using [Sqrt-Decomposition](#), it is possible to obtain a solution answering each query in $O(\sqrt{N})$ with preprocessing in $O(N)$ time.

Using a [Segment Tree](#) you can answer each query in $O(\log N)$ with preprocessing in $O(N)$ time.

Since there will almost never be any update to the stored values, a [Sparse Table](#) might be a better choice, allowing $O(1)$ query answering with $O(N \log N)$ build time.

Implementation

In the following implementation of the LCA algorithm a Segment Tree is used.

```
struct LCA {
    vector<int> height, euler, first, segtree;
    vector<bool> visited;
    int n;

    LCA(vector<vector<int>> &adj, int root = 0) {
        n = adj.size();
        height.resize(n);
        first.resize(n);
        euler.reserve(n * 2);
        visited.assign(n, false);
        dfs(adj, root);
        int m = euler.size();
        segtree.resize(m * 4);
        build(1, 0, m - 1);
    }

    void dfs(vector<vector<int>> &adj, int node, int h = 0) {
        visited[node] = true;
        height[node] = h;
        first[node] = euler.size();
        euler.push_back(node);
        for (auto to : adj[node]) {
            if (!visited[to]) {
                dfs(adj, to, h + 1);
                euler.push_back(node);
            }
        }
    }

    void build(int node, int b, int e) {
        if (b == e) {
            segtree[node] = euler[b];
        } else {
            int mid = (b + e) / 2;
            build(node << 1, b, mid);
            build(node << 1 | 1, mid + 1, e);
            int l = segtree[node << 1], r = segtree[node << 1 | 1];
            segtree[node] = (height[l] < height[r]) ? l : r;
        }
    }

    int query(int node, int b, int e, int L, int R) {
        if (b > R || e < L)
            return -1;
        if (b >= L && e <= R)
            return segtree[node];
        int mid = (b + e) >> 1;

        int left = query(node << 1, b, mid, L, R);
        int right = query(node << 1 | 1, mid + 1, e, L, R);
        if (left == -1) return right;
        if (right == -1) return left;
        return height[left] < height[right] ? left : right;
    }

    int lca(int u, int v) {
        int left = first[u], right = first[v];
        if (left > right)
            swap(left, right);
        return query(1, 0, euler.size() - 1, left, right);
    }
};
```

Practice Problems

- [SPOJ: LCA](#)

- [SPOJ: DISQUERY](#)
- [TIMUS: 1471. Distance in the Tree](#)
- [CODEFORCES: Design Tutorial: Inverse the Problem](#)
- [CODECHEF: Lowest Common Ancestor](#)
- [SPOJ - Lowest Common Ancestor](#)
- [SPOJ - Ada and Orange Tree](#)
- [DevSkill - Motoku \(archived\)](#)
- [UVA 12655 - Trucks](#)
- [Codechef - Pishty and Tree](#)
- [UVA - 12533 - Joining Couples](#)
- [Codechef - So close yet So Far](#)
- [Codeforces - Drivers Dissatisfaction](#)
- [UVA 11354 - Bond](#)
- [SPOJ - Query on a tree II](#)
- [Codeforces - Best Edge Weight](#)
- [Codeforces - Misha, Grisha and Underground](#)
- [SPOJ - Nlogonian Tickets](#)
- [Codeforces - Rowena Rawenclaws Diadem](#)

Contributors:

[Jakob Kogler](#) (34.93%) [Manuel Pineda](#) (31.51%) [Anthony](#) (13.7%) [Morass](#) (8.22%) [Sunil Sangwan](#) (4.11%) [Gabriel Silva Simões](#) (2.05%)
[Michael Hayter](#) (2.05%) [Oleksandr Kulkov](#) (2.05%) [Gaurang Tandon](#) (1.36%)

Translated From: e-maxx.ru

Lowest Common Ancestor - Binary Lifting

Let G be a tree. For every query of the form (u, v) we want to find the lowest common ancestor of the nodes u and v , i.e. we want to find a node w that lies on the path from u to the root node, that lies on the path from v to the root node, and if there are multiple nodes we pick the one that is farthest away from the root node. In other words the desired node w is the lowest ancestor of u and v . In particular if u is an ancestor of v , then u is their lowest common ancestor.

The algorithm described in this article will need $O(N \log N)$ for preprocessing the tree, and then $O(\log N)$ for each LCA query.

Algorithm

For each node we will precompute its ancestor above him, its ancestor two nodes above, its ancestor four above, etc. Let's store them in the array up , i.e. $up[i][j]$ is the 2^j -th ancestor above the node i with $i=1 \dots N$, $j=0 \dots \lceil \log(N) \rceil$. These information allow us to jump from any node to any ancestor above it in $O(\log N)$ time. We can compute this array using a DFS traversal of the tree.

For each node we will also remember the time of the first visit of this node (i.e. the time when the DFS discovers the node), and the time when we left it (i.e. after we visited all children and exit the DFS function). We can use this information to determine in constant time if a node is an ancestor of another node.

Suppose now we received a query (u, v) . We can immediately check whether one node is the ancestor of the other. In this case this node is already the LCA. If u is not the ancestor of v , and v not the ancestor of u , we climb the ancestors of u until we find the highest (i.e. closest to the root) node, which is not an ancestor of v (i.e. a node x , such that x is not an ancestor of v , but $up[x][0]$ is). We can find this node x in $O(\log N)$ time using the array up .

We will describe this process in more detail. Let $L = \lceil \log(N) \rceil$. Suppose first that $i = L$. If $up[u][i]$ is not an ancestor of v , then we can assign $u = up[u][i]$ and decrement i . If $up[u][i]$ is an ancestor, then we just decrement i . Clearly after doing this for all non-negative i the node u will be the desired node - i.e. u is still not an ancestor of v , but $up[u][0]$ is.

Now, obviously, the answer to LCA will be $up[u][0]$ - i.e., the smallest node among the ancestors of the node u , which is also an ancestor of v .

So answering a LCA query will iterate i from $\lceil \log(N) \rceil$ to 0 and checks in each iteration if one node is the ancestor of the other. Consequently each query can be answered in $O(\log N)$.

Implementation

```
int n, l;
vector<vector<int>> adj;

int timer;
vector<int> tin, tout;
vector<vector<int>> up;

void dfs(int v, int p)
{
    tin[v] = ++timer;
    up[v][0] = p;
    for (int i = 1; i <= l; ++i)
        up[v][i] = up[up[v][i-1]][i-1];

    for (int u : adj[v]) {
        if (u != p)
            dfs(u, v);
    }

    tout[v] = ++timer;
}
```

```
bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;
    for (int i = 1; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }
    return up[u][0];
}

void preprocess(int root) {
    tin.resize(n);
    tout.resize(n);
    timer = 0;
    l = ceil(log2(n));
    up.assign(n, vector<int>(l + 1));
    dfs(root, root);
}
```

Practice Problems

- [LeetCode - Kth Ancestor of a Tree Node](#)
- [Codechef - Longest Good Segment](#)
- [HackerEarth - Optimal Connectivity](#)

Contributors:

[Jakob Kogler](#) (86.0%) [Oleksandr Kulkov](#) (6.0%) [Hossein Yousefi](#) (4.0%) [Santhosh Anguluri](#) (3.0%) [Ashish Kumar](#) (1.0%)

Translated

From: e-maxx.ru

Lowest Common Ancestor - Farach-Colton and Bender Algorithm

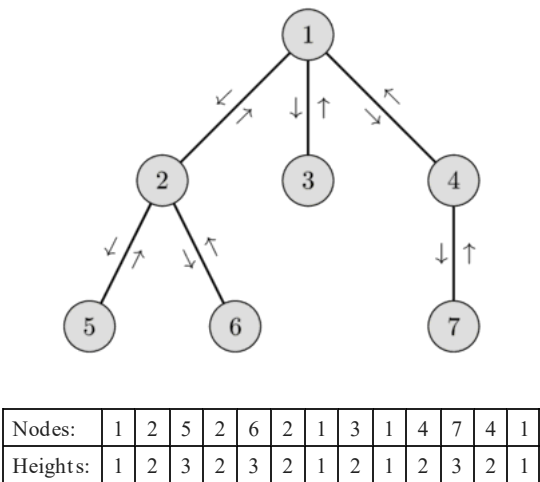
Let G be a tree. For every query of the form (u, v) we want to find the lowest common ancestor of the nodes u and v , i.e. we want to find a node w that lies on the path from u to the root node, that lies on the path from v to the root node, and if there are multiple nodes we pick the one that is farthest away from the root node. In other words the desired node w is the lowest ancestor of u and v . In particular if u is an ancestor of v , then u is their lowest common ancestor.

The algorithm which will be described in this article was developed by Farach-Colton and Bender. It is asymptotically optimal.

Algorithm

We use the classical reduction of the LCA problem to the RMQ problem. We traverse all nodes of the tree with DFS and keep an array with all visited nodes and the heights of these nodes. The LCA of two nodes u and v is the node between the occurrences of u and v in the tour, that has the smallest height.

In the following picture you can see a possible Euler-Tour of a graph and in the list below you can see the visited nodes and their heights.



You can read more about this reduction in the article [Lowest Common Ancestor](#). In that article the minimum of a range was either found by sqrt-decomposition in $O(\sqrt{N})$ or in $O(\log N)$ using a Segment tree. In this article we look at how we can solve the given range minimum queries in $O(1)$ time, while still only taking $O(N)$ time for preprocessing.

Note that the reduced RMQ problem is very specific: any two adjacent elements in the array differ exactly by one (since the elements of the array are nothing more than the heights of the nodes visited in order of traversal, and we either go to a descendant, in which case the next element is one bigger, or go back to the ancestor, in which case the next element is one lower). The Farach-Colton and Bender algorithm describes a solution for exactly this specialized RMQ problem.

Let's denote with A the array on which we want to perform the range minimum queries. And N will be the size of A .

There is an easy data structure that we can use for solving the RMQ problem with $O(N \log N)$ preprocessing and $O(1)$ for each query: the [Sparse Table](#). We create a table T where each element $T[i][j]$ is equal to the minimum of A in the interval $[i, i + 2^j - 1]$. Obviously $0 \leq j \leq \lceil \log N \rceil$, and therefore the size of the Sparse Table will be $O(N \log N)$. You can build the table easily in $O(N \log N)$ by noting that $T[i][j] = \min(T[i][j - 1], T[i + 2^{j-1}][j - 1])$.

How can we answer a query RMQ in $O(1)$ using this data structure? Let the received query be $[l, r]$, then the answer is $\min(T[l][sz], T[r - 2^{sz} + 1][sz])$, where sz is the biggest exponent such that 2^{sz} is not bigger than the range length $r - l + 1$. Indeed we

can take the range $[l, r]$ and cover it two segments of length 2^{sz} - one starting in l and the other ending in r . These segments overlap, but this doesn't interfere with our computation. To really achieve the time complexity of $O(1)$ per query, we need to know the values of sz for all possible lengths from 1 to N . But this can be easily precomputed.

Now we want to improve the complexity of the preprocessing down to $O(N)$.

We divide the array A into blocks of size $K = 0.5 \log N$ with $\log$ being the logarithm to base 2. For each block we calculate the minimum element and store them in an array B . B has the size $\frac{N}{K}$. We construct a sparse table from the array B . The size and the time complexity of it will be:

$$\begin{aligned} \frac{N}{K} \log\left(\frac{N}{K}\right) &= \frac{2N}{\log(N)} \log\left(\frac{2N}{\log(N)}\right) = \\ &= \frac{2N}{\log(N)} \left(1 + \log\left(\frac{N}{\log(N)}\right)\right) \leq \frac{2N}{\log(N)} + 2N = O(N) \end{aligned}$$

Now we only have to learn how to quickly answer range minimum queries within each block. In fact if the received range minimum query is $[l, r]$ and l and r are in different blocks then the answer is the minimum of the following three values: the minimum of the suffix of block of l starting at l , the minimum of the prefix of block of r ending at r , and the minimum of the blocks between those. The minimum of the blocks in between can be answered in $O(1)$ using the Sparse Table. So this leaves us only the range minimum queries inside blocks.

Here we will exploit the property of the array. Remember that the values in the array - which are just height values in the tree - will always differ by one. If we remove the first element of a block, and subtract it from every other item in the block, every block can be identified by a sequence of length $K - 1$ consisting of the number $+1$ and -1 . Because these blocks are so small, there are only a few different sequences that can occur. The number of possible sequences is:

$$2^{K-1} = 2^{0.5 \log(N)-1} = 0.5(2^{\log(N)})^{0.5} = 0.5\sqrt{N}$$

Thus the number of different blocks is $O(\sqrt{N})$, and therefore we can precompute the results of range minimum queries inside all different blocks in $O(\sqrt{N} K^2) = O(\sqrt{N} \log^2(N)) = O(N)$ time. For the implementation we can characterize a block by a bitmask of length $K - 1$ (which will fit in a standard int) and store the index of the minimum in an array `block[mask][l][r]` of size $O(\sqrt{N} \log^2(N))$.

So we learned how to precompute range minimum queries within each block, as well as range minimum queries over a range of blocks, all in $O(N)$. With these precomputations we can answer each query in $O(1)$, by using at most four precomputed values: the minimum of the block containing l , the minimum of the block containing r , and the two minima of the overlapping segments of the blocks between them.

Implementation

```
int n;
vector<vector<int>> adj;

int block_size, block_cnt;
vector<int> first_visit;
vector<int> euler_tour;
vector<int> height;
vector<int> log_2;
vector<vector<int>> st;
vector<vector<vector<int>>> blocks;
vector<int> block_mask;

void dfs(int v, int p, int h) {
    first_visit[v] = euler_tour.size();
    euler_tour.push_back(v);
    height[v] = h;

    for (int u : adj[v]) {
        if (u == p)
            continue;
        dfs(u, v, h + 1);
        euler_tour.push_back(v);
    }
}
```

```

    }
}

int min_by_h(int i, int j) {
    return height[euler_tour[i]] < height[euler_tour[j]] ? i : j;
}

void precompute_lca(int root) {
    // get euler tour & indices of first occurrences
    first_visit.assign(n, -1);
    height.assign(n, 0);
    euler_tour.reserve(2 * n);
    dfs(root, -1, 0);

    // precompute all log values
    int m = euler_tour.size();
    log_2.reserve(m + 1);
    log_2.push_back(-1);
    for (int i = 1; i <= m; i++)
        log_2.push_back(log_2[i / 2] + 1);

    block_size = max(1, log_2[m] / 2);
    block_cnt = (m + block_size - 1) / block_size;

    // precompute minimum of each block and build sparse table
    st.assign(block_cnt, vector<int>(log_2[block_cnt] + 1));
    for (int i = 0, j = 0, b = 0; i < m; i++, j++) {
        if (j == block_size)
            j = 0, b++;
        if (j == 0 || min_by_h(i, st[b][0]) == i)
            st[b][0] = i;
    }
    for (int l = 1; l <= log_2[block_cnt]; l++) {
        for (int i = 0; i < block_cnt; i++) {
            int ni = i + (1 << (l - 1));
            if (ni >= block_cnt)
                st[i][l] = st[i][l-1];
            else
                st[i][l] = min_by_h(st[i][l-1], st[ni][l-1]);
        }
    }

    // precompute mask for each block
    block_mask.assign(block_cnt, 0);
    for (int i = 0, j = 0, b = 0; i < m; i++, j++) {
        if (j == block_size)
            j = 0, b++;
        if (j > 0 && (i >= m || min_by_h(i - 1, i) == i - 1))
            block_mask[b] += 1 << (j - 1);
    }

    // precompute RMQ for each unique block
    int possibilities = 1 << (block_size - 1);
    blocks.resize(possibilities);
    for (int b = 0; b < block_cnt; b++) {
        int mask = block_mask[b];
        if (!blocks[mask].empty())
            continue;
        blocks[mask].assign(block_size, vector<int>(block_size));
        for (int l = 0; l < block_size; l++) {
            blocks[mask][l][l] = l;
            for (int r = l + 1; r < block_size; r++) {
                blocks[mask][l][r] = blocks[mask][l][r - 1];
                if (b * block_size + r < m)
                    blocks[mask][l][r] = min_by_h(b * block_size + blocks[mask][l][r],
                                                    b * block_size + r) - b * block_size;
            }
        }
    }
}

int lca_in_block(int b, int l, int r) {
    return blocks[block_mask[b]][l][r] + b * block_size;
}

int lca(int v, int u) {
    int l = first_visit[v];
    int r = first_visit[u];
    if (l > r)
        swap(l, r);

```

```
int b1 = l / block_size;
int br = r / block_size;
if (b1 == br)
    return euler_tour[lca_in_block(b1, l % block_size, r % block_size)];
int ans1 = lca_in_block(b1, l % block_size, block_size - 1);
int ans2 = lca_in_block(br, 0, r % block_size);
int ans = min_by_h(ans1, ans2);
if (b1 + 1 < br) {
    int l = log_2[br - b1 - 1];
    int ans3 = st[b1+1][l];
    int ans4 = st[br - (1 << l)][l];
    ans = min_by_h(ans, min_by_h(ans3, ans4));
}
return euler_tour[ans];
}
```

Contributors:

[Jakob Kogler](#) (93.81%) [Oleksandr Kulkov](#) (2.86%) [Mariia Mykhailova](#) (1.43%) [Michael Hayter](#) (1.43%) [Le Bao Hiep](#) (0.48%)

Translated From: e-maxx.ru

Solve RMQ (Range Minimum Query) by finding LCA (Lowest Common Ancestor)

Given an array $A[0..N-1]$. For each query of the form $[L, R]$ we want to find the minimum in the array A starting from position L and ending with position R . We will assume that the array A doesn't change in the process, i.e. this article describes a solution to the static RMQ problem.

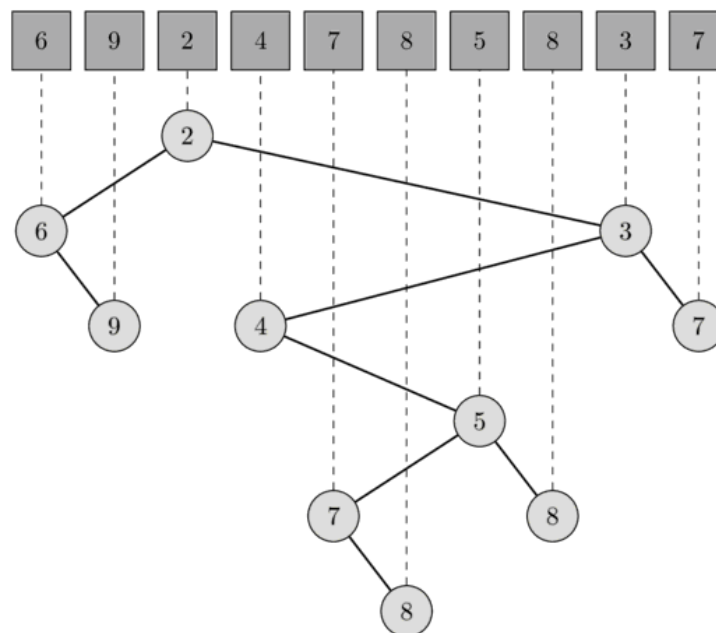
Here is a description of an asymptotically optimal solution. It stands apart from other solutions for the RMQ problem, since it is very different from them: it reduces the RMQ problem to the LCA problem, and then uses the [Farach-Colton and Bender algorithm](#), which reduces the LCA problem back to a specialized RMQ problem and solves that.

Algorithm

We construct a **Cartesian tree** from the array A . A Cartesian tree of an array A is a binary tree with the min-heap property (the value of parent node has to be smaller or equal than the value of its children) such that the in-order traversal of the tree visits the nodes in the same order as they are in the array A .

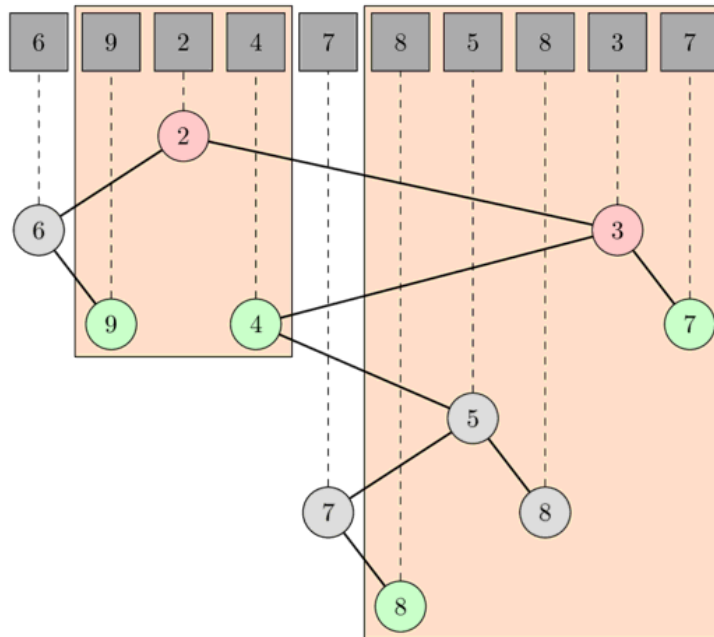
In other words, a Cartesian tree is a recursive data structure. The array A will be partitioned into 3 parts: the prefix of the array up to the minimum, the minimum, and the remaining suffix. The root of the tree will be a node corresponding to the minimum element of the array A , the left subtree will be the Cartesian tree of the prefix, and the right subtree will be a Cartesian tree of the suffix.

In the following image you can see one array of length 10 and the corresponding Cartesian tree.



The range minimum query $[l, r]$ is equivalent to the lowest common ancestor query $[l', r']$, where l' is the node corresponding to the element $A[l]$ and r' the node corresponding to the element $A[r]$. Indeed the node corresponding to the smallest element in the range has to be an ancestor of all nodes in the range, therefore also from l' and r' . This automatically follows from the min-heap property. And is also has to be the lowest ancestor, because otherwise l' and r' would be both in the left or in the right subtree, which generates a contradiction since in such a case the minimum wouldn't even be in the range.

In the following image you can see the LCA queries for the RMQ queries $[1, 3]$ and $[5, 9]$. In the first query the LCA of the nodes $A[1]$ and $A[3]$ is the node corresponding to $A[2]$ which has the value 2, and in the second query the LCA of $A[5]$ and $A[9]$ is the node corresponding to $A[8]$ which has the value 3.



Such a tree can be built in $O(N)$ time and the Farach-Colton and Benders algorithm can preprocess the tree in $O(N)$ and find the LCA in $O(1)$.

Construction of a Cartesian tree

We will build the Cartesian tree by adding the elements one after another. In each step we maintain a valid Cartesian tree of all the processed elements. It is easy to see, that adding an element $s[i]$ can only change the nodes in the most right path - starting at the root and repeatedly taking the right child - of the tree. The subtree of the node with the smallest, but greater or equal than $s[i]$, value becomes the left subtree of $s[i]$, and the tree with root $s[i]$ will become the new right subtree of the node with the biggest, but smaller than $s[i]$ value.

This can be implemented by using a stack to store the indices of the most right nodes.

```
vector<int> parent(n, -1);
stack<int> s;
for (int i = 0; i < n; i++) {
    int last = -1;
    while (!s.empty() && A[s.top()] >= A[i]) {
        last = s.top();
        s.pop();
    }
    if (!s.empty())
        parent[i] = s.top();
    if (last >= 0)
        parent[last] = i;
    s.push(i);
}
```

Contributors:

[Jakob Kogler](#) (82.35%) [Michael Hayter](#) (8.82%) [Oleksandr Kulkov](#) (8.82%)

Translated From: e-maxx.ru

Lowest Common Ancestor - Tarjan's off-line algorithm

We have a tree G with n nodes and we have m queries of the form (u, v) . For each query (u, v) we want to find the lowest common ancestor of the vertices u and v , i.e. the node that is an ancestor of both u and v and has the greatest depth in the tree. The node v is also an ancestor of v , so the LCA can also be one of the two nodes.

In this article we will solve the problem off-line, i.e. we assume that all queries are known in advance, and we therefore answer the queries in any order we like. The following algorithm allows to answer all m queries in $O(n + m)$ total time, i.e. for sufficiently large m in $O(1)$ for each query.

Algorithm

The algorithm is named after Robert Tarjan, who discovered it in 1979 and also made many other contributions to the [Disjoint Set Union](#) data structure, which will be heavily used in this algorithm.

The algorithm answers all queries with one DFS traversal of the tree. Namely a query (u, v) is answered at node u , if node v has already been visited previously, or vice versa.

So let's assume we are currently at node v , we have already made recursive DFS calls, and also already visited the second node u from the query (u, v) . Let's learn how to find the LCA of these two nodes.

Note that $LCA(u, v)$ is either the node v or one of its ancestors. So we need to find the lowest node among the ancestors of v (including v), for which the node u is a descendant. Also note that for a fixed v the visited nodes of the tree split into a set of disjoint sets. Each ancestor p of node v has his own set containing this node and all subtrees with roots in those of its children who are not part of the path from v to the root of the tree. The set which contains the node u determines the $LCA(u, v)$: the LCA is the representative of the set, namely the node on lies on the path between v and the root of the tree.

We only need to learn to efficiently maintain all these sets. For this purpose we apply the data structure DSU. To be able to apply Union by rank, we store the real representative (the value on the path between v and the root of the tree) of each set in the array `ancestor`.

Let's discuss the implementation of the DFS. Let's assume we are currently visiting the node v . We place the node in a new set in the DSU, `ancestor[v] = v`. As usual we process all children of v . For this we must first recursively call DFS from that node, and then add this node with all its subtree to the set of v . This can be done with the function `union_sets` and the following assignment `ancestor[find_set(v)] = v` (this is necessary, because `union_sets` might change the representative of the set).

Finally after processing all children we can answer all queries of the form (u, v) for which u has been already visited. The answer to the query, i.e. the LCA of u and v , will be the node `ancestor[find_set(u)]`. It is easy to see that a query will only be answered once.

Let's us determine the time complexity of this algorithm. Firstly we have $O(n)$ because of the DFS. Secondly we have the function calls of `union_sets` which happen n times, resulting also in $O(n)$. And thirdly we have the calls of `find_set` for every query, which gives $O(m)$. So in total the time complexity is $O(n + m)$, which means that for sufficiently large m this corresponds to $O(1)$ for answering one query.

Implementation

Here is an implementation of this algorithm. The implementation of DSU has been not included, as it can be used without any modifications.

```
vector<vector<int>> adj;
vector<vector<int>> queries;
vector<int> ancestor;
vector<bool> visited;

void dfs(int v)
{
    visited[v] = true;
```

```

    ancestor[v] = v;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
            union_sets(v, u);
            ancestor[find_set(v)] = v;
        }
    }
    for (int other_node : queries[v]) {
        if (visited[other_node])
            cout << "LCA of " << v << " and " << other_node
                 << " is " << ancestor[find_set(other_node)] << ".\n";
    }
}

void compute_LCAs() {
    // initialize n, adj and DSU
    // for (each query (u, v)) {
    //     queries[u].push_back(v);
    //     queries[v].push_back(u);
    // }

    ancestor.resize(n);
    visited.assign(n, false);
    dfs(0);
}

```

Contributors:

[Jakob Kogler](#) (92.55%)
 [Oleksandr Kulkov](#) (6.38%)
 [Anton Ålgmyr](#) (1.06%)

Translated From: e-maxx.ru

Maximum flow - Ford-Fulkerson and Edmonds-Karp

The Edmonds-Karp algorithm is an implementation of the Ford-Fulkerson method for computing a maximal flow in a flow network.

Flow network

First let's define what a **flow network**, a **flow**, and a **maximum flow** is.

A **network** is a directed graph G with vertices V and edges E combined with a function c , which assigns each edge $e \in E$ a non-negative integer value, the **capacity** of e . Such a network is called a **flow network**, if we additionally label two vertices, one as **source** and one as **sink**.

A **flow** in a flow network is function f , that again assigns each edge e a non-negative integer value, namely the flow. The function has to fulfill the following two conditions:

The flow of an edge cannot exceed the capacity.

$$f(e) \leq c(e)$$

And the sum of the incoming flow of a vertex u has to be equal to the sum of the outgoing flow of u except in the source and sink vertices.

$$\sum_{(v,u) \in E} f((v,u)) = \sum_{(u,v) \in E} f((u,v))$$

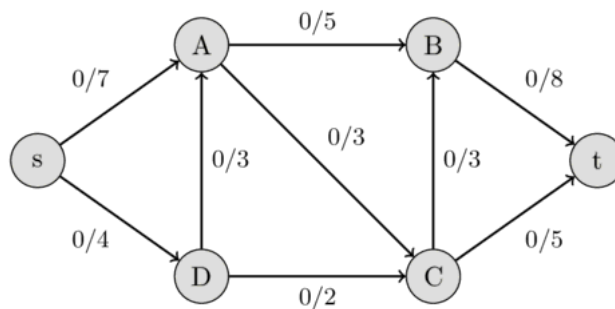
The source vertex s only has an outgoing flow, and the sink vertex t has only incoming flow.

It is easy to see that the following equation holds:

$$\sum_{(s,u) \in E} f((s,u)) = \sum_{(u,t) \in E} f((u,t))$$

A good analogy for a flow network is the following visualization: We represent edges as water pipes, the capacity of an edge is the maximal amount of water that can flow through the pipe per second, and the flow of an edge is the amount of water that currently flows through the pipe per second. This motivates the first flow condition. There cannot flow more water through a pipe than its capacity. The vertices act as junctions, where water comes out of some pipes, and then, these vertices distribute the water in some way to other pipes. This also motivates the second flow condition. All the incoming water has to be distributed to the other pipes in each junction. It cannot magically disappear or appear. The source s is origin of all the water, and the water can only drain in the sink t .

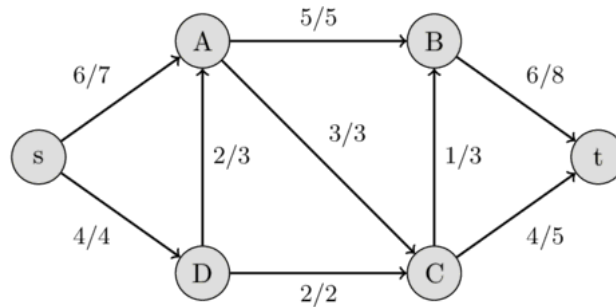
The following image shows a flow network. The first value of each edge represents the flow, which is initially 0, and the second value represents the capacity.



The value of the flow of a network is the sum of all the flows that get produced in the source s , or equivalently to the sum of all the flows that are consumed by the sink t . A **maximal flow** is a flow with the maximal possible value. Finding this maximal flow of a flow network is the problem that we want to solve.

In the visualization with water pipes, the problem can be formulated in the following way: how much water can we push through the pipes from the source to the sink?

The following image shows the maximal flow in the flow network.



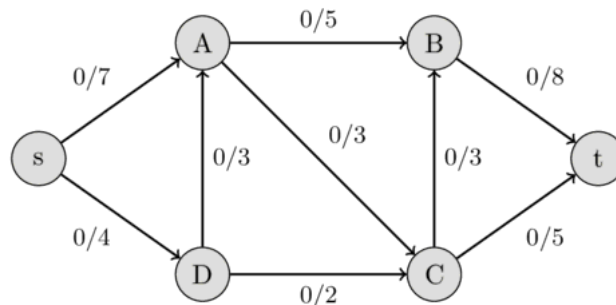
Ford-Fulkerson method

Let's define one more thing. A **residual capacity** of a directed edge is the capacity minus the flow. It should be noted that if there is a flow along some directed edge (u, v) , then the reversed edge has capacity 0 and we can define the flow of it as $f((v, u)) = -f((u, v))$. This also defines the residual capacity for all the reversed edges. We can create a **residual network** from all these edges, which is just a network with the same vertices and edges, but we use the residual capacities as capacities.

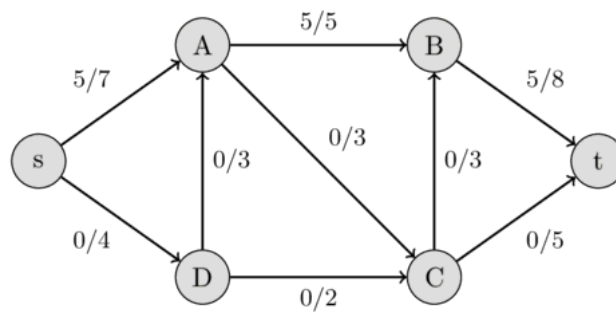
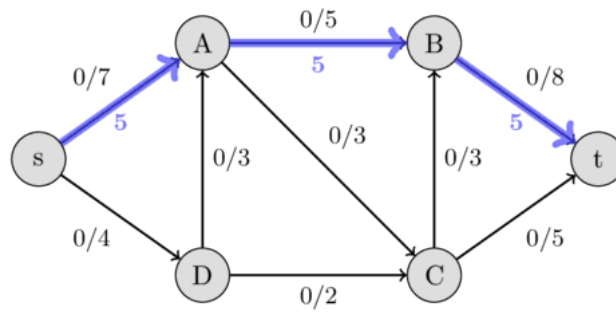
The Ford-Fulkerson method works as follows. First, we set the flow of each edge to zero. Then we look for an **augmenting path** from s to t . An augmenting path is a simple path in the residual graph where residual capacity is positive for all the edges along that path. If such a path is found, then we can increase the flow along these edges. We keep on searching for augmenting paths and increasing the flow. Once an augmenting path doesn't exist anymore, the flow is maximal.

Let us specify in more detail, what increasing the flow along an augmenting path means. Let C be the smallest residual capacity of the edges in the path. Then we increase the flow in the following way: we update $f((u, v)) += C$ and $f((v, u)) -= C$ for every edge (u, v) in the path.

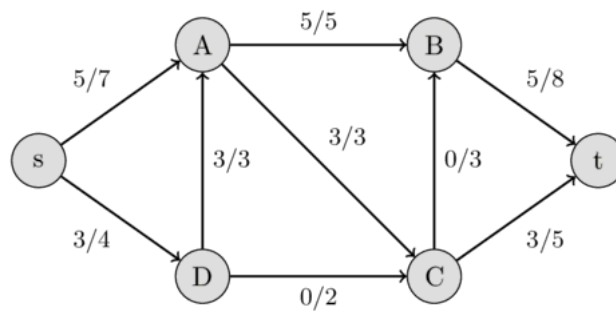
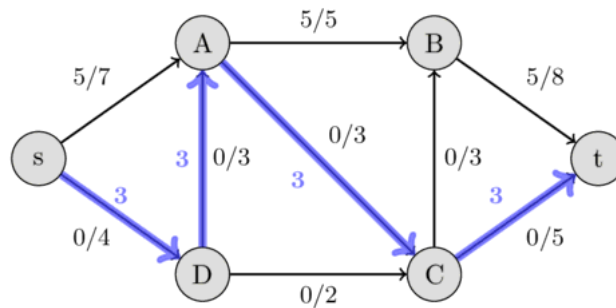
Here is an example to demonstrate the method. We use the same flow network as above. Initially we start with a flow of 0.



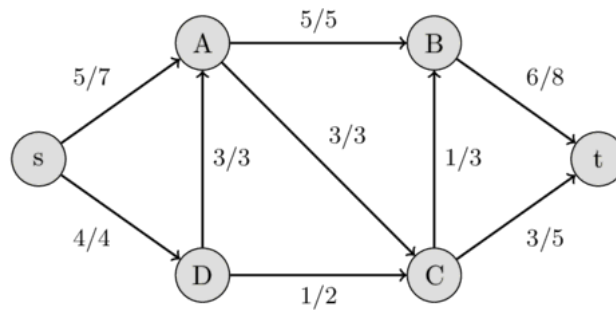
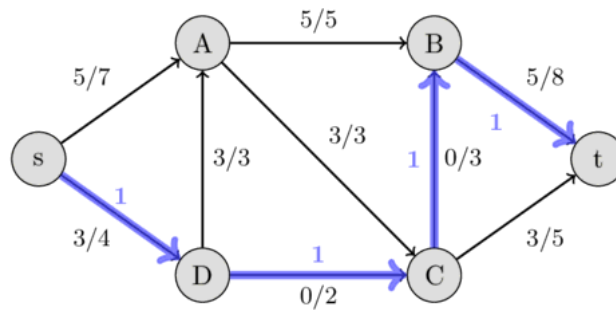
We can find the path $s - A - B - t$ with the residual capacities 7, 5, and 8. Their minimum is 5, therefore we can increase the flow along this path by 5. This gives a flow of 5 for the network.



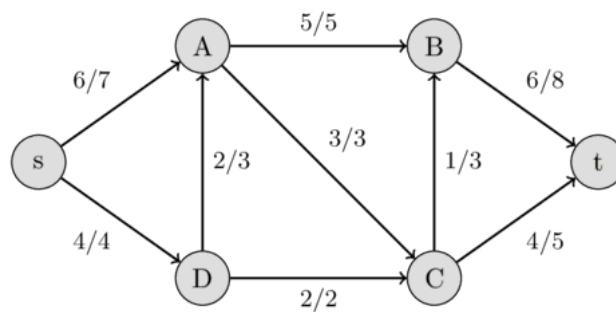
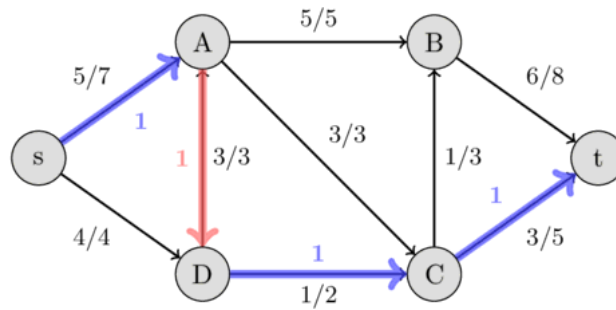
Again we look for an augmenting path, this time we find $s \rightarrow D \rightarrow A \rightarrow C \rightarrow t$ with the residual capacities 4, 3, 3, and 5. Therefore we can increase the flow by 3 and we get a flow of 8 for the network.



This time we find the path $s \rightarrow D \rightarrow C \rightarrow B \rightarrow t$ with the residual capacities 1, 2, 3, and 3, and hence, we increase the flow by 1.



This time we find the augmenting path $s - A - D - C - t$ with the residual capacities 2, 3, 1, and 2. We can increase the flow by 1. But this path is very interesting. It includes the reversed edge (A, D). In the original flow network, we are not allowed to send any flow from A to D. But because we already have a flow of 3 from D to A, this is possible. The intuition of it is the following: Instead of sending a flow of 3 from D to A, we only send 2 and compensate this by sending an additional flow of 1 from s to A, which allows us to send an additional flow of 1 along the path $D - C - t$.



Now, it is impossible to find an augmenting path between s and t , therefore this flow of 10 is the maximal possible. We have found the maximal flow.

It should be noted, that the Ford-Fulkerson method doesn't specify a method of finding the augmenting path. Possible approaches are using [DFS](#) or [BFS](#) which both work in $O(E)$. If all the capacities of the network are integers, then for each augmenting path the flow of the network increases by at least 1 (for more details see [Integral flow theorem](#)). Therefore, the complexity of Ford-Fulkerson is $O(EF)$, where F is the maximal flow of the network. In the case of rational capacities, the algorithm will also terminate, but the complexity is not bounded. In the case of irrational capacities, the algorithm might never terminate, and might not even converge to the maximal flow.

Edmonds-Karp algorithm

Edmonds-Karp algorithm is just an implementation of the Ford-Fulkerson method that uses [BFS](#) for finding augmenting paths. The algorithm was first published by Yefim Dinitz in 1970, and later independently published by Jack Edmonds and Richard Karp in 1972.

The complexity can be given independently of the maximal flow. The algorithm runs in $O(VE^2)$ time, even for irrational capacities. The intuition is, that every time we find an augmenting path one of the edges becomes saturated, and the distance from the edge to s will be longer if it appears later again in an augmenting path. The length of the simple paths is bounded by V .

Implementation

The matrix `capacity` stores the capacity for every pair of vertices. `adj` is the adjacency list of the **undirected graph**, since we have also to use the reversed of directed edges when we are looking for augmenting paths.

The function `maxflow` will return the value of the maximal flow. During the algorithm, the matrix `capacity` will actually store the residual capacity of the network. The value of the flow in each edge will actually not be stored, but it is easy to extend the implementation - by using an additional matrix - to also store the flow and return it.

```
int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[cur][next]);
                if (next == t)
                    return new_flow;
                q.push({next, new_flow});
            }
        }
    }

    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }
}
```

```

    }

    return flow;
}

```

Integral flow theorem

The theorem says, that if every capacity in the network is an integer, then the size of the maximum flow is an integer, and there is a maximum flow such that the flow in each edge is an integer as well. In particular, Ford-Fulkerson method finds such a flow.

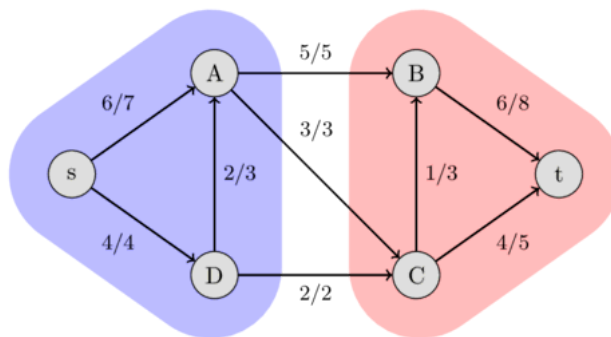
Max-flow min-cut theorem

A **s-t-cut** is a partition of the vertices of a flow network into two sets, such that a set includes the source s and the other one includes the sink t . The capacity of a s-t-cut is defined as the sum of capacities of the edges from the source side to the sink side.

Obviously, we cannot send more flow from s to t than the capacity of any s-t-cut. Therefore, the maximum flow is bounded by the minimum cut capacity.

The max-flow min-cut theorem goes even further. It says that the capacity of the maximum flow has to be equal to the capacity of the minimum cut.

In the following image, you can see the minimum cut of the flow network we used earlier. It shows that the capacity of the cut $\{s, A, D\}$ and $\{B, C, t\}$ is $5 + 3 + 2 = 10$, which is equal to the maximum flow that we found. Other cuts will have a bigger capacity, like the capacity between $\{s, A\}$ and $\{B, C, D, t\}$ is $4 + 3 + 5 = 12$.



A minimum cut can be found after performing a maximum flow computation using the Ford-Fulkerson method. One possible minimum cut is the following: the set of all the vertices that can be reached from s in the residual graph (using edges with positive residual capacity), and the set of all the other vertices. This partition can be easily found using DFS starting at s .

Practice Problems

- [Codeforces - Array and Operations](#)
- [Codeforces - Red-Blue Graph](#)
- [CSES - Download Speed](#)
- [CSES - Police Chase](#)
- [CSES - School Dance](#)
- [CSES - Distinct Routes](#)

Contributors:

[Jakob Kogler](#) (65.68%)
 [Michael Hayter](#) (11.86%)
 [Mohammad Humayun Khan](#) (11.44%)
 [Oleksandr Kulkov](#) (2.96%)
 [infixint943](#) (1.69%)
 [Aryaman](#) (1.27%)
[LUTLJS](#) (1.27%)
 [Bergur Snorrason](#) (0.85%)
 [Kaustubh Dwivedi](#) (0.42%)
 [Obiwac](#) (0.42%)
 [Piyush Naithani](#) (0.42%)
 [ShayekhBinIslam](#) (0.42%)
[Shiven Sinha](#) (0.42%)
 [Tarun Khullar](#) (0.42%)
 [wiktorkuchta](#) (0.42%)

Translated From: e-maxx.ru

Maximum flow - Push-relabel algorithm

The push-relabel algorithm (or also known as preflow-push algorithm) is an algorithm for computing the maximum flow of a flow network. The exact definition of the problem that we want to solve can be found in the article [Maximum flow - Ford-Fulkerson and Edmonds-Karp](#).

In this article we will consider solving the problem by pushing a preflow through the network, which will run in $O(V^4)$, or more precisely in $O(V^2E)$, time. The algorithm was designed by Andrew Goldberg and Robert Tarjan in 1985.

Definitions

During the algorithm we will have to handle a **preflow** - i.e. a function f that is similar to the flow function, but does not necessarily satisfies the flow conservation constraint. For it only the constraints

$$0 \leq f(e) \leq c(e)$$

and

$$\sum_{(v,u) \in E} f((v,u)) \geq \sum_{(u,v) \in E} f((u,v))$$

have to hold.

So it is possible for some vertex to receive more flow than it distributes. We say that this vertex has some excess flow, and define the amount of it with the **excess** function $x(u) = \sum_{(v,u) \in E} f((v,u)) - \sum_{(u,v) \in E} f((u,v))$.

In the same way as with the flow function, we can define the residual capacities and the residual graph with the preflow function.

The algorithm will start off with an initial preflow (some vertices having excess), and during the execution the preflow will be handled and modified. Giving away some details already, the algorithm will pick a vertex with excess, and push the excess to neighboring vertices. It will repeat this until all vertices, except the source and the sink, are free from excess. It is easy to see, that a preflow without excess is a valid flow. This makes the algorithm terminate with an actual flow.

There are still two problem, we have to deal with. First, how do we guarantee that this actually terminates? And secondly, how do we guarantee that this will actually give us a maximum flow, and not just any random flow?

To solve these problems we need the help of another function, namely the **labeling** functions h , often also called **height** function, which assigns each vertex an integer. We call a labeling is valid, if $h(s) = |V|$, $h(t) = 0$, and $h(u) \leq h(v) + 1$ if there is an edge (u, v) in the residual graph - i.e. the edge (u, v) has a positive capacity in the residual graph. In other words, if it is possible to increase the flow from u to v , then the height of v can be at most one smaller than the height of u , but it can be equal or even higher.

It is important to note, that if there exists a valid labeling function, then there doesn't exist an augmenting path from s to t in the residual graph. Because such a path will have a length of at most $|V| - 1$ edges, and each edge can decrease the height only by at most by one, which is impossible if the first height is $h(s) = |V|$ and the last height is $h(t) = 0$.

Using this labeling function we can state the strategy of the push-relabel algorithm: We start with a valid preflow and a valid labeling function. In each step we push some excess between vertices, and update the labels of vertices. We have to make sure, that after each step the preflow and the labeling are still valid. If then the algorithm determines, the preflow is a valid flow. And because we also have a valid labeling, there doesn't exists a path between s and t in the residual graph, which means that the flow is actually a maximum flow.

If we compare the Ford-Fulkerson algorithm with the push-relabel algorithm it seems like the algorithms are the duals of each other. The Ford-Fulkerson algorithm keeps a valid flow at all time and improves it until there doesn't exists an augmenting path any more, while in the push-relabel algorithm there doesn't exists an augmenting path at any time, and we will improve the preflow until it is a valid flow.

Algorithm

First we have to initialize the graph with a valid preflow and labeling function.

Using the empty preflow - like it is done in the Ford-Fulkerson algorithm - is not possible, because then there will be an augmenting path and this implies that there doesn't exist a valid labeling. Therefore we will initialize each edge outgoing from s with its maximal capacity:

$f((s, u)) = c((s, u))$. And all other edges with zero. In this case there exists a valid labeling, namely $h(s) = |V|$ for the source vertex and $h(u) = 0$ for all other.

Now let's describe the two operations in more detail.

With the `push` operation we try to push as much excess flow from one vertex u to a neighboring vertex v . We have one rule: we are only allowed to push flow from u to v if $h(u) = h(v) + 1$. In layman's terms, the excess flow has to flow downwards, but not too steeply. Of course we only can push $\min(x(u), c((u, v)) - f((u, v)))$ flow.

If a vertex has excess, but it is not possible to push the excess to any adjacent vertex, then we need to increase the height of this vertex. We call this operation `relabel`. We will increase it by as much as it is possible, while still maintaining validity of the labeling.

To recap, the algorithm in a nutshell is: We initialize a valid preflow and a valid labeling. While we can perform push or relabel operations, we perform them. Afterwards the preflow is actually a flow and we return it.

Complexity

It is easy to show, that the maximal label of a vertex is $2|V| - 1$. At this point all remaining excess can and will be pushed back to the source. This gives at most $O(V^2)$ relabel operations.

It can also be showed, that there will be at most $O(VE)$ saturating pushes (a push where the total capacity of the edge is used) and at most $O(V^2E)$ non-saturating pushes (a push where the capacity of an edge is not fully used) performed. If we pick a data structure that allows us to find the next vertex with excess in $O(1)$ time, then the total complexity of the algorithm is $O(V^2E)$.

Implementation

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess, seen;
queue<int> excess_vertices;

void push(int u, int v) {
    int d = min(excess[u], capacity[u][v] - flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
    if (d && excess[v] == d)
        excess_vertices.push(v);
}

void relabel(int u) {
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

void discharge(int u) {
    while (excess[u] > 0) {
        if (seen[u] < n) {
            int v = seen[u];
            if (capacity[u][v] - flow[u][v] > 0 && height[u] > height[v])
                push(u, v);
            else
                seen[u]++;
        } else {
```

```

        relabel(u);
        seen[u] = 0;
    }
}

int max_flow(int s, int t) {
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
    excess[s] = inf;
    for (int i = 0; i < n; i++) {
        if (i != s)
            push(s, i);
    }
    seen.assign(n, 0);

    while (!excess_vertices.empty()) {
        int u = excess_vertices.front();
        excess_vertices.pop();
        if (u != s && u != t)
            discharge(u);
    }

    int max_flow = 0;
    for (int i = 0; i < n; i++)
        max_flow += flow[i][t];
    return max_flow;
}

```

Here we use the queue `excess_vertices` to store all vertices that currently have excess. In that way we can pick the next vertex for a push or a relabel operation in constant time.

And to make sure that we don't spend too much time finding the adjacent vertex to whom we can push, we use a data structure called **current-arc**. Basically we will iterate over the edges in a circular order and always store the last edge that we used. This way, for a certain labeling value, we will switch the current edge only $O(n)$ time. And since the relabeling already takes $O(n)$ time, we don't make the complexity worse.

Contributors:

[Jakob Kogler](#) (88.3%) [arjan-bal](#) (4.68%) [Oleksandr Kulkov](#) (3.51%) [Mark Moretto](#) (2.34%) [wiktorkuchta](#) (1.17%)

Translated From: e-maxx.ru

Maximum flow - Push-relabel method improved

We will modify the [push-relabel method](#) to achieve a better runtime.

Description

The modification is extremely simple: In the previous article we chosen a vertex with excess without any particular rule. But it turns out, that if we always choose the vertices with the **greatest height**, and apply push and relabel operations on them, then the complexity will become better. Moreover, to select the vertices with the greatest height we actually don't need any data structures, we simply store the vertices with the greatest height in a list, and recalculate the list once all of them are processed (then vertices with already lower height will be added to the list), or whenever a new vertex with excess and a greater height appears (after relabeling a vertex).

Despite the simplicity, this modification reduces the complexity by a lot. To be precise, the complexity of the resulting algorithm is $O(VE + V^2\sqrt{E})$, which in the worst case is $O(V^3)$.

This modification was proposed by Cheriyan and Maheshwari in 1989.

Implementation

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess;

void push(int u, int v)
{
    int d = min(excess[u], capacity[u][v] - flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
}

void relabel(int u)
{
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

vector<int> find_max_height_vertices(int s, int t) {
    vector<int> max_height;
    for (int i = 0; i < n; i++) {
        if (i != s && i != t && excess[i] > 0) {
            if (!max_height.empty() && height[i] > height[max_height[0]])
                max_height.clear();
            if (max_height.empty() || height[i] == height[max_height[0]])
                max_height.push_back(i);
        }
    }
    return max_height;
}

int max_flow(int s, int t)
{
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
}
```

```
excess[s] = inf;
for (int i = 0; i < n; i++) {
    if (i != s)
        push(s, i);
}

vector<int> current;
while (!(current = find_max_height_vertices(s, t)).empty()) {
    for (int i : current) {
        bool pushed = false;
        for (int j = 0; j < n && excess[i]; j++) {
            if (capacity[i][j] - flow[i][j] > 0 && height[i] == height[j] + 1) {
                push(i, j);
                pushed = true;
            }
        }
        if (!pushed) {
            relabel(i);
            break;
        }
    }
}

return excess[t];
}
```

Contributors:

[Jakob Kogler](#) (92.71%) [Oleksandr Kulkov](#) (6.25%) [algorithm-apprentice](#) (1.04%)

Translated

From: e-maxx.ru

Maximum flow - Dinic's algorithm

Dinic's algorithm solves the maximum flow problem in $O(V^2E)$. The maximum flow problem is defined in this article [Maximum flow - Ford-Fulkerson and Edmonds-Karp](#). This algorithm was discovered by Yefim Dinitz in 1970.

Definitions

A **residual network** G^R of network G is a network which contains two edges for each edge $(v, u) \in G$:

- (v, u) with capacity $c_{vu}^R = c_{vu} - f_{vu}$
- (u, v) with capacity $c_{uv}^R = f_{vu}$

A **blocking flow** of some network is such a flow that every path from s to t contains at least one edge which is saturated by this flow. Note that a blocking flow is not necessarily maximal.

A **layered network** of a network G is a network built in the following way. Firstly, for each vertex v we calculate $level[v]$ - the shortest path (unweighted) from s to this vertex using only edges with positive capacity. Then we keep only those edges (v, u) for which $level[v] + 1 = level[u]$. Obviously, this network is acyclic.

Algorithm

The algorithm consists of several phases. On each phase we construct the layered network of the residual network of G . Then we find an arbitrary blocking flow in the layered network and add it to the current flow.

Proof of correctness

Let's show that if the algorithm terminates, it finds the maximum flow.

If the algorithm terminated, it couldn't find a blocking flow in the layered network. It means that the layered network doesn't have any path from s to t . It means that the residual network doesn't have any path from s to t . It means that the flow is maximum.

Number of phases

The algorithm terminates in less than V phases. To prove this, we must firstly prove two lemmas.

Lemma 1. The distances from s to each vertex don't decrease after each iteration, i. e. $level_{i+1}[v] \geq level_i[v]$.

Proof. Fix a phase i and a vertex v . Consider any shortest path P from s to v in G_{i+1}^R . The length of P equals $level_{i+1}[v]$. Note that G_{i+1}^R can only contain edges from G_i^R and back edges for edges from G_i^R . If P has no back edges for G_i^R , then $level_{i+1}[v] \geq level_i[v]$ because P is also a path in G_i^R . Now, suppose that P has at least one back edge. Let the first such edge be (u, w) . Then $level_{i+1}[u] \geq level_i[u]$ (because of the first case). The edge (u, w) doesn't belong to G_i^R , so the edge (w, u) was affected by the blocking flow on the previous iteration. It means that $level_i[u] = level_i[w] + 1$. Also, $level_{i+1}[w] = level_{i+1}[u] + 1$. From these two equations and $level_{i+1}[u] \geq level_i[u]$ we obtain $level_{i+1}[w] \geq level_i[w] + 2$. Now we can use the same idea for the rest of the path.

Lemma 2. $level_{i+1}[t] > level_i[t]$

Proof. From the previous lemma, $level_{i+1}[t] \geq level_i[t]$. Suppose that $level_{i+1}[t] = level_i[t]$. Note that G_{i+1}^R can only contain edges from G_i^R and back edges for edges from G_i^R . It means that there is a shortest path in G_i^R which wasn't blocked by the blocking flow. It's a contradiction.

From these two lemmas we conclude that there are less than V phases because $level[t]$ increases, but it can't be greater than $V - 1$.

Finding blocking flow

In order to find the blocking flow on each iteration, we may simply try pushing flow with DFS from s to t in the layered network while it can be pushed. In order to do it more quickly, we must remove the edges which can't be used to push anymore. To do this we can keep a pointer in each vertex which points to the next edge which can be used.

A single DFS run takes $O(k + V)$ time, where k is the number of pointer advances on this run. Summed up over all runs, number of pointer advances can not exceed E . On the other hand, total number of runs won't exceed E , as every run saturates at least one edge. In this way, total running time of finding a blocking flow is $O(VE)$.

Complexity

There are less than V phases, so the total complexity is $O(V^2E)$.

Unit networks

A **unit network** is a network in which for any vertex except s and t **either incoming or outgoing edge is unique and has unit capacity**. That's exactly the case with the network we build to solve the maximum matching problem with flows.

On unit networks Dinic's algorithm works in $O(E\sqrt{V})$. Let's prove this.

Firstly, each phase now works in $O(E)$ because each edge will be considered at most once.

Secondly, suppose there have already been $\sqrt{V}$ phases. Then all the augmenting paths with the length $\leq \sqrt{V}$ have been found. Let f be the current flow, f' be the maximum flow. Consider their difference $f' - f$. It is a flow in G^R of value $|f'| - |f|$ and on each edge it is either 0 or 1. It can be decomposed into $|f'| - |f|$ paths from s to t and possibly cycles. As the network is unit, they can't have common vertices, so the total number of vertices is $\geq (|f'| - |f|)\sqrt{V}$, but it is also $\leq V$, so in another $\sqrt{V}$ iterations we will definitely find the maximum flow.

Unit capacities networks

In a more generic settings when all edges have unit capacities, *but the number of incoming and outgoing edges is unbounded*, the paths can't have common edges rather than common vertices. In a similar way it allows to prove the bound of $\sqrt{E}$ on the number of iterations, hence the running time of Dinic algorithm on such networks is at most $O(E\sqrt{E})$.

Finally, it is also possible to prove that the number of phases on unit capacity networks doesn't exceed $O(V^{2/3})$, providing an alternative estimate of $O(EV^{2/3})$ on the networks with particularly large number of edges.

Implementation

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
```

```

edges.emplace_back(u, v, 0);
adj[v].push_back(m);
adj[u].push_back(m + 1);
m += 2;
}

bool bfs() {
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int id : adj[v]) {
            if (edges[id].cap == edges[id].flow)
                continue;
            if (level[edges[id].u] != -1)
                continue;
            level[edges[id].u] = level[v] + 1;
            q.push(edges[id].u);
        }
    }
    return level[t] != -1;
}

long long dfs(int v, long long pushed) {
    if (pushed == 0)
        return 0;
    if (v == t)
        return pushed;
    for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
        int id = adj[v][cid];
        int u = edges[id].u;
        if (level[v] + 1 != level[u])
            continue;
        long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
        if (tr == 0)
            continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}

long long flow() {
    long long f = 0;
    while (true) {
        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        q.push(s);
        if (!bfs())
            break;
        fill(ptr.begin(), ptr.end(), 0);
        while (long long pushed = dfs(s, flow_inf)) {
            f += pushed;
        }
    }
    return f;
}
};

```

Practice Problems

- [SPOJ: FASTFLOW](#)

Contributors:

[Yury Semenov](#) (83.75%)
 [Oleksandr Kulkov](#) (9.38%)
 [Danilo Pereira da Silva](#) (3.75%)
 [Jakob Kogler](#) (1.25%)
 [Naman Bhalla](#) (1.25%)
 [Ching Lam Lau](#) (0.62%)

Original

Maximum flow - MPM algorithm

MPM (Malhotra, Pramodh-Kumar and Maheshwari) algorithm solves the maximum flow problem in $O(V^3)$. This algorithm is similar to [Dinic's algorithm](#).

Algorithm

Like Dinic's algorithm, MPM runs in phases, during each phase we find the blocking flow in the layered network of the residual network of G . The main difference from Dinic's is how we find the blocking flow. Consider the layered network L . For each node we define its *inner potential* and *outer potential* as:

$$p_{in}(v) = \sum_{(u,v) \in L} (c(u,v) - f(u,v))$$

$$p_{out}(v) = \sum_{(v,u) \in L} (c(v,u) - f(v,u))$$

Also we set $p_{in}(s) = p_{out}(t) = \infty$. Given p_{in} and p_{out} we define the *potential* as $p(v) = \min(p_{in}(v), p_{out}(v))$. We call a node r a *reference node* if $p(r) = \min\{p(v)\}$. Consider a reference node r . We claim that the flow can be increased by $p(r)$ in such a way that $p(r)$ becomes 0. It is true because L is acyclic, so we can push the flow out of r by outgoing edges and it will reach t because each node has enough outer potential to push the flow out when it reaches it. Similarly, we can pull the flow from s . The construction of the blocked flow is based on this fact. On each iteration we find a reference node and push the flow from s to t through r . This process can be simulated by BFS. All completely saturated arcs can be deleted from L as they won't be used later in this phase anyway. Likewise, all the nodes different from s and t without outgoing or incoming arcs can be deleted.

Each phase works in $O(V^2)$ because there are at most V iterations (because at least the chosen reference node is deleted), and on each iteration we delete all the edges we passed through except at most V . Summing, we get $O(V^2 + E) = O(V^2)$. Since there are less than V phases (see the proof [here](#)), MPM works in $O(V^3)$ total.

Implementation

```
struct MPM{
    struct FlowEdge{
        int v, u;
        long long cap, flow;
        FlowEdge(){}
        FlowEdge(int _v, int _u, long long _cap, long long _flow)
            : v(_v), u(_u), cap(_cap), flow(_flow){}
        FlowEdge(int _v, int _u, long long _cap)
            : v(_v), u(_u), cap(_cap), flow(0){}
    };
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<char> alive;
    vector<long long> pin, pout;
    vector<list<int>> in, out;
    vector<vector<int>> adj;
    vector<long long> ex;
    int n, m = 0;
    int s, t;
    vector<int> level;
    vector<int> q;
    int qh, qt;
    void resize(int _n){
```

```

    n = _n;
    ex.resize(n);
    q.resize(n);
    pin.resize(n);
    pout.resize(n);
    adj.resize(n);
    level.resize(n);
    in.resize(n);
    out.resize(n);
}

MPM(){}

MPM(int _n, int _s, int _t){resize(_n); s = _s; t = _t;}

void add_edge(int v, int u, long long cap){
    edges.push_back(FlowEdge(v, u, cap));
    edges.push_back(FlowEdge(u, v, 0));
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
}

bool bfs(){
    while(qh < qt){
        int v = q[qh++];
        for(int id : adj[v]){
            if(edges[id].cap - edges[id].flow < 1)continue;
            if(level[edges[id].u] != -1)continue;
            level[edges[id].u] = level[v] + 1;
            q[qt++] = edges[id].u;
        }
    }
    return level[t] != -1;
}

long long pot(int v){
    return min(pin[v], pout[v]);
}

void remove_node(int v){
    for(int i : in[v]){
        int u = edges[i].v;
        auto it = find(out[u].begin(), out[u].end(), i);
        out[u].erase(it);
        pout[u] -= edges[i].cap - edges[i].flow;
    }
    for(int i : out[v]){
        int u = edges[i].u;
        auto it = find(in[u].begin(), in[u].end(), i);
        in[u].erase(it);
        pin[u] -= edges[i].cap - edges[i].flow;
    }
}

void push(int from, int to, long long f, bool forw){
    qh = qt = 0;
    ex.assign(n, 0);
    ex[from] = f;
    q[qt++] = from;
    while(qh < qt){
        int v = q[qh++];
        if(v == to)
            break;
        long long must = ex[v];
        auto it = forw ? out[v].begin() : in[v].begin();
        while(true){
            int u = forw ? edges[*it].u : edges[*it].v;
            long long pushed = min(must, edges[*it].cap - edges[*it].flow);
            if(pushed == 0)break;
            if(forw){
                pout[v] -= pushed;
                pin[u] -= pushed;
            }
            else{
                pin[v] -= pushed;
                pout[u] -= pushed;
            }
            if(ex[u] == 0)
                q[qt++] = u;
            ex[u] += pushed;
            edges[*it].flow += pushed;
            edges[(it)^1].flow -= pushed;
        }
    }
}

```

```

        must -= pushed;
        if(edges[*it].cap - edges[*it].flow == 0){
            auto jt = it;
            ++jt;
            if(forw){
                in[u].erase(find(in[u].begin(), in[u].end(), *it));
                out[v].erase(it);
            }
            else{
                out[u].erase(find(out[u].begin(), out[u].end(), *it));
                in[v].erase(it);
            }
            it = jt;
        }
        else break;
        if(!must)break;
    }
}
}
}
long long flow(){
    long long ans = 0;
    while(true){
        pin.assign(n, 0);
        pout.assign(n, 0);
        level.assign(n, -1);
        alive.assign(n, true);
        level[s] = 0;
        qh = 0; qt = 1;
        q[0] = s;
        if(!bfs())
            break;
        for(int i = 0; i < n; i++){
            out[i].clear();
            in[i].clear();
        }
        for(int i = 0; i < m; i++){
            if(edges[i].cap - edges[i].flow == 0)
                continue;
            int v = edges[i].v, u = edges[i].u;
            if(level[v] + 1 == level[u] && (level[u] < level[t] || u == t)){
                in[u].push_back(i);
                out[v].push_back(i);
                pin[u] += edges[i].cap - edges[i].flow;
                pout[v] += edges[i].cap - edges[i].flow;
            }
        }
        pin[s] = pout[t] = flow_inf;
        while(true){
            int v = -1;
            for(int i = 0; i < n; i++){
                if(!alive[i])continue;
                if(v == -1 || pot(i) < pot(v))
                    v = i;
            }
            if(v == -1)
                break;
            if(pot(v) == 0){
                alive[v] = false;
                remove_node(v);
                continue;
            }
            long long f = pot(v);
            ans += f;
            push(v, s, f, false);
            push(v, t, f, true);
            alive[v] = false;
            remove_node(v);
        }
    }
    return ans;
}
};

```

Contributors:

Yury Semenov (94.84%) Oleksandr Kulkov (2.35%) Jakob Kogler (1.41%) Anton Älgmyr (0.94%) Bergur Snorrason (0.47%)

Translated

From: e-maxx.ru

Flows with demands

In a normal flow network the flow of an edge is only limited by the capacity $c(e)$ from above and by 0 from below. In this article we will discuss flow networks, where we additionally require the flow of each edge to have a certain amount, i.e. we bound the flow from below by a **demand** function $d(e)$:

$$d(e) \leq f(e) \leq c(e)$$

So next each edge has a minimal flow value, that we have to pass along the edge.

This is a generalization of the normal flow problem, since setting $d(e) = 0$ for all edges e gives a normal flow network. Notice, that in the normal flow network it is extremely trivial to find a valid flow, just setting $f(e) = 0$ is already a valid one. However if the flow of each edge has to satisfy a demand, than suddenly finding a valid flow is already pretty complicated.

We will consider two problems:

1. finding an arbitrary flow that satisfies all constraints
2. finding a minimal flow that satisfies all constraints

Finding an arbitrary flow

We make the following changes in the network. We add a new source s' and a new sink t' , a new edge from the source s' to every other vertex, a new edge for every vertex to the sink t' , and one edge from t' to s . Additionally we define the new capacity function c' as:

- $c'((s', v)) = \sum_{u \in V} d((u, v))$ for each edge (s', v) .
- $c'((v, t')) = \sum_{w \in V} d((v, w))$ for each edge (v, t') .
- $c'((u, v)) = c((u, v)) - d((u, v))$ for each edge (u, v) in the old network.
- $c'((t', s)) = \infty$

If the new network has a saturating flow (a flow where each edge outgoing from s' is completely filled, which is equivalent to every edge incoming to t' is completely filled), then the network with demands has a valid flow, and the actual flow can be easily reconstructed from the new network. Otherwise there doesn't exist a flow that satisfies all conditions. Since a saturating flow has to be a maximum flow, it can be found by any maximum flow algorithm, like the [Edmonds-Karp algorithm](#) or the [Push-relabel algorithm](#).

The correctness of these transformations is more difficult to understand. We can think of it in the following way: Each edge $e = (u, v)$ with $d(e) > 0$ is originally replaced by two edges: one with the capacity $d(e)$, and the other with $c(e) - d(e)$. We want to find a flow that saturates the first edge (i.e. the flow along this edge must be equal to its capacity). The second edge is less important - the flow along it can be anything, assuming that it doesn't exceed its capacity. Consider each edge that has to be saturated, and we perform the following operation: we draw the edge from the new source s' to its end v , draw the edge from its start u to the new sink t' , remove the edge itself, and from the old sink t to the old source s we draw an edge of infinite capacity. By these actions we simulate the fact that this edge is saturated - from v there will be an additionally $d(e)$ flow outgoing (we simulate it with a new source that feeds the right amount of flow to v), and u will also push $d(e)$ additional flow (but instead along the old edge, this flow will go directly to the new sink t'). A flow with the value $d(e)$, that originally flowed along the path $s - \dots - u - v - \dots - t$ can now take the new path $s' - v - \dots - t' - s - \dots - u - t'$. The only thing that got simplified in the definition of the new network, is that if procedure created multiple edges between the same pair of vertices, then they are combined to one single edge with the summed capacity.

Minimal flow

Note that along the edge (t, s) (from the old sink to the old source) with the capacity ∞ flows the entire flow of the corresponding old network. I.e. the capacity of this edge effects the flow value of the old network. By giving this edge a sufficient large capacity (i.e. ∞), the flow of the old network is unlimited. By limiting this edge by smaller capacities, the flow value will decrease. However if we limit this edge by a too small value,

than the network will not have a saturated solution, e.g. the corresponding solution for the original network will not satisfy the demand of the edges. Obviously here can use a binary search to find the lowest value with which all constraints are still satisfied. This gives the minimal flow of the original network.

Contributors:

[Jakob Kogler](#) (89.83%) [Oleksandr Kulkov](#) (10.17%)

Translated From: e-maxx.ru

Minimum-cost flow - Successive shortest path algorithm

Given a network G consisting of n vertices and m edges. For each edge (generally speaking, oriented edges, but see below), the capacity (a non-negative integer) and the cost per unit of flow along this edge (some integer) are given. Also the source s and the sink t are marked.

For a given value K , we have to find a flow of this quantity, and among all flows of this quantity we have to choose the flow with the lowest cost. This task is called **minimum-cost flow problem**.

Sometimes the task is given a little differently: you want to find the maximum flow, and among all maximal flows we want to find the one with the least cost. This is called the **minimum-cost maximum-flow problem**.

Both these problems can be solved effectively with the algorithm of successive shortest paths.

Algorithm

This algorithm is very similar to the [Edmonds-Karp](#) for computing the maximum flow.

Simplest case

First we only consider the simplest case, where the graph is oriented, and there is at most one edge between any pair of vertices (e.g. if (i, j) is an edge in the graph, then (j, i) cannot be part in it as well).

Let U_{ij} be the capacity of an edge (i, j) if this edge exists. And let C_{ij} be the cost per unit of flow along this edge (i, j) . And finally let $F_{i,j}$ be the flow along the edge (i, j) . Initially all flow values are zero.

We **modify** the network as follows: for each edge (i, j) we add the **reverse edge** (j, i) to the network with the capacity $U_{ji} = 0$ and the cost $C_{ji} = -C_{ij}$. Since, according to our restrictions, the edge (j, i) was not in the network before, we still have a network that is not a multigraph (graph with multiple edges). In addition we will always keep the condition $F_{ji} = -F_{ij}$ true during the steps of the algorithm.

We define the **residual network** for some fixed flow F as follow (just like in the Ford-Fulkerson algorithm): the residual network contains only unsaturated edges (i.e. edges in which $F_{ij} < U_{ij}$), and the residual capacity of each such edge is $R_{ij} = U_{ij} - F_{ij}$.

Now we can talk about the **algorithms** to compute the minimum-cost flow. At each iteration of the algorithm we find the shortest path in the residual graph from s to t . In contrast to Edmonds-Karp, we look for the shortest path in terms of the cost of the path instead of the number of edges. If there doesn't exist a path anymore, then the algorithm terminates, and the stream F is the desired one. If a path was found, we increase the flow along it as much as possible (i.e. we find the minimal residual capacity R of the path, and increase the flow by it, and reduce the back edges by the same amount). If at some point the flow reaches the value K , then we stop the algorithm (note that in the last iteration of the algorithm it is necessary to increase the flow by only such an amount so that the final flow value doesn't surpass K).

It is not difficult to see, that if we set K to infinity, then the algorithm will find the minimum-cost maximum-flow. So both variations of the problem can be solved by the same algorithm.

Undirected graphs / multigraphs

The case of an undirected graph or a multigraph doesn't differ conceptually from the algorithm above. The algorithm will also work on these graphs. However it becomes a little more difficult to implement it.

An **undirected edge** (i, j) is actually the same as two oriented edges (i, j) and (j, i) with the same capacity and values. Since the above-described minimum-cost flow algorithm generates a back edge for each directed edge, so it splits the undirected edge into 4 directed edges, and we actually get a **multigraph**.

How do we deal with **multiple edges**? First the flow for each of the multiple edges must be kept separately. Secondly, when searching for the shortest path, it is necessary to take into account that it is important which of the multiple edges is used in the path. Thus instead of the usual ancestor array we additionally must store the edge number from which we came from along with the ancestor. Thirdly, as the flow increases

along a certain edge, it is necessary to reduce the flow along the back edge. Since we have multiple edges, we have to store the edge number for the reversed edge for each edge.

There are no other obstructions with undirected graphs or multigraphs.

Complexity

The algorithm here is generally exponential in the size of the input. To be more specific, in the worst case it may push only as much as 1 unit of flow on each iteration, taking $O(F)$ iterations to find a minimum-cost flow of size F , making a total runtime to be $O(F \cdot T)$, where T is the time required to find the shortest path from source to sink.

If [Bellman-Ford](#) algorithm is used for this, it makes the running time $O(F \cdot mn)$. It is also possible to modify [Dijkstra's algorithm](#), so that it needs $O(nm)$ pre-processing as an initial step and then works in $O(m \log n)$ per iteration, making the overall running time to be $O(mn + F \cdot m \log n)$. [Here](#) is a generator of a graph, on which such algorithm would require $O(2^{n/2} n^2 \log n)$ time.

The modified Dijkstra's algorithm uses so-called potentials from [Johnson's algorithm](#). It is possible to combine the ideas of this algorithm and Dinic's algorithm to reduce the number of iterations from F to $\min(F, nC)$, where C is the maximum cost found among edges. You may read further about potentials and their combination with Dinic algorithm [here](#).

Implementation

Here is an implementation using the [SPFA algorithm](#) for the simplest case.

```
struct Edge
{
    int from, to, capacity, cost;
};

vector<vector<int>> adj, cost, capacity;

const int INF = 1e9;

void shortest_paths(int n, int v0, vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(v0);
    p.assign(n, -1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]) {
                d[v] = d[u] + cost[u][v];
                p[v] = u;
                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
}

int min_cost_flow(int N, vector<Edge> edges, int K, int s, int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N, 0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.capacity;
    }

    int flow = 0;
    int cost = 0;
```

```
vector<int> d, p;
while (flow < K) {
    shortest_paths(N, s, d, p);
    if (d[t] == INF)
        break;

    // find max flow on that path
    int f = K - flow;
    int cur = t;
    while (cur != s) {
        f = min(f, capacity[p[cur]][cur]);
        cur = p[cur];
    }

    // apply flow
    flow += f;
    cost += f * d[t];
    cur = t;
    while (cur != s) {
        capacity[p[cur]][cur] -= f;
        capacity[cur][p[cur]] += f;
        cur = p[cur];
    }
}

if (flow < K)
    return -1;
else
    return cost;
}
```

Practice Problems

- [CSES - Task Assignment](#)
- [CSES - Grid Puzzle II](#)
- [AtCoder - Dream Team](#)

Contributors:

[Jakob Kogler](#) (85.12%) [Oleksandr Kulkov](#) (5.96%) [prpr](#) (4.76%) [ffelix-huang](#) (2.98%) [Le Bao Hiep](#) (0.6%) [Quang Mai](#) (0.6%)

Translated From: e-maxx.ru

Solving assignment problem using min-cost-flow

The **assignment problem** has two equivalent statements:

- Given a square matrix $A[1..N, 1..N]$, you need to select N elements in it so that exactly one element is selected in each row and column, and the sum of the values of these elements is the smallest.
- There are N orders and N machines. The cost of manufacturing on each machine is known for each order. Only one order can be performed on each machine. It is required to assign all orders to the machines so that the total cost is minimized.

Here we will consider the solution of the problem based on the algorithm for finding the [minimum cost flow \(min-cost-flow\)](#), solving the assignment problem in $O(N^3)$.

Description

Let's build a bipartite network: there is a source S , a drain T , in the first part there are N vertices (corresponding to rows of the matrix, or orders), in the second there are also N vertices (corresponding to the columns of the matrix, or machines). Between each vertex i of the first set and each vertex j of the second set, we draw an edge with bandwidth 1 and cost A_{ij} . From the source S we draw edges to all vertices i of the first set with bandwidth 1 and cost 0. We draw an edge with bandwidth 1 and cost 0 from each vertex of the second set j to the drain T .

We find in the resulting network the maximum flow of the minimum cost. Obviously, the value of the flow will be N . Further, for each vertex i of the first segment there is exactly one vertex j of the second segment, such that the flow $F_{ij} = 1$. Finally, this is a one-to-one correspondence between the vertices of the first segment and the vertices of the second part, which is the solution to the problem (since the found flow has a minimal cost, then the sum of the costs of the selected edges will be the lowest possible, which is the optimality criterion).

The complexity of this solution of the assignment problem depends on the algorithm by which the search for the maximum flow of the minimum cost is performed. The complexity will be $O(N^3)$ using [Dijkstra](#) or $O(N^4)$ using [Bellman-Ford](#). This is due to the fact that the flow is of size $O(N)$ and each iteration of Dijkstra algorithm can be performed in $O(N^2)$, while it is $O(N^3)$ for Bellman-Ford.

Implementation

The implementation given here is long, it can probably be significantly reduced. It uses the [SPFA algorithm](#) for finding shortest paths.

```
const int INF = 1000 * 1000 * 1000;

vector<int> assignment(vector<vector<int>> a) {
    int n = a.size();
    int m = n * 2 + 2;
    vector<vector<int>> f(m, vector<int>(m));
    int s = m - 2, t = m - 1;
    int cost = 0;
    while (true) {
        vector<int> dist(m, INF);
        vector<int> p(m);
        vector<bool> inq(m, false);
        queue<int> q;
        dist[s] = 0;
        p[s] = -1;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            inq[v] = false;
            if (v == s) {
                for (int i = 0; i < n; ++i) {
                    if (f[s][i] == 0) {
                        dist[i] = 0;
                        p[i] = s;
                    }
                }
            }
        }
    }
}
```

```

        inq[i] = true;
        q.push(i);
    }
}
} else {
    if (v < n) {
        for (int j = n; j < n + n; ++j) {
            if (f[v][j] < 1 && dist[j] > dist[v] + a[v][j - n]) {
                dist[j] = dist[v] + a[v][j - n];
                p[j] = v;
                if (!inq[j]) {
                    q.push(j);
                    inq[j] = true;
                }
            }
        }
    }
} else {
    for (int j = 0; j < n; ++j) {
        if (f[v][j] < 0 && dist[j] > dist[v] - a[j][v - n]) {
            dist[j] = dist[v] - a[j][v - n];
            p[j] = v;
            if (!inq[j]) {
                q.push(j);
                inq[j] = true;
            }
        }
    }
}
}
}

int curcost = INF;
for (int i = n; i < n + n; ++i) {
    if (f[i][t] == 0 && dist[i] < curcost) {
        curcost = dist[i];
        p[t] = i;
    }
}
if (curcost == INF)
    break;
cost += curcost;
for (int cur = t; cur != -1; cur = p[cur]) {
    int prev = p[cur];
    if (prev != -1)
        f[cur][prev] = -(f[prev][cur] == 1);
}
}

vector<int> answer(n);
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        if (f[i][j + n] == 1)
            answer[i] = j;
    }
}
return answer;
}

```

Contributors:

[Daili01](#) (78.57%)
 [prpr](#) (12.5%)
 [Oleksandr Kulkov](#) (7.14%)
 [Hasan-Mesbaul-Ali-Taher](#) (0.89%)
 [Jakob Kogler](#) (0.89%)

Translated

From: e-maxx.ru

Check whether a graph is bipartite

A bipartite graph is a graph whose vertices can be divided into two disjoint sets so that every edge connects two vertices from different sets (i.e. there are no edges which connect vertices from the same set). These sets are usually called sides.

You are given an undirected graph. Check whether it is bipartite, and if it is, output its sides.

Algorithm

There exists a theorem which claims that a graph is bipartite if and only if all its cycles have even length. However, in practice it's more convenient to use a different formulation of the definition: a graph is bipartite if and only if it is two-colorable.

Let's use a series of [breadth-first searches](#), starting from each vertex which hasn't been visited yet. In each search, assign the vertex from which we start to side 1. Each time we visit a yet unvisited neighbor of a vertex assigned to one side, we assign it to the other side. When we try to go to a neighbor of a vertex assigned to one side which has already been visited, we check that it has been assigned to the other side; if it has been assigned to the same side, we conclude that the graph is not bipartite. Once we've visited all vertices and successfully assigned them to sides, we know that the graph is bipartite and we have constructed its partitioning.

Implementation

```
int n;
vector<vector<int>> adj;

vector<int> side(n, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        q.push(st);
        side[st] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    side[u] = side[v] ^ 1;
                    q.push(u);
                } else {
                    is_bipartite &= side[u] != side[v];
                }
            }
        }
    }
}

cout << (is_bipartite ? "YES" : "NO") << endl;
```

Practice problems:

- [SPOJ - BUGLIFE](#)
- [Codeforces - Graph Without Long Directed Paths](#)
- [Codeforces - String Coloring \(easy version\)](#)
- [CSES : Building Teams](#)

Contributors:

Jakob Kogler (49.09%) Mariia Mykhailova (32.73%) Oleksandr Kulkov (10.91%) Aman Kumar Singh (1.82%) Aryaman (1.82%) gampu (1.82%)
pasthec (1.82%)

Translated From: e-maxx.ru

Kuhn's Algorithm for Maximum Bipartite Matching

Problem

You are given a bipartite graph G containing n vertices and m edges. Find the maximum matching, i.e., select as many edges as possible so that no selected edge shares a vertex with any other selected edge.

Algorithm Description

Required Definitions

- A **matching** M is a set of pairwise non-adjacent edges of a graph (in other words, no more than one edge from the set should be incident to any vertex of the graph M). The **cardinality** of a matching is the number of edges in it. All those vertices that have an adjacent edge from the matching (i.e., which have degree exactly one in the subgraph formed by M) are called **saturated** by this matching.
- A **maximal matching** is a matching M of a graph G that is not a subset of any other matching.
- A **maximum matching** (also known as maximum-cardinality matching) is a matching that contains the largest possible number of edges. Every maximum matching is a maximal matching.
- A **path** of length k here means a *simple* path (i.e. not containing repeated vertices or edges) containing k edges, unless specified otherwise.
- An **alternating path** (in a bipartite graph, with respect to some matching) is a path in which the edges alternately belong / do not belong to the matching.
- An **augmenting path** (in a bipartite graph, with respect to some matching) is an alternating path whose initial and final vertices are unsaturated, i.e., they do not belong in the matching.
- The **symmetric difference** (also known as the **disjunctive union**) of sets A and B , represented by $A \oplus B$, is the set of all elements that belong to exactly one of A or B , but not to both. That is, $A \oplus B = (A - B) \cup (B - A) = (A \cup B) - (A \cap B)$.

Berge's lemma

This lemma was proven by the French mathematician **Claude Berge** in 1957, although it already was observed by the Danish mathematician **Julius Petersen** in 1891 and the Hungarian mathematician **Denés König** in 1931.

Formulation

A matching M is maximum $\Leftrightarrow$ there is no augmenting path relative to the matching M .

Proof

Both sides of the bi-implication will be proven by contradiction.

1. A matching M is maximum $\Rightarrow$ there is no augmenting path relative to the matching M .

Let there be an augmenting path P relative to the given maximum matching M . This augmenting path P will necessarily be of odd length, having one more edge not in M than the number of edges it has that are also in M . We create a new matching M' by including all edges in the original matching M except those also in the P , and the edges in P that are not in M . This is a valid matching because the initial and final vertices of P are unsaturated by M , and the rest of the vertices are saturated only by the matching $P \cap M$. This new matching M' will have one more edge than M , and so M could not have been maximum.

Formally, given an augmenting path P w.r.t. some maximum matching M , the matching $M' = P \oplus M$ is such that $|M'| = |M| + 1$, a contradiction.

2. A matching M is maximum $\Leftarrow$ there is no augmenting path relative to the matching M .

Let there be a matching M' of greater cardinality than M . We consider the symmetric difference $Q = M \oplus M'$. The subgraph Q is no longer necessarily a matching. Any vertex in Q has a maximum degree of 2, which means that all connected components in it are one of the three -

- an isolated vertex
- a (simple) path whose edges are alternately from M and M'
- a cycle of even length whose edges are alternately from M and M'

Since M' has a cardinality greater than M , Q has more edges from M' than M . By the Pigeonhole principle, at least one connected component will be a path having more edges from M' than M . Because any such path is alternating, it will have initial and final vertices unsaturated by M , making it an augmenting path for M , which contradicts the premise. ■

Kuhn's algorithm

Kuhn's algorithm is a direct application of Berge's lemma. It is essentially described as follows:

First, we take an empty matching. Then, while the algorithm is able to find an augmenting path, we update the matching by alternating it along this path and repeat the process of finding the augmenting path. As soon as it is not possible to find such a path, we stop the process - the current matching is the maximum.

It remains to detail the way to find augmenting paths. Kuhn's algorithm simply searches for any of these paths using [depth-first](#) or [breadth-first](#) traversal. The algorithm looks through all the vertices of the graph in turn, starting each traversal from it, trying to find an augmenting path starting at this vertex.

The algorithm is more convenient to describe if we assume that the input graph is already split into two parts (although, in fact, the algorithm can be implemented in such a way that the input graph is not explicitly split into two parts).

The algorithm looks at all the vertices v of the first part of the graph: $v = 1 \dots n_1$. If the current vertex v is already saturated with the current matching (i.e., some edge adjacent to it has already been selected), then skip this vertex. Otherwise, the algorithm tries to saturate this vertex, for which it starts a search for an augmenting path starting from this vertex.

The search for an augmenting path is carried out using a special depth-first or breadth-first traversal (usually depth-first traversal is used for ease of implementation). Initially, the depth-first traversal is at the current unsaturated vertex v of the first part. Let's look through all edges from this vertex. Let the current edge be an edge (v, to) . If the vertex to is not yet saturated with matching, then we have succeeded in finding an augmenting path: it consists of a single edge (v, to) ; in this case, we simply include this edge in the matching and stop searching for the augmenting path from the vertex v . Otherwise, if to is already saturated with some edge (to, p) , then will go along this edge: thus we will try to find an augmenting path passing through the edges (v, to) , (to, p) , $\dots$. To do this, simply go to the vertex p in our traversal - now we try to find an augmenting path from this vertex.

So, this traversal, launched from the vertex v , will either find an augmenting path, and thereby saturate the vertex v , or it will not find such an augmenting path (and, therefore, this vertex v cannot be saturated).

After all the vertices $v = 1 \dots n_1$ have been scanned, the current matching will be maximum.

Running time

Kuhn's algorithm can be thought of as a series of n depth/breadth-first traversal runs on the entire graph. Therefore, the whole algorithm is executed in time $O(nm)$, which in the worst case is $O(n^3)$.

However, this estimate can be improved slightly. It turns out that for Kuhn's algorithm, it is important which part of the graph is chosen as the first and which as the second. Indeed, in the implementation described above, the depth/breadth-first traversal starts only from the vertices of the first part, so the entire algorithm is executed in time $O(n_1 m)$, where n_1 is the number of vertices of the first part. In the worst case, this is $O(n_1^2 n_2)$ (where n_2 is the number of vertices of the second part). This shows that it is more profitable when the first part contains fewer vertices than the second. On very unbalanced graphs (when n_1 and n_2 are very different), this translates into a significant difference in runtimes.

Implementation

Standard implementation

Let us present here an implementation of the above algorithm based on depth-first traversal and accepting a bipartite graph in the form of a graph explicitly split into two parts. This implementation is very concise, and perhaps it should be remembered in this form.

Here n is the number of vertices in the first part, k - in the second part, $g[v]$ is the list of edges from the top of the first part (i.e. the list of numbers of the vertices to which these edges lead from v). The vertices in both parts are numbered independently, i.e. vertices in the first part are numbered $1 \dots n$, and those in the second are numbered $1 \dots k$.

Then there are two auxiliary arrays: `mt` and `used`. The first - `mt` - contains information about the current matching. For convenience of programming, this information is contained only for the vertices of the second part: `mt[i]` - this is the number of the vertex of the first part connected by an edge with the vertex i of the second part (or -1 , if no matching edge comes out of it). The second array is `used`: the usual array of "visits" to the vertices in the depth-first traversal (it is needed just so that the depth-first traversal does not enter the same vertex twice).

A function `try_kuhn` is a depth-first traversal. It returns `true` if it was able to find an augmenting path from the vertex v , and it is considered that this function has already performed the alternation of matching along the found chain.

Inside the function, all the edges outgoing from the vertex v of the first part are scanned, and then the following is checked: if this edge leads to an unsaturated vertex to , or if this vertex to is saturated, but it is possible to find an increasing chain by recursively starting from `mt[to]`, then we say that we have found an augmenting path, and before returning from the function with the result `true`, we alternate the current edge: we redirect the edge adjacent to to to the vertex v .

The main program first indicates that the current matching is empty (the list `mt` is filled with numbers -1). Then the vertex v of the first part is searched by `try_kuhn`, and a depth-first traversal is started from it, having previously zeroed the array `used`.

It is worth noting that the size of the matching is easy to get as the number of calls `try_kuhn` in the main program that returned the result `true`. The desired maximum matching itself is contained in the array `mt`.

```
int n, k;
vector<vector<int>> g;
vector<int> mt;
vector<bool> used;

bool try_kuhn(int v) {
    if (used[v])
        return false;
    used[v] = true;
    for (int to : g[v]) {
        if (mt[to] == -1 || try_kuhn(mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int main() {
    //... reading the graph ...

    mt.assign(k, -1);
    for (int v = 0; v < n; ++v) {
        used.assign(n, false);
        try_kuhn(v);
    }

    for (int i = 0; i < k; ++i)
        if (mt[i] != -1)
            printf("%d %d\n", mt[i] + 1, i + 1);
}
```

We repeat once again that Kuhn's algorithm is easy to implement in such a way that it works on graphs that are known to be bipartite, but their explicit splitting into two parts has not been given. In this case, it will be necessary to abandon the convenient division into two parts, and store all the information for all vertices of the graph. For this, an array of lists `g` is now specified not only for the vertices of the first part, but for all the vertices of the graph (of course, now the vertices of both parts are numbered in a common numbering - from 1 to n). Arrays `mt` and `used` are now also defined for the vertices of both parts, and, accordingly, they need to be kept in this state.

Improved implementation

Let us modify the algorithm as follows. Before the main loop of the algorithm, we will find an **arbitrary matching** by some simple algorithm (a simple **heuristic algorithm**), and only then we will execute a loop with calls to the `try_kuhn()` function, which will improve this matching. As a

result, the algorithm will work noticeably faster on random graphs - because in most graphs, you can easily find a matching of a sufficiently large size using heuristics, and then improve the found matching to the maximum using the usual Kuhn's algorithm. Thus, we will save on launching a depth-first traversal from those vertices that we have already included using the heuristic into the current matching.

For example, you can simply iterate over all the vertices of the first part, and for each of them, find an arbitrary edge that can be added to the matching, and add it. Even such a simple heuristic can speed up Kuhn's algorithm several times.

Please note that the main loop will have to be slightly modified. Since when calling the function `try_kuhn` in the main loop, it is assumed that the current vertex is not yet included in the matching, you need to add an appropriate check.

In the implementation, only the code in the `main()` function will change:

```
int main() {
    // ... reading the graph ...

    mt.assign(k, -1);
    vector<bool> used1(n, false);
    for (int v = 0; v < n; ++v) {
        for (int to : g[v]) {
            if (mt[to] == -1) {
                mt[to] = v;
                used1[v] = true;
                break;
            }
        }
    }
    for (int v = 0; v < n; ++v) {
        if (used1[v])
            continue;
        used.assign(n, false);
        try_kuhn(v);
    }

    for (int i = 0; i < k; ++i)
        if (mt[i] != -1)
            printf("%d %d\n", mt[i] + 1, i + 1);
}
```

Another good heuristic is as follows. At each step, it will search for the vertex of the smallest degree (but not isolated), select any edge from it and add it to the matching, then remove both these vertices with all incident edges from the graph. Such greed works very well on random graphs; in many cases it even builds the maximum matching (although there is a test case against it, on which it will find a matching that is much smaller than the maximum).

Notes

- Kuhn's algorithm is a subroutine in the **Hungarian algorithm**, also known as the **Kuhn-Munkres algorithm**.
- Kuhn's algorithm runs in $O(nm)$ time. It is generally simple to implement, however, more efficient algorithms exist for the maximum bipartite matching problem - such as the **Hopcroft-Karp-Karzanov algorithm**, which runs in $O(\sqrt{nm})$ time.
- The **minimum vertex cover problem** is NP-hard for general graphs. However, **König's theorem** gives that, for bipartite graphs, the cardinality of the maximum matching equals the cardinality of the minimum vertex cover. Hence, we can use maximum bipartite matching algorithms to solve the minimum vertex cover problem in polynomial time for bipartite graphs.

Practice Problems

- [Kattis - Gopher II](#)
- [Kattis - Borders](#)

Contributors:

[ameya-dubey](#) (88.41%) [Oleksandr Kulkov](#) (5.15%) [Jakob Kogler](#) (3.43%) [matbensh](#) (2.58%) [Eric Pei](#) (0.43%)

Translated From: e-maxx.ru

Hungarian algorithm for solving the assignment problem

Statement of the assignment problem

There are several standard formulations of the assignment problem (all of which are essentially equivalent). Here are some of them:

- There are n jobs and n workers. Each worker specifies the amount of money they expect for a particular job. Each worker can be assigned to only one job. The objective is to assign jobs to workers in a way that minimizes the total cost.
- Given an $n \times n$ matrix A , the task is to select one number from each row such that exactly one number is chosen from each column, and the sum of the selected numbers is minimized.
- Given an $n \times n$ matrix A , the task is to find a permutation p of length n such that the value $\sum A[i][p[i]]$ is minimized.
- Consider a complete bipartite graph with n vertices per part, where each edge is assigned a weight. The objective is to find a perfect matching with the minimum total weight.

It is important to note that all the above scenarios are "**square**" problems, meaning both dimensions are always equal to n . In practice, similar "**rectangular**" formulations are often encountered, where n is not equal to m , and the task is to select $\min(n, m)$ elements. However, it can be observed that a "rectangular" problem can always be transformed into a "square" problem by adding rows or columns with zero or infinite values, respectively.

We also note that by analogy with the search for a **minimum** solution, one can also pose the problem of finding a **maximum** solution. However, these two problems are equivalent to each other: it is enough to multiply all the weights by -1 .

Hungarian algorithm

Historical reference

The algorithm was developed and published by Harold **Kuhn** in 1955. Kuhn himself gave it the name "Hungarian" because it was based on the earlier work by Hungarian mathematicians Dénes König and Jenő Egerváry.

In 1957, James **Munkres** showed that this algorithm runs in (strictly) polynomial time, independently from the cost.

Therefore, in literature, this algorithm is known not only as the "Hungarian", but also as the "Kuhn-Munkres algorithm" or "Munkres algorithm".

However, it was recently discovered in 2006 that the same algorithm was invented **a century before Kuhn** by the German mathematician Carl Gustav **Jacobi**. His work, *About the research of the order of a system of arbitrary ordinary differential equations*, which was published posthumously in 1890, contained, among other findings, a polynomial algorithm for solving the assignment problem. Unfortunately, since the publication was in Latin, it went unnoticed among mathematicians.

It is also worth noting that Kuhn's original algorithm had an asymptotic complexity of $O(n^4)$, and only later Jack **Edmonds** and Richard **Karp** (and independently **Tomizawa**) showed how to improve it to an asymptotic complexity of $O(n^3)$.

The $O(n^4)$ algorithm

To avoid ambiguity, we note right away that we are mainly concerned with the assignment problem in a matrix formulation (i.e., given a matrix A , you need to select n cells from it that are in different rows and columns). We index arrays starting with 1, i.e., for example, a matrix A has indices $A[1 \dots n][1 \dots n]$.

We will also assume that all numbers in matrix A are **non-negative** (if this is not the case, you can always make the matrix non-negative by adding some constant to all numbers).

Let's call a **potential** two arbitrary arrays of numbers $u[1 \dots n]$ and $v[1 \dots n]$, such that the following condition is satisfied:

$$u[i] + v[j] \leq A[i][j], \quad i = 1 \dots n, j = 1 \dots n$$

(As you can see, $u[i]$ corresponds to the i -th row, and $v[j]$ corresponds to the j -th column of the matrix).

Let's call **the value f of the potential** the sum of its elements:

$$f = \sum_{i=1}^n u[i] + \sum_{j=1}^n v[j].$$

On one hand, it is easy to see that the cost of the desired solution sol is **not less than** the value of any potential.

Lemma. $sol \geq f$.

Proof

The desired solution of the problem consists of n cells of the matrix A , so $u[i] + v[j] \leq A[i][j]$ for each of them. Since all the elements in sol are in different rows and columns, summing these inequalities over all the selected $A[i][j]$, you get f on the left side of the inequality, and sol on the right side.

On the other hand, it turns out that there is always a solution and a potential that turns this inequality into **equality**. The Hungarian algorithm described below will be a constructive proof of this fact. For now, let's just pay attention to the fact that if any solution has a cost equal to any potential, then this solution is **optimal**.

Let's fix some potential. Let's call an edge (i, j) **rigid** if $u[i] + v[j] = A[i][j]$.

Recall an alternative formulation of the assignment problem, using a bipartite graph. Denote with H a bipartite graph composed only of rigid edges. The Hungarian algorithm will maintain, for the current potential, **the maximum-number-of-edges matching M** of the graph H . As soon as M contains n edges, then the solution to the problem will be just M (after all, it will be a solution whose cost coincides with the value of a potential).

Let's proceed directly to **the description of the algorithm**.

Step 1. At the beginning, the potential is assumed to be zero ($u[i] = v[j] = 0$ for all i), and the matching M is assumed to be empty.

Step 2. Further, at each step of the algorithm, we try, without changing the potential, to increase the cardinality of the current matching M by one (recall that the matching is searched in the graph of rigid edges H). To do this, the usual [Kuhn Algorithm for finding the maximum matching in bipartite graphs](#) is used. Let us recall the algorithm here. All edges of the matching M are oriented in the direction from the right part to the left one, and all other edges of the graph H are oriented in the opposite direction.

Recall (from the terminology of searching for matchings) that a vertex is called saturated if an edge of the current matching is adjacent to it. A vertex that is not adjacent to any edge of the current matching is called unsaturated. A path of odd length, in which the first edge does not belong to the matching, and for all subsequent edges there is an alternating belonging to the matching (belongs/does not belong) - is called an augmenting path. From all unsaturated vertices in the left part, a [depth-first](#) or [breadth-first](#) traversal is started. If, as a result of the search, it was possible to reach an unsaturated vertex of the right part, we have found an augmenting path from the left part to the right one. If we include odd edges of the path and remove the even ones in the matching (i.e. include the first edge in the matching, exclude the second, include the third, etc.), then we will increase the matching cardinality by one.

If there was no augmenting path, then the current matching M is maximal in the graph H .

Step 3. If at the current step, it is not possible to increase the cardinality of the current matching, then a recalculation of the potential is performed in such a way that, at the next steps, there will be more opportunities to increase the matching.

Denote by Z_1 the set of vertices of the left part that were visited during the last traversal of Kuhn's algorithm, and through Z_2 the set of visited vertices of the right part.

Let's calculate the value Δ :

$$\Delta = \min_{i \in Z_1, j \notin Z_2} A[i][j] - u[i] - v[j].$$

Lemma. $\Delta > 0$.

Proof

Suppose $\Delta = 0$. Then there exists a rigid edge (i, j) with $i \in Z_1$ and $j \notin Z_2$. It follows that the edge (i, j) must be oriented from the right part to the left one, i.e. (i, j) must be included in the matching M . However, this is impossible, because we could not get to the saturated vertex i except by going along the edge from j to i . So $\Delta > 0$.

Now let's **recalculate the potential** in this way:

- for all vertices $i \in Z_1$, do $u[i] \leftarrow u[i] + \Delta$,
- for all vertices $j \in Z_2$, do $v[j] \leftarrow v[j] - \Delta$.

Lemma. The resulting potential is still a correct potential.

Proof

We will show that, after recalculation, $u[i] + v[j] \leq A[i][j]$ for all i, j . For all the elements of A with $i \in Z_1$ and $j \in Z_2$, the sum $u[i] + v[j]$ does not change, so the inequality remains true. For all the elements with $i \notin Z_1$ and $j \in Z_2$, the sum $u[i] + v[j]$ decreases by Δ , so the inequality is still true. For the other elements whose $i \in Z_1$ and $j \notin Z_2$, the sum increases, but the inequality is still preserved, since the value Δ is, by definition, the maximum increase that does not change the inequality.

Lemma. The old matching M of rigid edges is valid, i.e. all edges of the matching will remain rigid.

Proof

For some rigid edge (i, j) to stop being rigid as a result of a change in potential, it is necessary that equality $u[i] + v[j] = A[i][j]$ turns into inequality $u[i] + v[j] < A[i][j]$. However, this can happen only when $i \notin Z_1$ and $j \in Z_2$. But $i \notin Z_1$ implies that the edge (i, j) could not be a matching edge.

Lemma. After each recalculation of the potential, the number of vertices reachable by the traversal, i.e. $|Z_1| + |Z_2|$, strictly increases.

Proof

First, note that any vertex that was reachable before recalculation, is still reachable. Indeed, if some vertex is reachable, then there is some path from reachable vertices to it, starting from the unsaturated vertex of the left part; since for edges of the form (i, j) , $i \in Z_1$, $j \in Z_2$ the sum $u[i] + v[j]$ does not change, this entire path will be preserved after changing the potential. Secondly, we show that after a recalculation, at least one new vertex will be reachable. This follows from the definition of Δ : the edge (i, j) which Δ refers to will become rigid, so vertex j will be reachable from vertex i .

Due to the last lemma, **no more than n potential recalculations can occur** before an augmenting path is found and the matching cardinality of M is increased. Thus, sooner or later, a potential that corresponds to a perfect matching M^* will be found, and M^* will be the answer to the

problem. If we talk about the complexity of the algorithm, then it is $O(n^4)$: in total there should be at most n increases in matching, before each of which there are no more than n potential recalculations, each of which is performed in time $O(n^2)$.

We will not give the implementation for the $O(n^4)$ algorithm here, since it will turn out to be no shorter than the implementation for the $O(n^3)$ one, described below.

The $O(n^3)$ algorithm

Now let's learn how to implement the same algorithm in $O(n^3)$ (for rectangular problems $n \times m$, $O(n^2m)$).

The key idea is to **consider matrix rows one by one**, and not all at once. Thus, the algorithm described above will take the following form:

1. Consider the next row of the matrix A .
2. While there is no increasing path starting in this row, recalculate the potential.
3. As soon as an augmenting path is found, propagate the matching along it (thus including the last edge in the matching), and restart from step 1 (to consider the next line).

To achieve the required complexity, it is necessary to implement steps 2-3, which are performed for each row of the matrix, in time $O(n^2)$ (for rectangular problems in $O(nm)$).

To do this, recall two facts proved above:

- With a change in the potential, the vertices that were reachable by Kuhn's traversal will remain reachable.
- In total, only $O(n)$ recalculations of the potential could occur before an augmenting path was found.

From this follow these **key ideas** that allow us to achieve the required complexity:

- To check for the presence of an augmenting path, there is no need to start the Kuhn traversal again after each potential recalculation. Instead, you can make the Kuhn traversal in an **iterative form**: after each recalculation of the potential, look at the added rigid edges and, if their left ends were reachable, mark their right ends reachable as well and continue the traversal from them.
- Developing this idea further, we can present the algorithm as follows: at each step of the loop, the potential is recalculated. Subsequently, a column that has become reachable is identified (which will always exist as new reachable vertices emerge after every potential recalculation). If the column is unsaturated, an augmenting chain is discovered. Conversely, if the column is saturated, the matching row also becomes reachable.
- To quickly recalculate the potential (faster than the $O(n^2)$ naive version), you need to maintain auxiliary minima for each of the columns:

$$\min v[j] = \min_{i \in Z_1} A[i][j] - u[i] - v[j].$$

It's easy to see that the desired value Δ is expressed in terms of them as follows:

$$\Delta = \min_{j \notin Z_2} \min v[j].$$

Thus, finding Δ can now be done in $O(n)$.

It is necessary to update the array $\min v$ when new visited rows appear. This can be done in $O(n)$ for the added row (which adds up over all rows to $O(n^2)$). It is also necessary to update the array $\min v$ when recalculating the potential, which is also done in time $O(n)$ ($\min v$ changes only for columns that have not yet been reached: namely, it decreases by Δ).

Thus, the algorithm takes the following form: in the outer loop, we consider matrix rows one by one. Each row is processed in time $O(n^2)$, since only $O(n)$ potential recalculations could occur (each in time $O(n)$), and the array $\min v$ is maintained in time $O(n^2)$; Kuhn's algorithm will work in time $O(n^2)$ (since it is presented in the form of $O(n)$ iterations, each of which visits a new column).

The resulting complexity is $O(n^3)$ or, if the problem is rectangular, $O(n^2m)$.

Implementation of the Hungarian algorithm

The implementation below was developed by **Andrey Lopatin** several years ago. It is distinguished by amazing conciseness: the entire algorithm consists of **30 lines of code**.

The implementation finds a solution for the rectangular matrix $A[1 \dots n][1 \dots m]$, where $n \leq m$. The matrix is 1-based for convenience and code brevity: this implementation introduces a dummy zero row and zero column, which allows us to write many cycles in a general form, without additional checks.

Arrays $u[0 \dots n]$ and $v[0 \dots m]$ store potential. Initially, they are set to zero, which is consistent with a matrix of zero rows (Note that it is unimportant for this implementation whether or not the matrix A contains negative numbers).

The array $p[0 \dots m]$ contains a matching: for each column $j = 1 \dots m$, it stores the number $p[j]$ of the selected row (or 0 if nothing has been selected yet). For the convenience of implementation, $p[0]$ is assumed to be equal to the number of the current row.

The array $\text{minv}[1 \dots m]$ contains, for each column j , the auxiliary minima necessary for a quick recalculation of the potential, as described above.

The array $\text{way}[1 \dots m]$ contains information about where these minimums are reached so that we can later reconstruct the augmenting path. Note that, to reconstruct the path, it is sufficient to store only column values, since the row numbers can be taken from the matching (i.e., from the array p). Thus, $\text{way}[j]$, for each column j , contains the number of the previous column in the path (or 0 if there is none).

The algorithm itself is an outer **loop through the rows of the matrix**, inside which the i -th row of the matrix is considered. The first *do-while* loop runs until a free column j_0 is found. Each iteration of the loop marks visited a new column with the number j_0 (calculated at the last iteration; and initially equal to zero - i.e. we start from a dummy column), as well as a new row i_0 - adjacent to it in the matching (i.e. $p[j_0]$; and initially when $j_0 = 0$ the i -th row is taken). Due to the appearance of a new visited row i_0 , you need to recalculate the array minv and Δ accordingly. If Δ is updated, then the column j_1 becomes the minimum that has been reached (note that with such an implementation Δ could turn out to be equal to zero, which means that the potential cannot be changed at the current step: there is already a new reachable column). After that, the potential and the minv array are recalculated. At the end of the "do-while" loop, we found an augmenting path ending in a column j_0 that can be "unrolled" using the ancestor array way .

The constant INF is "infinity", i.e. some number, obviously greater than all possible numbers in the input matrix A .

```
vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
for (int i=1; i<=n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<int> minv (m+1, INF);
    vector<bool> used (m+1, false);
    do {
        used[j0] = true;
        int i0 = p[j0], delta = INF, j1;
        for (int j=1; j<=m; ++j)
            if (!used[j]) {
                int cur = A[i0][j]-u[i0]-v[j];
                if (cur < minv[j])
                    minv[j] = cur, way[j] = j0;
                if (minv[j] < delta)
                    delta = minv[j], j1 = j;
            }
        for (int j=0; j<=m; ++j)
            if (used[j])
                u[p[j]] += delta, v[j] -= delta;
            else
                minv[j] -= delta;
        j0 = j1;
    } while (p[j0] != 0);
    do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
    }
```

```
    } while (j0);
}
```

To restore the answer in a more familiar form, i.e. finding for each row $i = 1 \dots n$ the number $ans[i]$ of the column selected in it, can be done as follows:

```
vector<int> ans (n+1);
for (int j=1; j<=m; ++j)
    ans[p[j]] = j;
```

The cost of the matching can simply be taken as the potential of the zero column (taken with the opposite sign). Indeed, as you can see from the code, $-v[0]$ contains the sum of all the values of Δ , i.e. total change in potential. Although several values of $u[i]$ and $v[j]$ could change at once, the total change in the potential is exactly equal to Δ , since until there is an augmenting path, the number of reachable rows is exactly one more than the number of the reachable columns (only the current row i does not have a "pair" in the form of a visited column):

```
int cost = -v[0];
```

Connection to the Successive Shortest Path Algorithm

The Hungarian algorithm can be seen as the [Successive Shortest Path Algorithm](#), adapted for the assignment problem. Without going into the details, let's provide an intuition regarding the connection between them.

The Successive Path algorithm uses a modified version of Johnson's algorithm as reweighting technique. This one is divided into four steps:

- Use the [Bellman-Ford](#) algorithm, starting from the sink s and, for each node, find the minimum weight $h(v)$ of a path from s to v .

For every step of the main algorithm:

- Reweight the edges of the original graph in this way: $w(u, v) \leftarrow w(u, v) + h(u) - h(v)$.
- Use [Dijkstra's](#) algorithm to find the shortest-paths subgraph of the original network.
- Update potentials for the next iteration.

Given this description, we can observe that there is a strong analogy between $h(v)$ and potentials: it can be checked that they are equal up to a constant offset. In addition, it can be shown that, after reweighting, the set of all zero-weight edges represents the shortest-path subgraph where the main algorithm tries to increase the flow. This also happens in the Hungarian algorithm: we create a subgraph made of rigid edges (the ones for which the quantity $A[i][j] - u[i] - v[j]$ is zero), and we try to increase the size of the matching.

In step 4, all the $h(v)$ are updated: every time we modify the flow network, we should guarantee that the distances from the source are correct (otherwise, in the next iteration, Dijkstra's algorithm might fail). This sounds like the update performed on the potentials, but in this case, they are not equally incremented.

To deepen the understanding of potentials, refer to this [article](#).

Task examples

Here are a few examples related to the assignment problem, from very trivial to less obvious tasks:

- Given a bipartite graph, it is required to find in it **the maximum matching with the minimum weight** (i.e., first of all, the size of the matching is maximized, and secondly, its cost is minimized).
To solve it, we simply build an assignment problem, putting the number "infinity" in place of the missing edges. After that, we solve the problem with the Hungarian algorithm, and remove edges of infinite weight from the answer (they could enter the answer if the problem does not have a solution in the form of a perfect matching).
- Given a bipartite graph, it is required to find in it **the maximum matching with the maximum weight**.
The solution is again obvious, all weights must be multiplied by minus one.
- The task of **detecting moving objects in images**: two images were taken, as a result of which two sets of coordinates were obtained. It is required to correlate the objects in the first and second images, i.e. determine for each point of the second image, which point of the first image it corresponded to. In this case, it is required to minimize the sum of distances between the compared points (i.e., we are looking for

a solution in which the objects have taken the shortest path in total).

To solve, we simply build and solve an assignment problem, where the weights of the edges are the Euclidean distances between points.

- The task of **detecting moving objects by locators**: there are two locators that can't determine the position of an object in space, but only its direction. Both locators (located at different points) received information in the form of n such directions. It is required to determine the position of objects, i.e. determine the expected positions of objects and their corresponding pairs of directions in such a way that the sum of distances from objects to direction rays is minimized.

Solution: again, we simply build and solve the assignment problem, where the vertices of the left part are the n directions from the first locator, the vertices of the right part are the n directions from the second locator, and the weights of the edges are the distances between the corresponding rays.

- Covering a **directed acyclic graph with paths**: given a directed acyclic graph, it is required to find the smallest number of paths (if equal, with the smallest total weight) so that each vertex of the graph lies in exactly one path.

The solution is to build the corresponding bipartite graph from the given graph and find the maximum matching of the minimum weight in it. See separate article for more details.

- **Tree coloring book**. Given a tree in which each vertex, except for leaves, has exactly $k - 1$ children. It is required to choose for each vertex one of the k colors available so that no two adjacent vertices have the same color. In addition, for each vertex and each color, the cost of painting this vertex with this color is known, and it is required to minimize the total cost.

To solve this problem, we use dynamic programming. Namely, let's learn how to calculate the value $d[v][c]$, where v is the vertex number, c is the color number, and the value $d[v][c]$ itself is the minimum cost needed to color all the vertices in the subtree rooted at v , and the vertex v itself with color c . To calculate such a value $d[v][c]$, it is necessary to distribute the remaining $k - 1$ colors among the children of the vertex v , and for this, it is necessary to build and solve the assignment problem (in which the vertices of the left part are colors, the vertices of the right part are children, and the weights of the edges are the corresponding values of d).

Thus, each value $d[v][c]$ is calculated using the solution of the assignment problem, which ultimately gives the asymptotic $O(nk^4)$.

- If, in the assignment problem, the weights are not on the edges, but on the vertices, and only **on the vertices of the same part**, then it's not necessary to use the Hungarian algorithm: just sort the vertices by weight and run the usual [Kuhn algorithm](#) (for more details, see a [separate article](#)).
- Consider the following **special case**. Let each vertex of the left part be assigned some number $\alpha[i]$, and each vertex of the right part $\beta[j]$. Let the weight of any edge (i, j) be equal to $\alpha[i] \cdot \beta[j]$ (the numbers $\alpha[i]$ and $\beta[j]$ are known). Solve the assignment problem. To solve it without the Hungarian algorithm, we first consider the case when both parts have two vertices. In this case, as you can easily see, it is better to connect the vertices in the reverse order: connect the vertex with the smaller $\alpha[i]$ to the vertex with the larger $\beta[j]$. This rule can be easily generalized to an arbitrary number of vertices: you need to sort the vertices of the first part in increasing order of $\alpha[i]$ values, the second part in decreasing order of $\beta[j]$ values, and connect the vertices in pairs in that order. Thus, we obtain a solution with complexity of $O(n \log n)$.
- **The Problem of Potentials**. Given a matrix $A[1 \dots n][1 \dots m]$, it is required to find two arrays $u[1 \dots n]$ and $v[1 \dots m]$ such that, for any i and j , $u[i] + v[j] \leq a[i][j]$ and the sum of elements of arrays u and v is maximum. Knowing the Hungarian algorithm, the solution to this problem will not be difficult: the Hungarian algorithm just finds such a potential u, v that satisfies the condition of the problem. On the other hand, without knowledge of the Hungarian algorithm, it seems almost impossible to solve such a problem.

Remark

This task is also called the **dual problem** of the assignment problem: minimizing the total cost of the assignment is equivalent to maximizing the sum of the potentials.

Literature

- Ravindra Ahuja, Thomas Magnanti, James Orlin. [Network Flows \[1993\]](#)
- Harold Kuhn. [The Hungarian Method for the Assignment Problem \[1955\]](#)
- James Munkres. [Algorithms for Assignment and Transportation Problems \[1957\]](#)

Practice Problems

- [UVA - Crime Wave - The Sequel](#)
- [UVA - Warehouse](#)
- [SGU - Beloved Sons](#)
- [UVA - The Great Wall Game](#)
- [UVA - Jogging Trails](#)

Contributors:

[Alessandro Minisini](#) (93.29%) [Oleksandr Kulkov](#) (6.71%)

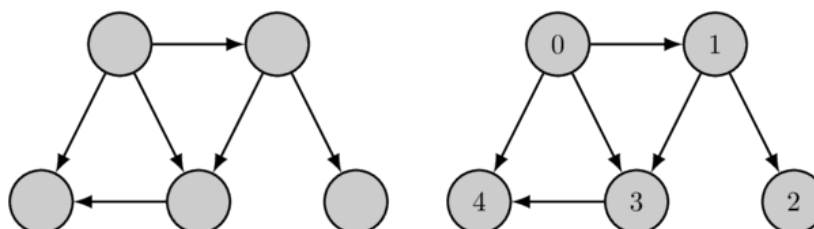
Translated From: e-maxx.ru

Topological Sorting

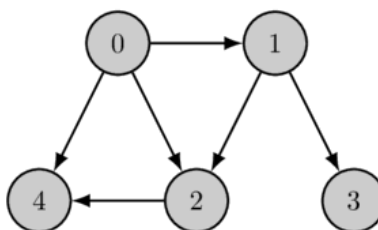
You are given a directed graph with n vertices and m edges. You have to find an **order of the vertices**, so that every edge leads from the vertex with a smaller index to a vertex with a larger one.

In other words, you want to find a permutation of the vertices (**topological order**) which corresponds to the order defined by all edges of the graph.

Here is one given graph together with its topological order:



Topological order can be **non-unique** (for example, if there exist three vertices a , b , c for which there exist paths from a to b and from a to c but not paths from b to c or from c to b). The example graph also has multiple topological orders, a second topological order is the following:



A Topological order may **not exist** at all. It only exists, if the directed graph contains no cycles. Otherwise, there is a contradiction: if there is a cycle containing the vertices a and b , then a needs to have a smaller index than b (since you can reach b from a) and also a bigger one (as you can reach a from b). The algorithm described in this article also shows by construction, that every acyclic directed graph contains at least one topological order.

A common problem in which topological sorting occurs is the following. There are n variables with unknown values. For some variables, we know that one of them is less than the other. You have to check whether these constraints are contradictory, and if not, output the variables in ascending order (if several answers are possible, output any of them). It is easy to notice that this is exactly the problem of finding the topological order of a graph with n vertices.

The Algorithm

To solve this problem, we will use [depth-first search](#).

Let's assume that the graph is acyclic. What does the depth-first search do?

When starting from some vertex v , DFS tries to traverse along all edges outgoing from v . It stops at the edges for which the ends have been already been visited previously, and traverses along the rest of the edges and continues recursively at their ends.

Thus, by the time of the function call $\text{dfs}(v)$ has finished, all vertices that are reachable from v have been either directly (via one edge) or indirectly visited by the search.

Let's append the vertex v to a list, when we finish $\text{dfs}(v)$. Since all reachable vertices have already been visited, they will already be in the list when we append v . Let's do this for every vertex in the graph, with one or multiple depth-first search runs. For every directed edge $v \rightarrow u$ in the graph, u will appear earlier in this list than v , because u is reachable from v . So if we just label the vertices in this list with $n - 1, n - 2, \dots, 1, 0$, we have found a topological order of the graph. In other words, the list represents the reversed topological order.

These explanations can also be presented in terms of exit times of the DFS algorithm. The exit time for vertex v is the time at which the function call $\text{dfs}(v)$ finished (the times can be numbered from 0 to $n - 1$). It is easy to understand that exit time of any vertex v is always greater than the exit time of any vertex reachable from it (since they were visited either before the call $\text{dfs}(v)$ or during it). Thus, the desired topological ordering are the vertices in descending order of their exit times.

Implementation

Here is an implementation which assumes that the graph is acyclic, i.e. the desired topological ordering exists. If necessary, you can easily check that the graph is acyclic, as described in the article on [depth-first search](#).

```
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

The main function of the solution is `topological_sort`, which initializes DFS variables, launches DFS and receives the answer in the vector `ans`. It is worth noting that when the graph is not acyclic, `topological_sort` result would still be somewhat meaningful in a sense that if a vertex u is reachable from vertex v , but not vice versa, the vertex v will always come first in the resulting array. This property of the provided implementation is used in [Kosaraju's algorithm](#) to extract strongly connected components and their topological sorting in a directed graph with cycles.

Practice Problems

- [SPOJ TOPOSORT - Topological Sorting](#) [difficulty: easy]
- [UVA 10305 - Ordering Tasks](#) [difficulty: easy]
- [UVA 124 - Following Orders](#) [difficulty: easy]
- [UVA 200 - Rare Order](#) [difficulty: easy]
- [Codeforces 510C - Fox and Names](#) [difficulty: easy]
- [SPOJ RPLA - Answer the boss!](#)
- [CSES - Course Schedule](#)
- [CSES - Longest Flight Route](#)
- [CSES - Game Routes](#)

Contributors:

Jakob Kogler (33.33%) Vikesh Tiwari (32.32%) Michael Hayter (7.07%) Oleksandr Kulkov (7.07%) Euler (5.05%) Bhuvnesh Jain (3.03%)
 Dequan Kong (3.03%) Angkur Mondal (2.02%) Aryaman (2.02%) Franklin (2.02%) Animesh Kumar (1.01%) Mariia Mykhailova (1.01%)

Pratik Devlekar (1.01%)

Translated

From: e-maxx.ru

From: e-maxx.ru

Edge connectivity / Vertex connectivity

Definition

Given an undirected graph G with n vertices and m edges. Both the edge connectivity and the vertex connectivity are characteristics describing the graph.

Edge connectivity

The **edge connectivity** λ of the graph G is the minimum number of edges that need to be deleted, such that the graph G gets disconnected.

For example an already disconnected graph has an edge connectivity of 0, a connected graph with at least one bridge has an edge connectivity of 1, and a connected graph with no bridges has an edge connectivity of at least 2.

We say that a set S of edges **separates** the vertices s and t , if, after removing all edges in S from the graph G , the vertices s and t end up in different connected components.

It is clear, that the edge connectivity of a graph is equal to the minimum size of such a set separating two vertices s and t , taken among all possible pairs (s, t) .

Vertex connectivity

The **vertex connectivity** κ of the graph G is the minimum number of vertices that need to be deleted, such that the graph G gets disconnected.

For example an already disconnected graph has the vertex connectivity 0, and a connected graph with an articulation point has the vertex connectivity 1. We define that a complete graph has the vertex connectivity $n - 1$. For all other graphs the vertex connectivity doesn't exceed $n - 2$, because you can find a pair of vertices which are not connected by an edge, and remove all other $n - 2$ vertices.

We say that a set T of vertices **separates** the vertices s and t , if, after removing all vertices in T from the graph G , the vertices end up in different connected components.

It is clear, that the vertex connectivity of a graph is equal to the minimal size of such a set separating two vertices s and t , taken among all possible pairs (s, t) .

Properties

The Whitney inequalities

The **Whitney inequalities** (1932) gives a relation between the edge connectivity λ , the vertex connectivity κ , and the minimum degree of any vertex in the graph δ :

$$\kappa \leq \lambda \leq \delta$$

Intuitively if we have a set of edges of size λ , which make the graph disconnected, we can choose one of each end point, and create a set of vertices, that also disconnect the graph. And this set has size $\leq \lambda$.

And if we pick the vertex and the minimal degree δ , and remove all edges connected to it, then we also end up with a disconnected graph. Therefore the second inequality $\lambda \leq \delta$.

It is interesting to note, that the Whitney inequalities cannot be improved: i.e. for any triple of numbers satisfying this inequality there exists at least one corresponding graph. One such graph can be constructed in the following way: The graph will consists of $2(\delta + 1)$ vertices, the first $\delta + 1$ vertices form a clique (all pairs of vertices are connected via an edge), and the second $\delta + 1$ vertices form a second clique. In addition

we connect the two cliques with λ edges, such that it uses λ different vertices in the first clique, and only κ vertices in the second clique. The resulting graph will have the three characteristics.

The Ford-Fulkerson theorem

The **Ford-Fulkerson theorem** implies, that the biggest number of edge-disjoint paths connecting two vertices, is equal to the smallest number of edges separating these vertices.

Computing the values

Edge connectivity using maximum flow

This method is based on the Ford-Fulkerson theorem.

We iterate over all pairs of vertices (s, t) and between each pair we find the largest number of disjoint paths between them. This value can be found using a maximum flow algorithm: we use s as the source, t as the sink, and assign each edge a capacity of 1. Then the maximum flow is the number of disjoint paths.

The complexity for the algorithm using [Edmonds-Karp](#) is $O(V^2VE^2) = O(V^3E^2)$. But we should note, that this includes a hidden factor, since it is practically impossible to create a graph such that the maximum flow algorithm will be slow for all sources and sinks. Especially the algorithm will run pretty fast for random graphs.

Special algorithm for edge connectivity

The task of finding the edge connectivity is equal to the task of finding the **global minimum cut**.

Special algorithms have been developed for this task. One of them is the Stoer-Wagner algorithm, which works in $O(V^3)$ or $O(VE)$ time.

Vertex connectivity

Again we iterate over all pairs of vertices s and t , and for each pair we find the minimum number of vertices that separates s and t .

By doing this, we can apply the same maximum flow approach as described in the previous sections.

We split each vertex x with $x \neq s$ and $x \neq t$ into two vertices x_1 and x_2 . We connect these to vertices with a directed edge (x_1, x_2) with the capacity 1, and replace all edges (u, v) by the two directed edges (u_2, v_1) and (v_2, u_1) , both with the capacity of 1. The by the construction the value of the maximum flow will be equal to the minimum number of vertices that are needed to separate s and t .

This approach has the same complexity as the flow approach for finding the edge connectivity.

Contributors:

[Jakob Kogler](#) (89.47%) [Oleksandr Kulkov](#) (8.42%) [Isaac Bode](#) (2.11%)

Translated

From: e-maxx.ru

Paint the edges of the tree

This is a fairly common task. Given a tree G with N vertices. There are two types of queries: the first one is to paint an edge, the second one is to query the number of colored edges between two vertices.

Here we will describe a fairly simple solution (using a [segment tree](#)) that will answer each query in $O(\log N)$ time. The preprocessing step will take $O(N)$ time.

Algorithm

First, we need to find the [LCA](#) to reduce each query of the second kind (i, j) into two queries (l, i) and (l, j) , where l is the LCA of i and j . The answer of the query (i, j) will be the sum of both subqueries. Both these queries have a special structure, the first vertex is an ancestor of the second one. For the rest of the article we will only talk about these special kind of queries.

We will start by describing the **preprocessing** step. Run a depth-first search from the root of the tree and record the Euler tour of this depth-first search (each vertex is added to the list when the search visits it first and every time we return from one of its children). The same technique can be used in the LCA preprocessing.

This list will contain each edge (in the sense that if i and j are the ends of the edge, then there will be a place in the list where i and j are neighbors in the list), and it appear exactly two times: in the forward direction (from i to j , where vertex i is closer to the root than vertex j) and in the opposite direction (from j to i).

We will build two lists for these edges. The first one will store the color of all edges in the forward direction, and the second one the color of all edges in the opposite direction. We will use 1 if the edge is colored, and 0 otherwise. Over these two lists we will build each a segment tree (for sum with a single modification), let's call them T_1 and T_2 .

Let us answer a query of the form (i, j) , where i is the ancestor of j . We need to determine how many edges are painted on the path between i and j . Let's find i and j in the Euler tour for the first time, let it be the positions p and q (this can be done in $O(1)$ if we calculate these positions in advance during preprocessing). Then the **answer** to the query is the sum $T_1[p..q-1]$ minus the sum $T_2[p..q-1]$.

Why? Consider the segment $[p; q]$ in the Euler tour. It contains all edges of the path we need from i to j but also contains a set of edges that lie on other paths from i . However there is one big difference between the edges we need and the rest of the edges: the edges we need will be listed only once in the forward direction, and all the other edges appear twice: once in the forward and once in the opposite direction. Hence, the difference $T_1[p..q-1] - T_2[p..q-1]$ will give us the correct answer (minus one is necessary because otherwise, we will capture an extra edge going out from vertex j). The sum query in the segment tree is executed in $O(\log N)$.

Answering the **first type of query** (painting an edge) is even easier - we just need to update T_1 and T_2 , namely to perform a single update of the element that corresponds to our edge (finding the edge in the list, again, is possible in $O(1)$, if you perform this search during preprocessing). A single modification in the segment tree is performed in $O(\log N)$.

Implementation

Here is the full implementation of the solution, including LCA computation:

```
const int INF = 1000 * 1000 * 1000;

typedef vector<vector<int>> graph;

vector<int> dfs_list;
vector<int> edges_list;
vector<int> h;
```

```

void dfs(int v, const graph& g, const graph& edge_ids, int cur_h = 1) {
    h[v] = cur_h;
    dfs_list.push_back(v);
    for (size_t i = 0; i < g[v].size(); ++i) {
        if (h[g[v][i]] == -1) {
            edges_list.push_back(edge_ids[v][i]);
            dfs(g[v][i], g, edge_ids, cur_h + 1);
            edges_list.push_back(edge_ids[v][i]);
            dfs_list.push_back(v);
        }
    }
}

vector<int> lca_tree;
vector<int> first;

void lca_tree_build(int i, int l, int r) {
    if (l == r) {
        lca_tree[i] = dfs_list[l];
    } else {
        int m = (l + r) >> 1;
        lca_tree_build(i + i, l, m);
        lca_tree_build(i + i + 1, m + 1, r);
        int lt = lca_tree[i + i], rt = lca_tree[i + i + 1];
        lca_tree[i] = h[lt] < h[rt] ? lt : rt;
    }
}

void lca_prepare(int n) {
    lca_tree.assign(dfs_list.size() * 8, -1);
    lca_tree_build(1, 0, (int)dfs_list.size() - 1);

    first.assign(n, -1);
    for (int i = 0; i < (int)dfs_list.size(); ++i) {
        int v = dfs_list[i];
        if (first[v] == -1)
            first[v] = i;
    }
}

int lca_tree_query(int i, int tl, int tr, int l, int r) {
    if (tl == l && tr == r)
        return lca_tree[i];
    int m = (tl + tr) >> 1;
    if (r <= m)
        return lca_tree_query(i + i, tl, m, l, r);
    if (l > m)
        return lca_tree_query(i + i + 1, m + 1, tr, l, r);
    int lt = lca_tree_query(i + i, tl, m, l, m);
    int rt = lca_tree_query(i + i + 1, m + 1, tr, m + 1, r);
    return h[lt] < h[rt] ? lt : rt;
}

int lca(int a, int b) {
    if (first[a] > first[b])
        swap(a, b);
    return lca_tree_query(1, 0, (int)dfs_list.size() - 1, first[a], first[b]);
}

vector<int> first1, first2;
vector<char> edge_used;
vector<int> tree1, tree2;

void query_prepare(int n) {
    first1.resize(n - 1, -1);
    first2.resize(n - 1, -1);
    for (int i = 0; i < (int)edges_list.size(); ++i) {
        int j = edges_list[i];
        if (first1[j] == -1)
            first1[j] = i;
        else
            first2[j] = i;
    }

    edge_used.resize(n - 1);
    tree1.resize(edges_list.size() * 8);

```

```

    tree2.resize(edges_list.size() * 8);
}

void sum_tree_update(vector<int>& tree, int i, int l, int r, int j, int delta) {
    tree[i] += delta;
    if (l < r) {
        int m = (l + r) >> 1;
        if (j <= m)
            sum_tree_update(tree, i + i, l, m, j, delta);
        else
            sum_tree_update(tree, i + i + 1, m + 1, r, j, delta);
    }
}

int sum_tree_query(const vector<int>& tree, int i, int tl, int tr, int l, int r) {
    if (l > r || tl > tr)
        return 0;
    if (tl == l && tr == r)
        return tree[i];
    int m = (tl + tr) >> 1;
    if (r <= m)
        return sum_tree_query(tree, i + i, tl, m, l, r);
    if (l > m)
        return sum_tree_query(tree, i + i + 1, m + 1, tr, l, r);
    return sum_tree_query(tree, i + i, tl, m, l, m) +
           sum_tree_query(tree, i + i + 1, m + 1, tr, m + 1, r);
}

int query(int v1, int v2) {
    return sum_tree_query(tree1, 1, 0, (int)edges_list.size() - 1, first[v1], first[v2] - 1) -
           sum_tree_query(tree2, 1, 0, (int)edges_list.size() - 1, first[v1], first[v2] - 1);
}

int main() {
    // reading the graph
    int n;
    scanf("%d", &n);
    graph g(n), edge_ids(n);
    for (int i = 0; i < n - 1; ++i) {
        int v1, v2;
        scanf("%d%d", &v1, &v2);
        --v1, --v2;
        g[v1].push_back(v2);
        g[v2].push_back(v1);
        edge_ids[v1].push_back(i);
        edge_ids[v2].push_back(i);
    }

    h.assign(n, -1);
    dfs(0, g, edge_ids);
    lca_prepare(n);
    query_prepare(n);

    for (;;) {
        if () {
            // request for painting edge x;
            // if start = true, then the edge is painted, otherwise the painting
            // is removed
            edge_used[x] = start;
            sum_tree_update(tree1, 1, 0, (int)edges_list.size() - 1, first1[x],
                           start ? 1 : -1);
            sum_tree_update(tree2, 1, 0, (int)edges_list.size() - 1, first2[x],
                           start ? 1 : -1);
        } else {
            // query the number of colored edges on the path between v1 and v2
            int l = lca(v1, v2);
            int result = query(1, v1) + query(1, v2);
            // result - the answer to the request
        }
    }
}

```

Contributors:

[Amandeep Singh](#) (95.17%)
 [Oleksandr Kulkov](#) (2.9%)
 [Jakob Kogler](#) (0.97%)
 [Anton Ālgmyr](#) (0.48%)
 [polyomino](#) (0.48%)

Translated From: e-maxx.ru

2-SAT

SAT (Boolean satisfiability problem) is the problem of assigning Boolean values to variables to satisfy a given Boolean formula. The Boolean formula will usually be given in CNF (conjunctive normal form), which is a conjunction of multiple clauses, where each clause is a disjunction of literals (variables or negation of variables). 2-SAT (2-satisfiability) is a restriction of the SAT problem, in 2-SAT every clause has exactly two literals. Here is an example of such a 2-SAT problem. Find an assignment of a , b , c such that the following formula is true:

$$(a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg c)$$

SAT is NP-complete, there is no known efficient solution for it. However 2SAT can be solved efficiently in $O(n + m)$ where n is the number of variables and m is the number of clauses.

Algorithm:

First we need to convert the problem to a different form, the so-called implicative normal form. Note that the expression $a \vee b$ is equivalent to $\neg a \Rightarrow b \wedge \neg b \Rightarrow a$ (if one of the two variables is false, then the other one must be true).

We now construct a directed graph of these implications: for each variable x there will be two vertices v_x and $v_{\neg x}$. The edges will correspond to the implications.

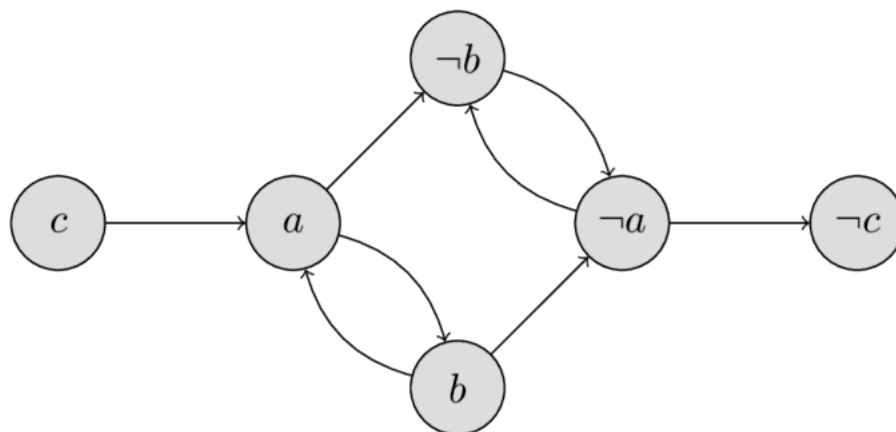
Let's look at the example in 2-CNF form:

$$(a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b) \wedge (a \vee \neg c)$$

The oriented graph will contain the following vertices and edges:

$$\begin{array}{llll} \neg a \Rightarrow \neg b & a \Rightarrow b & a \Rightarrow \neg b & \neg a \Rightarrow \neg c \\ b \Rightarrow a & \neg b \Rightarrow \neg a & b \Rightarrow \neg a & c \Rightarrow a \end{array}$$

You can see the implication graph in the following image:



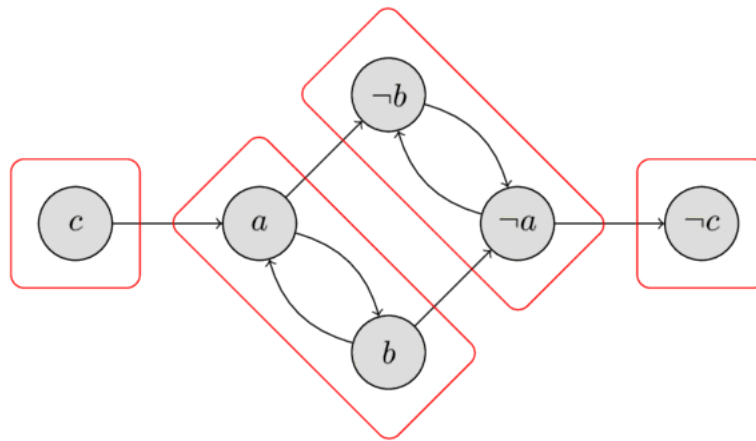
It is worth paying attention to the property of the implication graph: if there is an edge $a \Rightarrow b$, then there also is an edge $\neg b \Rightarrow \neg a$.

Also note, that if x is reachable from $\neg x$, and $\neg x$ is reachable from x , then the problem has no solution. Whatever value we choose for the variable x , it will always end in a contradiction - if x will be assigned true then the implication tell us that $\neg x$ should also be true and visa versa. It turns out, that this condition is not only necessary, but also sufficient. We will prove this in a few paragraphs below. First recall, if a vertex is reachable from a second one, and the second one is reachable from the first one, then these two vertices are in the same strongly connected component. Therefore we can formulate the criterion for the existence of a solution as follows:

In order for this 2-SAT problem to have a solution, it is necessary and sufficient that for any variable x the vertices x and $\neg x$ are in different strongly connected components of the strong connection of the implication graph.

This criterion can be verified in $O(n + m)$ time by finding all strongly connected components.

The following image shows all strongly connected components for the example. As we can check easily, neither of the four components contain a vertex x and its negation $\neg x$, therefore the example has a solution. We will learn in the next paragraphs how to compute a valid assignment, but just for demonstration purposes the solution $a = \text{false}$, $b = \text{false}$, $c = \text{false}$ is given.



Now we construct the algorithm for finding the solution of the 2-SAT problem on the assumption that the solution exists.

Note that, in spite of the fact that the solution exists, it can happen that $\neg x$ is reachable from x in the implication graph, or that (but not simultaneously) x is reachable from $\neg x$. In that case the choice of either true or false for x will lead to a contradiction, while the choice of the other one will not. Let's learn how to choose a value, such that we don't generate a contradiction.

Let us sort the strongly connected components in topological order (i.e. $\text{comp}[v] \leq \text{comp}[u]$ if there is a path from v to u) and let $\text{comp}[v]$ denote the index of strongly connected component to which the vertex v belongs. Then, if $\text{comp}[x] < \text{comp}[\neg x]$ we assign x with false and true otherwise.

Let us prove that with this assignment of the variables we do not arrive at a contradiction. Suppose x is assigned with true. The other case can be proven in a similar way.

First we prove that the vertex x cannot reach the vertex $\neg x$. Because we assigned true it has to hold that the index of strongly connected component of x is greater than the index of the component of $\neg x$. This means that $\neg x$ is located on the left of the component containing x , and the later vertex cannot reach the first.

Secondly we prove that there doesn't exist a variable y , such that the vertices y and $\neg y$ are both reachable from x in the implication graph. This would cause a contradiction, because $x = \text{true}$ implies that $y = \text{true}$ and $\neg y = \text{true}$. Let us prove this by contradiction. Suppose that y and $\neg y$ are both reachable from x , then by the property of the implication graph $\neg x$ is reachable from both y and $\neg y$. By transitivity this results that $\neg x$ is reachable by x , which contradicts the assumption.

So we have constructed an algorithm that finds the required values of variables under the assumption that for any variable x the vertices x and $\neg x$ are in different strongly connected components. Above showed the correctness of this algorithm. Consequently we simultaneously proved the above criterion for the existence of a solution.

Implementation:

Now we can implement the entire algorithm. First we construct the graph of implications and find all strongly connected components. This can be accomplished with Kosaraju's algorithm in $O(n + m)$ time. In the second traversal of the graph Kosaraju's algorithm visits the strongly connected components in topological order, therefore it is easy to compute $\text{comp}[v]$ for each vertex v .

Afterwards we can choose the assignment of x by comparing $\text{comp}[x]$ and $\text{comp}[\neg x]$. If $\text{comp}[x] = \text{comp}[\neg x]$ we return false to indicate that there doesn't exist a valid assignment that satisfies the 2-SAT problem.

Below is the implementation of the solution of the 2-SAT problem for the already constructed graph of implication adj and the transpose graph adj^T (in which the direction of each edge is reversed). In the graph the vertices with indices $2k$ and $2k + 1$ are the two vertices corresponding to variable k with $2k + 1$ corresponding to the negated variable.

```
struct TwoSatSolver {
    int n_vars;
    int n_vertices;
    vector<vector<int>> adj, adj_t;
    vector<bool> used;
    vector<int> order, comp;
    vector<bool> assignment;

    TwoSatSolver(int n_vars) : n_vars(n_vars), n_vertices(2 * n_vars), adj(n_vertices), adj_t(n_vertices),
        used(n_vertices), order(), comp(n_vertices, -1), assignment(n_vars) {
        order.reserve(n_vertices);
    }

    void dfs1(int v) {
        used[v] = true;
        for (int u : adj[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }

    void dfs2(int v, int cl) {
        comp[v] = cl;
        for (int u : adj_t[v]) {
            if (comp[u] == -1)
                dfs2(u, cl);
        }
    }

    bool solve_2SAT() {
        order.clear();
        used.assign(n_vertices, false);
        for (int i = 0; i < n_vertices; ++i) {
            if (!used[i])
                dfs1(i);
        }

        comp.assign(n_vertices, -1);
        for (int i = 0, j = 0; i < n_vertices; ++i) {
            int v = order[n_vertices - i - 1];
            if (comp[v] == -1)
                dfs2(v, j++);
        }

        assignment.assign(n_vars, false);
        for (int i = 0; i < n_vertices; i += 2) {
            if (comp[i] == comp[i + 1])
                return false;
            assignment[i / 2] = comp[i] > comp[i + 1];
        }
        return true;
    }

    void add_disjunction(int a, bool na, int b, bool nb) {
        // na and nb signify whether a and b are to be negated
        a = 2 * a ^ na;
        b = 2 * b ^ nb;
        int neg_a = a ^ 1;
        int neg_b = b ^ 1;
        adj[neg_a].push_back(b);
        adj[neg_b].push_back(a);
        adj_t[b].push_back(neg_a);
        adj_t[a].push_back(neg_b);
    }
}
```

```

static void example_usage() {
    TwoSatSolver solver(3); // a, b, c
    solver.add_disjunction(0, false, 1, true); // a v not b
    solver.add_disjunction(0, true, 1, true); // not a v not b
    solver.add_disjunction(1, false, 2, false); // b v c
    solver.add_disjunction(0, false, 0, false); // a v a
    assert(solver.solve_2SAT() == true);
    auto expected = vector<bool>(True, False, True);
    assert(solver.assignment == expected);
}
};

```

Practice Problems

- [Codeforces: The Door Problem](#)
- [Kattis: Illumination](#)
- [UVA: Rectangles](#)
- [Codeforces : Radio Stations](#)
- [CSES : Giant Pizza](#)
- [Codeforces: +-1](#)

Contributors:

[Jakob Kogler](#) (72.4%)
 [Daniel Paleka](#) (13.02%)
 [Jatin Sharma](#) (3.12%)
 [Michael Hayter](#) (3.12%)
 [Oleksandr Kulkov](#) (3.12%)
 [Vatsal Sharma](#) (1.04%)
[matbensh](#) (1.04%)
[Aryaman](#) (0.52%)
[Benjamin Shi](#) (0.52%)
[Sarthak Vishwakarma](#) (0.52%)
[Tarun Khullar](#) (0.52%)
[sumitrawat10](#) (0.52%)
[wiktorkuchta](#) (0.52%)

Translated

From: e-maxx.ru

Heavy-light decomposition

Heavy-light decomposition is a fairly general technique that allows us to effectively solve many problems that come down to **queries on a tree**.

Description

Let there be a tree G of n vertices, with an arbitrary root.

The essence of this tree decomposition is to **split the tree into several paths** so that we can reach the root vertex from any v by traversing at most $\log n$ paths. In addition, none of these paths should intersect with another.

It is clear that if we find such a decomposition for any tree, it will allow us to reduce certain single queries of the form "calculate something on the path from a to b " to several queries of the type "calculate something on the segment $[l, r]$ of the k^{th} path".

Construction algorithm

We calculate for each vertex v the size of its subtree $s(v)$, i.e. the number of vertices in the subtree of the vertex v including itself.

Next, consider all the edges leading to the children of a vertex v . We call an edge **heavy** if it leads to a vertex c such that:

$$s(c) \geq \frac{s(v)}{2} \iff \text{edge } (v, c) \text{ is heavy}$$

All other edges are labeled **light**.

It is obvious that at most one heavy edge can emanate from one vertex downward, because otherwise the vertex v would have at least two children of size $\geq \frac{s(v)}{2}$, and therefore the size of subtree of v would be too big, $s(v) \geq 1 + 2 \cdot \frac{s(v)}{2} > s(v)$, which leads to a contradiction.

Now we will decompose the tree into disjoint paths. Consider all the vertices from which no heavy edges come down. We will go up from each such vertex until we reach the root of the tree or go through a light edge. As a result, we will get several paths which are made up of zero or more heavy edges plus one light edge. The path which has an end at the root is an exception to this and will not have a light edge. Let these be called **heavy paths** - these are the desired paths of heavy-light decomposition.

Proof of correctness

First, we note that the heavy paths obtained by the algorithm will be **disjoint**. In fact, if two such paths have a common edge, it would imply that there are two heavy edges coming out of one vertex, which is impossible.

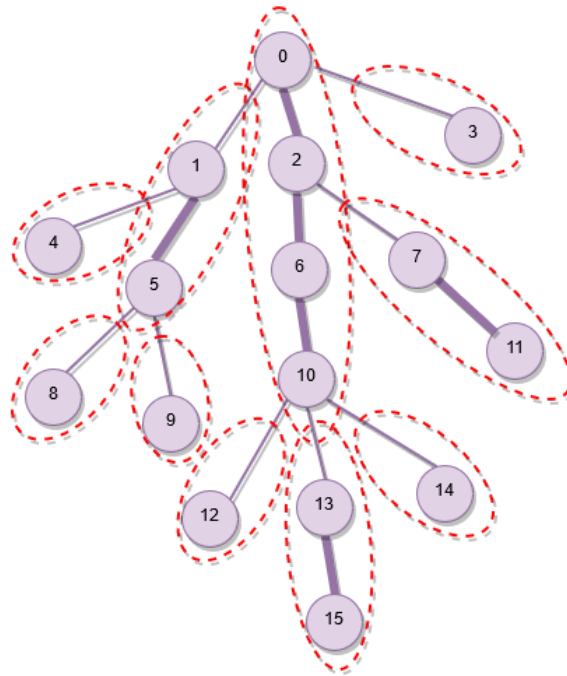
Secondly, we will show that going down from the root of the tree to an arbitrary vertex, we will **change no more than $\log n$ heavy paths along the way**. Moving down a light edge reduces the size of the current subtree to half or lower:

$$s(c) < \frac{s(v)}{2} \iff \text{edge } (v, c) \text{ is light}$$

Thus, we can go through at most $\log n$ light edges before subtree size reduces to one.

Since we can move from one heavy path to another only through a light edge (each heavy path, except the one starting at the root, has one light edge), we cannot change heavy paths more than $\log n$ times along the path from the root to any vertex, as required.

The following image illustrates the decomposition of a sample tree. The heavy edges are thicker than the light edges. The heavy paths are marked by dotted boundaries.



Sample problems

When solving problems, it is sometimes more convenient to consider the heavy-light decomposition as a set of **vertex disjoint** paths (rather than edge disjoint paths). To do this, it suffices to exclude the last edge from each heavy path if it is a light edge, then no properties are violated, but now each vertex belongs to exactly one heavy path.

Below we will look at some typical tasks that can be solved with the help of heavy-light decomposition.

Separately, it is worth paying attention to the problem of the **sum of numbers on the path**, since this is an example of a problem that can be solved by simpler techniques.

Maximum value on the path between two vertices

Given a tree, each vertex is assigned a value. There are queries of the form (a, b) , where a and b are two vertices in the tree, and it is required to find the maximum value on the path between the vertices a and b .

We construct in advance a heavy-light decomposition of the tree. Over each heavy path we will construct a **segment tree**, which will allow us to search for a vertex with the maximum assigned value in the specified segment of the specified heavy path in $O(\log n)$. Although the number of heavy paths in heavy-light decomposition can reach $n - 1$, the total size of all paths is bounded by $O(n)$, therefore the total size of the segment trees will also be linear.

In order to answer a query (a, b) , we find the **lowest common ancestor** of a and b as l , by any preferred method. Now the task has been reduced to two queries (a, l) and (b, l) , for each of which we can do the following: find the heavy path that the lower vertex lies in, make a query on this path, move to the top of this path, again determine which heavy path we are on and make a query on it, and so on, until we get to the path containing l .

One should be careful with the case when, for example, a and l are on the same heavy path - then the maximum query on this path should be done not on any prefix, but on the internal section between a and l .

Responding to the subqueries (a, l) and (b, l) each requires going through $O(\log n)$ heavy paths and for each path a maximum query is made on some section of the path, which again requires $O(\log n)$ operations in the segment tree. Hence, one query (a, b) takes $O(\log^2 n)$ time.

If you additionally calculate and store maximums of all prefixes for each heavy path, then you get a $O(\log n)$ solution because all maximum queries are on prefixes except at most once when we reach the ancestor l .

Sum of the numbers on the path between two vertices

Given a tree, each vertex is assigned a value. There are queries of the form (a, b) , where a and b are two vertices in the tree, and it is required to find the sum of the values on the path between the vertices a and b . A variant of this task is possible where additionally there are update operations that change the number assigned to one or more vertices.

This task can be solved similar to the previous problem of maximums with the help of heavy-light decomposition by building segment trees on heavy paths. Prefix sums can be used instead if there are no updates. However, this problem can be solved by simpler techniques too.

If there are no updates, then it is possible to find out the sum on the path between two vertices in parallel with the LCA search of two vertices by [binary lifting](#) — for this, along with the 2^k -th ancestors of each vertex it is also necessary to store the sum on the paths up to those ancestors during the preprocessing.

There is a fundamentally different approach to this problem - to consider the [Euler tour](#) of the tree, and build a segment tree on it. This algorithm is considered in an [article about a similar problem](#). Again, if there are no updates, storing prefix sums is enough and a segment tree is not required.

Both of these methods provide relatively simple solutions taking $O(\log n)$ for one query.

Repainting the edges of the path between two vertices

Given a tree, each edge is initially painted white. There are updates of the form (a, b, c) , where a and b are two vertices and c is a color, which instructs that all the edges on the path from a to b must be repainted with color c . After all repaintings, it is required to report how many edges of each color were obtained.

Similar to the above problems, the solution is to simply apply heavy-light decomposition and make a [segment tree](#) over each heavy path.

Each repainting on the path (a, b) will turn into two updates (a, l) and (b, l) , where l is the lowest common ancestor of the vertices a and b . $O(\log n)$ per path for $O(\log n)$ paths leads to a complexity of $O(\log^2 n)$ per update.

Implementation

Certain parts of the above discussed approach can be modified to make implementation easier without losing efficiency.

- The definition of **heavy edge** can be changed to **the edge leading to the child with largest subtree**, with ties broken arbitrarily. This may result in some light edges being converted to heavy, which means some heavy paths will combine to form a single path, but all heavy paths will remain disjoint. It is also still guaranteed that going down a light edge reduces subtree size to half or less.
- Instead of building segment tree over every heavy path, a single segment tree can be used with disjoint segments allocated to each heavy path.
- It has been mentioned that answering queries requires calculation of the LCA. While LCA can be calculated separately, it is also possible to integrate LCA calculation in the process of answering queries.

To perform heavy-light decomposition:

```
vector<int> parent, depth, heavy, head, pos;
int cur_pos;

int dfs(int v, vector<vector<int>>> const& adj) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c, adj);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
}
```

```

    }
    return size;
}

void decompose(int v, int h, vector<vector<int>> const& adj) {
    head[v] = h, pos[v] = cur_pos++;
    if (heavy[v] != -1)
        decompose(heavy[v], h, adj);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c, adj);
    }
}

void init(vector<vector<int>> const& adj) {
    int n = adj.size();
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    cur_pos = 0;

    dfs(0, adj);
    decompose(0, 0, adj);
}

```

The adjacency list of the tree must be passed to the `init` function, and decomposition is performed assuming vertex `0` as root.

The `dfs` function is used to calculate `heavy[v]`, the child at the other end of the heavy edge from `v`, for every vertex `v`. Additionally `dfs` also stores the parent and depth of each vertex, which will be useful later during queries.

The `decompose` function assigns for each vertex `v` the values `head[v]` and `pos[v]`, which are respectively the head of the heavy path `v` belongs to and the position of `v` on the single segment tree that covers all vertices.

To answer queries on paths, for example the maximum query discussed, we can do something like this:

```

int query(int a, int b) {
    int res = 0;
    for (; head[a] != head[b]; b = parent[head[b]]) {
        if (depth[head[a]] > depth[head[b]])
            swap(a, b);
        int cur_heavy_path_max = segment_tree_query(pos[head[b]], pos[b]);
        res = max(res, cur_heavy_path_max);
    }
    if (depth[a] > depth[b])
        swap(a, b);
    int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
    res = max(res, last_heavy_path_max);
    return res;
}

```

Practice problems

- [SPOJ - QTREE - Query on a tree](#)
- [CSES - Path Queries II](#)
- [Codeforces - Subway Lines](#)
- [Codeforces - Tree Queries](#)
- [Codeforces - Tree or not Tree](#)
- [Codeforces - The Tree](#)

Contributors:

[Soumik Sarkar](#) (86.98%) [Oleksandr Kulkov](#) (3.12%) [Jakob Kogler](#) (2.6%) [Mirco Paul](#) (2.6%) [Ankit Sangwan](#) (2.08%) [Michael Hayter](#) (1.56%)
[Tanmay Sinha](#) (0.52%) [TheOrnithologistMan](#) (0.52%)